TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN HỌC PHẦN

AN TOÀN VÀ BẢO MẬT HỆ THỐNG THÔNG TIN

ĐỀ TÀI:

ỨNG DỤNG THUẬT TOÁN MÃ HÓA XÁC THỰC CHACHA20-POLY1305 BẢO VỆ DỮ LIỆU CÁ NHÂN TRONG HỆ THỐNG QUẢN LÝ VÀ XÉT DUYỆT VAY TÍN DỤNG TIÊU DÙNG

Giảng viên hướng dẫn: ……

Nhóm sinh viên thực hiện:

1. ……. – MSSV: ……
2. ……. – MSSV: ……
3. ……. – MSSV: ……

Lớp / Nhóm học phần: ……

TP. Hồ Chí Minh, tháng … năm 2026

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn chân thành đến Quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, đã truyền đạt cho nhóm những kiến thức nền tảng về an toàn và bảo mật hệ thống thông tin, phân tích thiết kế hướng đối tượng và công nghệ phần mềm. Đặc biệt, nhóm xin cảm ơn giảng viên hướng dẫn đã định hướng đề tài, góp ý về cách tiếp cận "bảo mật ngay từ thiết kế" (Security by Design) và kiên nhẫn theo dõi tiến độ của nhóm trong suốt học phần.

Đề tài là cơ hội để nhóm đi từ lý thuyết mật mã trên lớp đến một hệ thống chạy được: tự cài đặt lại thuật toán ChaCha20-Poly1305 theo đặc tả RFC 8439, kiểm chứng bằng các vector thử chuẩn, rồi đưa thuật toán vào bảo vệ dữ liệu cá nhân của khách hàng trong một hệ thống xét duyệt vay tín dụng tiêu dùng. Trong quá trình thực hiện, nhóm không tránh khỏi thiếu sót; nhóm rất mong nhận được ý kiến đóng góp của Quý Thầy Cô để báo cáo được hoàn thiện hơn.

Nhóm xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN.....2
MỤC LỤC.....3
DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT.....5
DANH MỤC BẢNG BIỂU VÀ HÌNH ẢNH.....7
CHƯƠNG 1: TỔNG QUAN VÀ GIỚI THIỆU ĐỀ TÀI.....9
1.1. Bối cảnh và tính cấp thiết của đề tài.....9
1.2. Lý do chọn đề tài.....10
1.3. Mục tiêu đề tài.....10
1.3.1. Mục tiêu lý thuyết.....10
1.3.2. Mục tiêu thực nghiệm / ứng dụng.....11
1.4. Đối tượng và phạm vi nghiên cứu.....11
1.5. Cấu trúc của báo cáo.....12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC GIẢI PHÁP AN TOÀN BẢO MẬT...13
2.1. Tổng quan về an toàn bảo mật dữ liệu.....13
2.1.1. Các nguyên tắc cơ bản.....13
2.1.2. Mô hình đe dọa (Threat Model) và nguy cơ tấn công.....14
2.2. Tìm hiểu thuật toán ChaCha20-Poly1305.....15
2.2.1. Giới thiệu và lịch sử phát triển.....15
2.2.2. Các chế độ hoạt động.....16
2.2.3. Nguyên lý và quy trình mã hóa (Encryption).....17
2.2.4. Nguyên lý và quy trình giải mã (Decryption).....19
2.2.5. Đánh giá độ an toàn và hiệu năng.....21
2.3. Các cơ chế hỗ trợ và hạ tầng liên quan.....22
2.3.1. Quản lý và phân phối khóa.....22
2.3.2. Chữ ký số / Mã xác thực thông điệp.....24
2.3.3. Cơ chế kiểm soát truy cập và xác thực người dùng.....26
CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG.....28

3.1. Giới thiệu tổng quan hệ thống.....28
3.2. Phân tích yêu cầu hệ thống.....28
3.2.1. Yêu cầu chức năng (Functional Requirements).....28
3.2.2. Yêu cầu an toàn bảo mật và phi chức năng (Non-functional Requirements) 29
3.3. Thiết kế kiến trúc hệ thống.....30
3.3.1. Mô hình tổng thể.....30
3.3.2. Công nghệ và công cụ sử dụng.....31
3.3.3. Sơ đồ ca sử dụng (Use Case Diagram).....32
3.3.4. Sơ đồ tuần tự xử lý bảo mật (Sequence Diagram).....33
3.4. Hiện thực hóa các cơ chế bảo mật.....37
3.4.1. Quy trình sinh và lưu trữ khóa an toàn.....37
3.4.2. Hiện thực mã hóa dữ liệu tĩnh (Data at Rest) và dữ liệu truyền tải (Data in Transit).....38
3.4.3. Xác thực tính toàn vẹn và chống giả mạo.....41
3.5. Kết quả thực nghiệm và giao diện chương trình.....42
3.5.1. Giao diện chức năng người dùng (User Side).....42
3.5.2. Giao diện quản trị hệ thống (Admin Side).....47
3.5.3. Thử nghiệm mã hóa, giải mã và đo đạc thời gian/hiệu năng.....53
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....61
4.1. Các kết quả đã đạt được.....61
4.2. Hạn chế của đề tài.....61
4.3. Hướng phát triển trong tương lai.....62
TÀI LIỆU THAM KHẢO.....63
PHỤ LỤC.....65
Phụ lục A – Mã nguồn chính.....65
Phụ lục B – Bảng số liệu thử nghiệm.....66
Phụ lục C – Hướng dẫn cài đặt.....67
Phụ lục D – Kết quả chạy script minh họa ChaCha20-Poly1305.....67

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ / Viết tắt	Tiếng Anh	Giải thích
AAD	Additional Authenticated Data	Dữ liệu đi kèm được xác thực nhưng không mã hóa; trong hệ thống là `02 ‖ "bảng.cột:id"`
AEAD	Authenticated Encryption with Associated Data	Mã hóa xác thực có dữ liệu đi kèm: vừa giữ bí mật vừa phát hiện sửa đổi
AES	Advanced Encryption Standard	Chuẩn mã hóa khối của NIST (FIPS 197); định dạng bản mã cũ của hệ thống dùng AES-256-GCM
AES-NI	AES New Instructions	Tập lệnh phần cứng tăng tốc AES trên CPU x86
Argon2id	–	Hàm băm mật khẩu tốn bộ nhớ, thắng cuộc thi Password Hashing Competition 2015 (RFC 9106)
ARX	Add–Rotate–XOR	Lối thiết kế mật mã chỉ dùng phép cộng, xoay bit và XOR
Blind index	–	Mã băm có khóa của một giá trị nhạy cảm để tra cứu chính xác mà không cần giải mã
CCCD	–	Căn cước công dân (12 chữ số)
CIA	Confidentiality, Integrity, Availability	Ba thuộc tính cơ bản: bí mật, toàn vẹn, sẵn sàng
CIC	Credit Information Center	Trung tâm Thông tin tín dụng (hệ thống giả lập trong đồ án)
CSPRNG	Cryptographically Secure PRNG	Bộ sinh số ngẫu nhiên an toàn mật mã (`os.urandom`, `secrets`)
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
HKDF	HMAC-based Key Derivation Function	Hàm dẫn xuất khóa dựa trên HMAC (RFC 5869)
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm (RFC 2104)
IDOR	Insecure Direct Object Reference	Truy cập trực tiếp đối tượng không được phép bằng cách đổi mã trên URL
MAC	Message Authentication Code	Mã xác thực thông điệp
Nonce	Number used once	Giá trị chỉ dùng một lần với một khóa

Thuật ngữ / Viết tắt	Tiếng Anh	Giải thích
PII	Personally Identifiable Information	Dữ liệu cá nhân định danh: CCCD, thu nhập, số tài khoản
Poly1305	–	Mã xác thực một lần dựa trên đa thức trên trường số nguyên modulo $2^{130} - 5$
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
RFC	Request for Comments	Tài liệu đặc tả chuẩn của IETF
SoD	Segregation of Duties	Phân tách nhiệm vụ: một người không làm trọn quy trình
SR	Security Requirement	Yêu cầu bảo mật (SR01–SR14 trong đề cương đồ án)
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege	Mô hình phân loại mối đe dọa của Microsoft
Step-up	Step-up authentication	Xác thực lại trước thao tác nhạy cảm
Tag	Authentication tag	Thẻ xác thực 128 bit do Poly1305 sinh ra
TLS	Transport Layer Security	Giao thức bảo mật tầng truyền tải (TLS 1.3: RFC 8446)
TOTP	Time-based One-Time Password	Mật khẩu một lần theo thời gian (RFC 6238)
Hồ sơ vay	Loan application	Một đề nghị vay của khách hàng (thuật ngữ theo `CONTEXT.md`)
Khoản vay	Loan	Nghĩa vụ trả nợ phát sinh sau khi giải ngân
Snapshot	–	Bản ghi chuẩn hóa các trường quan trọng của hồ sơ vay lúc phê duyệt

DANH MỤC BẢNG BIỂU VÀ HÌNH ẢNH

Danh mục bảng

Số hiệu	Tên bảng	Mục
Bảng 2.1	Ánh xạ nguyên tắc an toàn vào hệ thống	2.1.1
Bảng 2.2	Mô hình đe dọa STRIDE và biện pháp giảm thiểu	2.1.2
Bảng 2.3	Các biến thể và chế độ dùng họ ChaCha	2.2.2
Bảng 2.4	Trạng thái hàm khối ChaCha20 theo ví dụ RFC 8439 mục 2.3.2	2.2.3
Bảng 2.5	So sánh ChaCha20-Poly1305 với AES-256-GCM	2.2.5
Bảng 2.6	Các bí mật và vòng đời khóa	2.3.1
Bảng 2.7	Các cơ chế xác thực thông điệp trong hệ thống	2.3.2
Bảng 3.1	Yêu cầu chức năng	3.2.1
Bảng 3.2	Yêu cầu bảo mật và mức độ đáp ứng	3.2.2
Bảng 3.3	Công nghệ và công cụ	3.3.2
Bảng 3.4	Các trường được mã hóa và ngữ cảnh AAD	3.4.2
Bảng 3.5	Kết quả kiểm chứng vector thử RFC 8439	3.5.3
Bảng 3.6	Kịch bản tấn công giả mạo và kết quả	3.5.3
Bảng 3.7	Thông lượng mã hóa theo kích thước dữ liệu	3.5.3
Bảng 3.8	Thời gian mã hóa, giải mã một trường	3.5.3
Bảng 3.9	Thời gian các cơ chế hỗ trợ	3.5.3
Bảng 4.1	Đối chiếu kết quả với mục tiêu	4.1

Danh mục hình

Số hiệu	Tên hình	Mục
Hình 2.1	Ma trận trạng thái và hàm quarter-round của ChaCha20	2.2.3
Hình 2.2	Quy trình mã hóa ChaCha20-Poly1305	2.2.3
Hình 2.3	Quy trình giải mã: xác thực trước, giải mã sau	2.2.4
Hình 2.4	Các khóa bí mật và khóa dẫn xuất	2.3.1
Hình 2.5	Nhật ký kiểm toán dạng chuỗi băm	2.3.2
Hình 3.1	Kiến trúc triển khai và vị trí các cơ chế bảo mật	3.3.1
Hình 3.2	Sơ đồ ca sử dụng các chức năng liên quan bảo mật	3.3.3

Số hiệu	Tên hình	Mục
Hình 3.3	Sơ đồ tuần tự đăng nhập hai yếu tố	3.3.4
Hình 3.4	Sơ đồ tuần tự đăng ký khóa TOTP	3.3.4
Hình 3.5	Sơ đồ tuần tự mã hóa, giải mã trường nhạy cảm	3.3.4
Hình 3.6	Sơ đồ tuần tự phê duyệt và giải ngân có kiểm tra toàn vẹn	3.3.4
Hình 3.7	Định dạng bản mã lưu trong cơ sở dữ liệu	3.4.2
Hình 3.8 – 3.13	Giao diện phía khách hàng	3.5.1
Hình 3.14 – 3.22	Giao diện phía nhân viên và quản trị	3.5.2
Hình 3.23 – 3.27	Kết quả thử nghiệm tấn công giả mạo	3.5.3
Hình 3.28	Biểu đồ thông lượng mã hóa	3.5.3

CHƯƠNG 1: TỔNG QUAN VÀ GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và tính cấp thiết của đề tài

Thị trường tài chính tiêu dùng tại Việt Nam phát triển nhanh: khách hàng có thể đăng ký vay tiền mặt, vay mua điện máy, điện thoại trả góp ngay trên điện thoại. Để xét duyệt một hồ sơ vay, công ty tài chính phải thu thập những dữ liệu nhạy cảm bậc nhất của một cá nhân: số căn cước công dân (CCCD), ảnh CCCD, thu nhập hằng tháng, nơi làm việc, số tài khoản ngân hàng nhận tiền và lịch sử tín dụng. Một cơ sở dữ liệu như vậy là mục tiêu có giá trị cao đối với tội phạm mạng, vì dữ liệu bị lộ có thể dùng ngay để giả danh, mở khoản vay hộ, lừa đảo qua điện thoại hoặc rao bán.

Về mặt kỹ thuật, các nguy cơ chính đối với loại hệ thống này gồm:

- **Rò rỉ cơ sở dữ liệu**: qua lỗi SQL Injection, bản sao lưu để quên trên máy chủ, ổ đĩa bị đánh cắp hoặc tài khoản quản trị cơ sở dữ liệu bị lộ. Nếu dữ liệu lưu dạng rõ, chỉ một tệp sao lưu bị lộ là toàn bộ CCCD và thu nhập của khách hàng bị lộ theo.
- **Lạm quyền nội bộ**: nhân viên xem hồ sơ không thuộc phạm vi công việc, xuất dữ liệu hàng loạt, hoặc nhân viên có quyền quản trị cơ sở dữ liệu sửa thẳng số tài khoản nhận tiền của một hồ sơ đã được phê duyệt để chuyển hướng tiền giải ngân.
- **Chiếm đoạt tài khoản**: dò mật khẩu, dùng lại mật khẩu bị lộ ở nơi khác (credential stuffing), đánh cắp phiên đăng nhập.
- **Chối bỏ và xóa dấu vết**: người có quyền sửa nhật ký để che giấu thao tác của mình.

Thiệt hại của một vụ lộ lọt dữ liệu rất lớn: theo báo cáo *Cost of a Data Breach 2024* của IBM, chi phí trung bình của một vụ vi phạm dữ liệu trên thế giới là 4,88 triệu USD, và ngành tài chính thuộc nhóm có chi phí cao nhất [24]. Bên cạnh thiệt hại tài chính, khung pháp lý tại Việt Nam ngày càng chặt chẽ: Nghị định 13/2023/NĐ-CP và Luật Bảo vệ dữ liệu cá nhân năm 2025 xếp dữ liệu tài chính, tín dụng vào nhóm dữ liệu cá nhân nhạy cảm, yêu cầu bên xử lý áp dụng biện pháp bảo vệ phù hợp [21][22]; Thông tư 09/2020/TT-NHNN quy định về an toàn hệ thống thông tin trong hoạt động ngân hàng [23].

Vì vậy, một hệ thống xét duyệt vay hiện đại không thể chỉ dựa vào "tường lửa và mật khẩu". Dữ liệu nhạy cảm cần được **mã hóa ở mức trường** sao cho bản sao cơ sở dữ

liệu bị lộ vẫn vô dụng nếu không có khóa, và mọi sửa đổi trái phép trên bản mã phải **bị phát hiện**, chứ không chỉ bị che giấu. Đây là bài toán mà các thuật toán mã hóa xác thực (AEAD) như ChaCha20-Poly1305 được thiết kế để giải quyết.

1.2. Lý do chọn đề tài

Nhóm chọn thuật toán **ChaCha20-Poly1305** làm trọng tâm vì bốn lý do:

1. **Tính mới và được dùng rộng rãi hiện nay.** ChaCha20-Poly1305 được IETF chuẩn hóa năm 2015 (RFC 7539) và bản hiện hành năm 2018 (RFC 8439) [1] [2]. Thuật toán là một trong các bộ mã của TLS 1.3 (RFC 8446) [10], là bộ mã duy nhất của giao thức VPN WireGuard (đã nằm trong nhân Linux từ phiên bản 5.6), là bộ mã mặc định được ưu tiên của OpenSSH, và được Google đưa vào Chrome trên Android từ năm 2014 [11][12]. Hằng ngày, một phần lớn kết nối HTTPS, QUIC và VPN trên thế giới được bảo vệ bằng thuật toán này.
2. **Thiết kế hiện đại, chạy an toàn mà không cần phần cứng đặc biệt.** ChaCha20 chỉ dùng phép cộng, xoay bit và XOR (ARX), không tra bảng nên tự nhiên chạy trong thời gian hằng, miễn nhiễm với tấn công kênh kề dựa trên bộ nhớ đệm, vốn là điểm yếu của AES cài đặt bằng bảng tra trên các máy không có lệnh AES-NI [15][16].
3. **Phù hợp để học và thuyết trình.** Cấu trúc thuật toán đủ đơn giản để nhóm tự cài đặt lại bằng khoảng 100 dòng Python, in ra trạng thái của từng vòng và kiểm chứng với các vector thử của RFC 8439. Nhờ vậy, báo cáo không dừng ở mức mô tả mà có số liệu kiểm chứng được.
4. **Gắn trực tiếp với đồ án của nhóm.** Hệ thống quản lý và xét duyệt vay tín dụng tiêu dùng (đồ án kết hợp học phần OOAD) có yêu cầu bảo mật SR06: *mã hóa ở mức trường cho số CCCD, số tài khoản nhận tiền, thu nhập; khóa mã hóa lưu tách khỏi cơ sở dữ liệu*. Nhóm đã đưa ChaCha20-Poly1305 vào hệ thống thành định dạng bản mã mới (ADR 0005), vẫn đọc được dữ liệu cũ mã hóa bằng AES-256-GCM, nên toàn bộ Chương 3 mô tả mã nguồn thật đang chạy chứ không phải mô phỏng.

1.3. Mục tiêu đề tài

1.3.1. Mục tiêu lý thuyết

- Hệ thống hóa các nguyên tắc an toàn thông tin (bí mật, toàn vẹn, sẵn sàng, xác thực) và áp dụng mô hình đe dọa STRIDE cho hệ thống xét duyệt vay.
- Tìm hiểu đầy đủ thuật toán ChaCha20-Poly1305: lịch sử, các biến thể và chế độ dùng, cấu trúc ma trận trạng thái, hàm quarter-round, hàm khối, cách mã hóa

dòng, mã xác thực Poly1305 và cách ghép thành AEAD; quy trình giải mã "xác thực trước, giải mã sau".

- Đánh giá độ an toàn (biên an toàn trước thám mã, giới hạn nonce, tấn công kênh kề) và hiệu năng so với AES-256-GCM.
- Tìm hiểu các cơ chế hỗ trợ: quản lý và dẫn xuất khóa (HKDF), mã xác thực thông điệp (Poly1305, HMAC-SHA256), chuỗi băm nhật ký, kiểm soát truy cập RBAC, phân tách nhiệm vụ, băm mật khẩu Argon2id và xác thực hai yếu tố TOTP.

1.3.2. Mục tiêu thực nghiệm / ứng dụng

- Đưa ChaCha20-Poly1305 vào mô-đun mã hóa mức trường `FieldCipher` của hệ thống, có phiên bản định dạng để tương thích dữ liệu cũ, kèm kiểm thử tự động.
- Tự cài đặt ChaCha20, Poly1305 và AEAD bằng Python thuần để minh họa, kiểm chứng với vector thử RFC 8439 và đối chiếu với thư viện `cryptography`.
- Chạy hệ thống thật (MySQL 8.4), thực hiện các kịch bản tấn công giả mạo trên cơ sở dữ liệu và ghi nhận phản ứng của hệ thống.
- Đo hiệu năng mã hóa, giải mã của ChaCha20-Poly1305 so với AES-256-GCM khi có và không có tăng tốc phần cứng AES-NI, cùng chi phí của các cơ chế hỗ trợ.

1.4. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: thuật toán mã hóa xác thực ChaCha20-Poly1305 theo RFC 8439; các cơ chế bảo mật hỗ trợ (HKDF, HMAC-SHA256, SHA-256, Argon2id, TOTP, RBAC); hệ thống quản lý và xét duyệt vay tín dụng tiêu dùng do nhóm xây dựng.

Trong phạm vi:

- Mã hóa dữ liệu tĩnh mức trường cho CCCD, thu nhập, số tài khoản nhận tiền, khóa TOTP của nhân viên và dữ liệu chờ xác nhận OTP.
- Bảo vệ toàn vẹn: thẻ xác thực Poly1305 gắn với ngữ cảnh bản ghi, chữ ký HMAC của snapshot phê duyệt, chuỗi băm nhật ký kiểm toán.
- Xác thực người dùng, phân quyền và các biện pháp chống tấn công tầng ứng dụng có liên quan.
- Dữ liệu truyền tải: thử nghiệm TLS 1.3 với bộ mã `TLS_CHACHA20_POLY1305_SHA256` ở máy chủ ứng dụng.

Ngoài phạm vi: mã hóa tệp giấy tờ (ảnh CCCD) lưu trên đĩa; quản lý khóa bằng HSM/KMS; triển khai thật reverse proxy Nginx; mật mã hậu lượng tử; các hệ thống ngoài (CIC, SMS, cổng thanh toán) vốn là giả lập.

1.5. Cấu trúc của báo cáo

Ngoài phần mở đầu, tài liệu tham khảo và phụ lục, báo cáo gồm bốn chương:

- **Chương 1 – Tổng quan và giới thiệu đề tài:** bối cảnh, lý do chọn thuật toán, mục tiêu, phạm vi.
- **Chương 2 – Cơ sở lý thuyết và các giải pháp an toàn bảo mật:** nguyên tắc an toàn, mô hình đe dọa, thuật toán ChaCha20-Poly1305 và các cơ chế hỗ trợ.
- **Chương 3 – Phân tích, thiết kế và hiện thực hệ thống:** yêu cầu, kiến trúc, sơ đồ ca sử dụng và tuần tự, hiện thực mã hóa, kết quả thực nghiệm và giao diện.
- **Chương 4 – Kết luận và hướng phát triển:** kết quả đạt được, hạn chế và hướng mở rộng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC GIẢI PHÁP AN TOÀN BẢO MẬT

2.1. Tổng quan về an toàn bảo mật dữ liệu

2.1.1. Các nguyên tắc cơ bản

An toàn thông tin thường được mô tả qua bộ ba **CIA** cùng thuộc tính **xác thực** [3]:

- **Bí mật (Confidentiality):** chỉ người được phép mới đọc được thông tin. Công cụ chính là mã hóa và kiểm soát truy cập. Trong hệ thống, CCCD và thu nhập được mã hóa, nhân viên chỉ thấy dữ liệu đã che (`079******234`) trừ khi có quyền xem đầy đủ.
- **Toàn vẹn (Integrity):** thông tin không bị sửa, xóa, chèn trái phép, hoặc nếu bị sửa thì phải bị phát hiện. Công cụ chính là mã xác thực thông điệp (MAC), hàm băm và chữ ký số. Mã hóa đơn thuần **không** bảo đảm toàn vẹn: với mã hóa dòng, lật một bit bản mã sẽ lật đúng bit tương ứng của bản rõ mà không ai biết. Đây là lý do cần mã hóa *xác thực*.
- **Sẵn sàng (Availability):** người dùng hợp lệ truy cập được dịch vụ khi cần. Công cụ: giới hạn tần suất, sao lưu, dự phòng.
- **Xác thực (Authentication):** xác minh đúng danh tính của người dùng (xác thực thực thể) và nguồn gốc của dữ liệu (xác thực thông điệp). Mở rộng từ đó là **truy vết** (accountability) và **chống chối bỏ** (non-repudiation): mọi thao tác quan trọng đều quy được về một người cụ thể.

Bảng 2.1. Ánh xạ nguyên tắc an toàn vào hệ thống

Nguyên tắc	Mối đe dọa điển hình	Cơ chế trong hệ thống
Bí mật	Lộ tệp sao lưu CSDL, nhân viên tò mò	ChaCha20-Poly1305 mức trường, khóa ngoài CSDL; che dữ liệu theo vai trò; ghi `VIEW_PII`
Toàn vẹn	Sửa số tài khoản nhận tiền sau phê duyệt	Tag Poly1305 gắn ngữ cảnh; HMAC-SHA256 snapshot; trigger BR07; chuỗi băm nhật ký
Sẵn sàng	Dò mật khẩu hàng loạt	Giới hạn tần suất 10 lượt/phút/IP; khóa tài khoản tạm 15 phút
Xác thực	Mật khẩu nhân viên bị lộ	Argon2id + TOTP bắt buộc với nhân viên; xác thực lại khi giải ngân
Truy vết, chống chối bỏ	Quản lý phủ nhận đã duyệt	Nhật ký kiểm toán chuỗi băm, chỉ ghi thêm; quyết định phê duyệt lưu người ký

2.1.2. Mô hình đe dọa (Threat Model) và nguy cơ tấn công

Tài sản cần bảo vệ: dữ liệu cá nhân của khách hàng (CCCD, thu nhập, số tài khoản), quyết định phê duyệt và số tiền giải ngân, nhật ký kiểm toán, khóa TOTP của nhân viên, và các khóa bí mật của hệ thống.

Tác nhân đe dọa:

1. *Kẻ tấn công bên ngoài* qua Internet: dò mật khẩu, khai thác lỗ hổng web (SQL Injection, CSRF, IDOR), đánh cắp phiên.
2. *Người trong nội bộ có ý đồ xấu*: nhân viên muốn tự duyệt hồ sơ, xem dữ liệu ngoài phạm vi, hoặc sửa hồ sơ sau phê duyệt.
3. *Kẻ có được bản sao CSDL*: qua tệp sao lưu bị lộ, ổ đĩa, hoặc lỗi SQL Injection chỉ đọc.
4. *Kẻ có quyền quản trị CSDL* (`root`): đọc và sửa trực tiếp mọi bảng, gỡ trigger.

Giả định tin cậy: máy chủ ứng dụng và biến môi trường chứa khóa nằm trong vùng tin cậy (Trusted Computing Base). Khóa **không** nằm trong CSDL, nên tác nhân 3 và 4 có toàn bộ bản mã nhưng không có khóa. Kẻ đã chiếm được máy chủ ứng dụng (đọc được bộ nhớ và biến môi trường) nằm ngoài khả năng bảo vệ của mã hóa mức ứng dụng.

Bảng 2.2. Mô hình đe dọa STRIDE và biện pháp giảm thiểu

STRIDE	Mối đe dọa trong bối cảnh cho vay	Biện pháp chính trong hệ thống
Spoofing – Giả mạo	Đăng nhập bằng mật khẩu nhân viên bị lộ	Argon2id; TOTP bắt buộc; khóa tạm sau 5 lần sai (dùng chung bộ đếm mật khẩu và TOTP)
Tampering – Sửa đổi	Sửa số tài khoản nhận tiền, số tiền sau khi duyệt; sửa nhật ký	Tag Poly1305 + AAD ngữ cảnh; HMAC snapshot và đối chiếu trước giải ngân; trigger; chuỗi băm
Repudiation – Chối bỏ	Quản lý phủ nhận đã phê duyệt	Nhật ký chuỗi băm chỉ ghi thêm; quyết định gắn người duyệt và mã băm snapshot
Information disclosure – Lộ lọt	Lộ tệp sao lưu; IDOR; nhân viên xem hàng loạt	Mã hóa mức trường; blind index; kiểm tra sở hữu (trả 404); cảnh báo khi xem PII vượt ngưỡng
Denial of service – Từ chối dịch vụ	Gửi hàng loạt yêu cầu đăng nhập, OTP	Giới hạn tần suất theo IP và theo người dùng (HTTP 429)
Elevation of	Nhân viên tín dụng gọi thẳng API	Kiểm tra quyền ở máy chủ cho mọi

STRIDE	Mối đe dọa trong bối cảnh cho vay	Biện pháp chính trong hệ thống
privilege – Leo thang	phê duyệt; tự gán vai trò	yêu cầu; SoD; cấm quản trị viên tự gán vai trò nghiệp vụ

Trọng tâm của đề tài là hai dòng *Tampering* và *Information disclosure* với tác nhân 3 và 4: đây chính là chỗ thuật toán mã hóa xác thực phát huy tác dụng.

2.2. Tìm hiểu thuật toán ChaCha20-Poly1305

2.2.1. Giới thiệu và lịch sử phát triển

ChaCha20-Poly1305 là một thuật toán **mã hóa xác thực có dữ liệu đi kèm (AEAD)**, ghép từ hai thành phần do nhà mật mã học Daniel J. Bernstein thiết kế:

- **ChaCha20**: mã dòng (stream cipher) khóa 256 bit, sinh dòng khóa giả ngẫu nhiên để XOR với bản rõ, bảo đảm tính **bí mật**.
- **Poly1305**: mã xác thực thông điệp một lần (one-time MAC) sinh thẻ xác thực 128 bit, bảo đảm **toàn vẹn và xác thực nguồn gốc**.

Các mốc phát triển chính:

Năm	Sự kiện
2005	Bernstein công bố mã dòng **Salsa20** và mã xác thực **Poly1305-AES** [5][6]
2008	Salsa20/12 được chọn vào danh mục cuối của dự án eSTREAM (châu Âu). Bernstein công bố **ChaCha**, biến thể của Salsa20 có độ khuếch tán mỗi vòng tốt hơn [4]
2013–2014	Google triển khai ChaCha20-Poly1305 trong TLS cho Chrome trên Android, vì điện thoại thời đó không có lệnh AES phần cứng; OpenSSH 6.5 thêm `chacha20-poly1305@openssh.com` [11]
2015	IETF ban hành **RFC 7539** chuẩn hóa ChaCha20-Poly1305 [2]; RFC 7634 dùng cho IPsec
2016	RFC 7905 thêm bộ mã ChaCha20-Poly1305 cho TLS 1.2; nhân Linux 4.8 dùng ChaCha20 làm bộ sinh số ngẫu nhiên
2018	**RFC 8439** thay thế RFC 7539 [1]; **TLS 1.3** (RFC 8446) có bộ mã `TLS_CHACHA20_POLY1305_SHA256` [10]; Java 11 hỗ trợ sẵn (JEP 329)
2020	**WireGuard** vào nhân Linux 5.6, dùng ChaCha20-Poly1305 làm bộ mã duy nhất [12]
2021	QUIC/HTTP/3 (RFC 9001) dùng ChaCha20-Poly1305 và dùng ChaCha20 để bảo vệ phần đầu gói [27]

Thiết kế của Bernstein theo triết lý “mật mã hiện đại”: thuật toán đơn giản, dễ cài đặt đúng, chạy nhanh bằng phần mềm thuần, không có nhánh hay tra bảng phụ thuộc dữ liệu bí mật. So với AES (chuẩn hóa năm 2001) [9], ChaCha20-Poly1305 mới hơn và

hiện là lựa chọn thay thế chính thức của AES-GCM trong hầu hết giao thức bảo mật hiện đại.

2.2.2. Các chế độ hoạt động

Mã khối như AES chỉ mã hóa được một khối 128 bit cố định, nên cần "chế độ hoạt động" để mã hóa dữ liệu dài. ChaCha20 thì khác: ngay từ thiết kế, nó đã là một **hàm sinh dòng khóa theo bộ đếm**. Mỗi lần gọi hàm khối với bộ ba (khóa, nonce, bộ đếm) cho ra 64 byte dòng khóa; tăng bộ đếm thì được 64 byte tiếp theo. Nói cách khác, ChaCha20 tự vận hành theo kiểu bộ đếm (counter mode) nên không cần đệm (padding), mã hóa song song được và truy cập ngẫu nhiên được tới khối bất kỳ.

Họ ChaCha có các biến thể và cách dùng sau:

Bảng 2.3. Các biến thể và chế độ dùng họ ChaCha

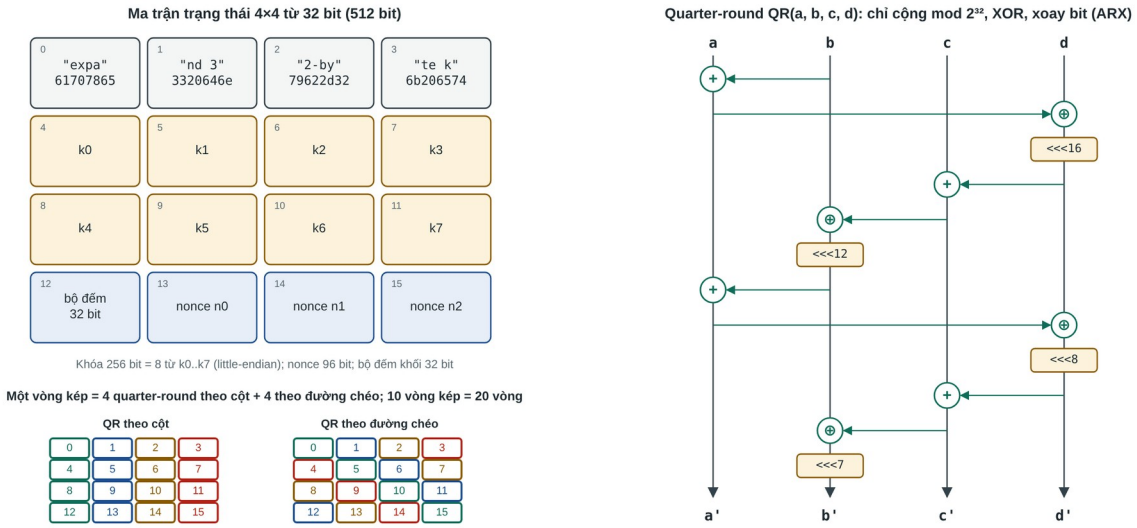
Chế độ / biến thể	Nonce	Bộ đếm	Thuộc tính	Nơi dùng
ChaCha20 thuần (mã dòng)	96 bit	32 bit	Chỉ bí mật, **không** phát hiện sửa đổi (bản mã dễ uốn)	Bộ sinh số ngẫu nhiên nhân Linux, bảo vệ đầu gói QUIC
ChaCha20 bản gốc của Bernstein	64 bit	64 bit	Như trên, giới hạn độ dài lớn hơn	libsodium (`chacha20`), bản thiết kế 2008
ChaCha20-Poly1305 (AEAD, RFC 8439)	96 bit	32 bit	Bí mật + toàn vẹn + xác thực AAD; tối đa 2^{32} khối × 64 byte ≈ 256 GiB mỗi bản tin	TLS 1.3, WireGuard, OpenSSH, **hệ thống của nhóm**
XChaCha20-Poly1305	192 bit	32 bit	Nonce dài, sinh ngẫu nhiên thoải mái; dẫn xuất khóa con bằng HChaCha20	libsodium (`xchacha20poly1305_ietf`)
ChaCha8 / ChaCha12	–	–	Ít vòng hơn, nhanh hơn, biên an toàn thấp hơn	Adiantum (mã hóa đĩa Android, XChaCha12)

Hệ thống của nhóm dùng **chế độ AEAD theo RFC 8439** với:

- nonce 96 bit sinh ngẫu nhiên từ `os.urandom` cho **mỗi** lần mã hóa;
- AAD là `byte định dạng ‖ "bảng.cột:id bản ghi"`, ví dụ `02 ‖ "customers.national_id:3f2b…"`. AAD không được mã hóa và không được lưu, nhưng tham gia vào việc tính thẻ xác thực: bản mã chỉ giải mã được đúng ở cột và bản ghi đã sinh ra nó.

2.2.3. Nguyên lý và quy trình mã hóa (Encryption)

a) Ma trận trạng thái. Hàm khối ChaCha20 làm việc trên một ma trận 4×4 gồm 16 từ 32 bit (512 bit), xếp như Hình 2.1: hàng đầu là 4 hằng số tạo từ chuỗi ASCII `"expand 32-byte k"`, hai hàng giữa là khóa 256 bit (8 từ), hàng cuối là bộ đếm khối 32 bit và nonce 96 bit. Mọi từ được đọc theo thứ tự little-endian.



Hình 2.1. Ma trận trạng thái và hàm quarter-round của ChaCha20

b) Hàm quarter-round. Đơn vị xử lý cơ bản là hàm QR trên bốn từ a, b, c, d, chỉ gồm cộng modulo 2^{32} ($\boxplus$), XOR ($\oplus$) và xoay trái ($\lll$):

```
a ← a ⊞ b;     d ← (d ⊕ a) ⋘ 16;
c ← c ⊞ d;     b ← (b ⊕ c) ⋘ 12;
a ← a ⊞ b;     d ← (d ⊕ a) ⋘ 8;
c ← c ⊞ d;     b ← (b ⊕ c) ⋘ 7.
```

Phép cộng modulo 2^{32} tạo tính phi tuyến (do bit nhớ lan sang trái), XOR trộn dữ liệu và phép xoay đưa bit cao xuống thấp. Kết hợp ba phép này qua nhiều vòng tạo ra hiệu ứng thác lũ: đổi một bit đầu vào làm thay đổi khoảng một nửa số bit đầu ra. Với vector thử ở RFC 8439 mục 2.1.1, đầu vào a=, b=, c=, d= cho ra a=, b=, c=, d=, đúng như cài đặt của nhóm in ra (Phụ lục D).

c) Vòng kép và hàm khối. Một *vòng kép* gồm 4 lần QR theo **cột** (các ô 0-4-8-12, 1-5-9-13, 2-6-10-14, 3-7-11-15) và 4 lần QR theo **đường chéo** (0-5-10-15, 1-6-11-12, 2-7-8-13, 3-4-9-14). ChaCha20 chạy 10 vòng kép, tức 20 vòng. Cuối cùng, cộng (modulo 2^{32}) ma trận sau 20 vòng với ma trận ban đầu rồi ghi ra 64 byte theo little-endian. Bước cộng lại trạng thái đầu này làm hàm khối **không khả nghịch**: biết đầu ra cũng không chạy ngược được 20 vòng để lấy lại khóa.

Bảng 2.4. Trạng thái hàm khối ChaCha20 theo ví dụ RFC 8439 mục 2.3.2 (khóa `00 01 … 1f`, nonce `00000009 0000004a 00000000`, bộ đếm 1; số liệu do script của nhóm in ra)

Hàng	Trạng thái ban đầu	Sau 20 vòng	Đầu ra = sau 20 vòng ⊞ ban đầu
0	`61707865 3320646e` `79622d32 6b206574`	`837778ab e238d763` `a67ae21e 5950bb2f`	`e4e7f110 15593bd1` `1fdd0f50 c47120a3`
1	`03020100 07060504` `0b0a0908 0f0e0d0c`	`c4f2d0c7 fc62bb2f` `8fa018fc 3f5ec7b7`	`c7f4d1c7 0368c033` `9aaa2204 4e6cd4c3`
2	`13121110 17161514` `1b1a1918 1f1e1d1c`	`335271c2 f29489f3` `eabda8fc 82e46ebd`	`466482d2 09aa9f07` `05d7c214 a2028bd9`
3	`00000001 09000000` `4a000000 00000000`	`d19c12b4 b04e16de` `9e83d0cb 4e3c50a2`	`d19c12b5 b94e16de` `e883d0cb 4e3c50a2`

Ghi ra little-endian, 64 byte dòng khóa bắt đầu bằng `10 f1 e7 e4 d1 3b 59 15 …`, khớp từng byte với RFC 8439.

d) Mã hóa dòng. Bản rõ P được chia thành các đoạn 64 byte P_0, P_1, … (đoạn cuối có thể ngắn hơn, không cần đệm):

$$C_j = P_j \oplus \text{ChaCha20Block}(K, \text{bộ đếm}_0 + j, N)$$

e) Mã xác thực Poly1305. Poly1305 nhận khóa một lần 256 bit gồm hai nửa r và s (mỗi nửa 128 bit). r bị "kẹp" (clamp) bằng phép AND với `0x0ffffffc0ffffffc0ffffffc0fffffff` để phép nhân được cài đặt hiệu quả. Thông điệp được chia thành khối 16 byte; mỗi khối được thêm một byte `0x01` rồi đọc thành số nguyên n_i (little-endian). Bộ tích lũy được cập nhật trên trường số nguyên tố $p = 2^{130} - 5$:

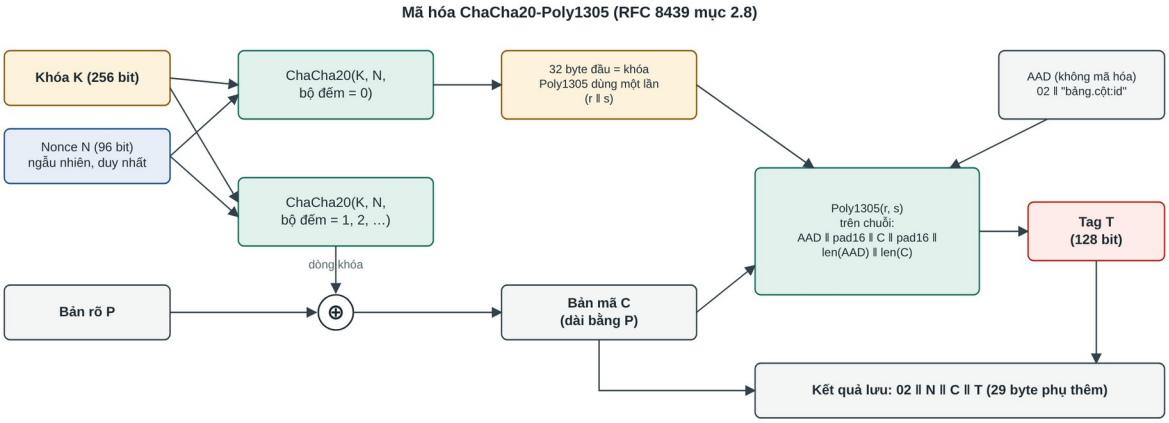
$$\text{acc} \leftarrow ((\text{acc} + n_i) \cdot r) \bmod (2^{130} - 5), \quad \text{tag} = (\text{acc} + s) \bmod 2^{128}$$

Thực chất, tag là giá trị của đa thức $\Sigma_i\, n_i \cdot r^{k-i+1}$ tại điểm bí mật r, cộng thêm mặt nạ s. Vì r, s chỉ dùng **một lần**, kẻ tấn công không có đủ phương trình để giải ra r và giả mạo tag.

f) Ghép thành AEAD (RFC 8439 mục 2.8). Hình 2.2 mô tả quy trình mã hóa đầy đủ:

1. Sinh khóa Poly1305 một lần: lấy 32 byte đầu của ChaCha20Block(K, 0, N).
2. Mã hóa bản rõ bằng ChaCha20 với bộ đếm bắt đầu từ **1**: C = P ⊕ ChaCha20(K, 1, N).
3. Dựng dữ liệu xác thực: AAD ‖ pad16 ‖ C ‖ pad16 ‖ len(AAD) ‖ len(C), độ dài là số 64 bit little-endian.

4. Tính T = Poly1305(r‖s, dữ liệu trên), thẻ dài 128 bit.
5. Kết quả: C ‖ T. Hệ thống ghép thêm byte định dạng và nonce ở đầu: `02 ‖ N ‖ C ‖ T`.



Hình 2.2. Quy trình mã hóa ChaCha20-Poly1305

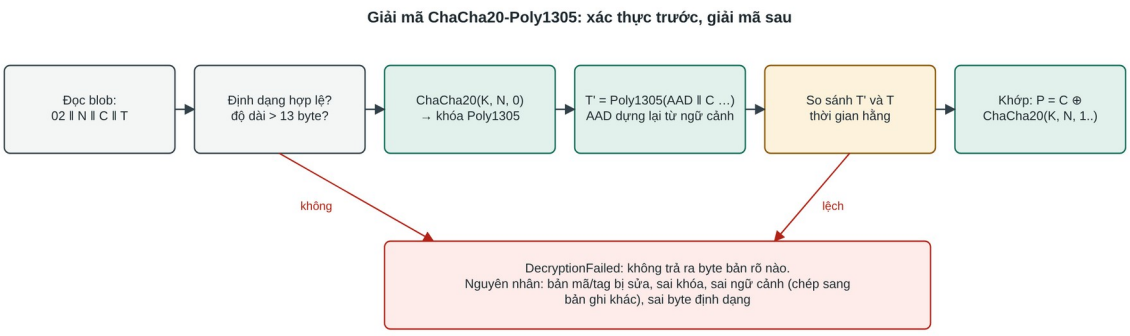
Đây là cách ghép **mã hóa rồi xác thực** (Encrypt-then-MAC): thẻ được tính trên bản mã chứ không phải bản rõ, nên khi giải mã có thể loại bản mã giả mạo trước khi đụng tới khóa giải mã. Procter (2014) đã chứng minh cách ghép này đạt an toàn AEAD tiêu chuẩn với giả thiết ChaCha20 là hàm giả ngẫu nhiên [7].

g) Ví dụ trên dữ liệu của hệ thống. Mã hóa số CCCD `079095001234` (12 byte) với ngữ cảnh `customers.national_id:3f2b0c4e1111422283339444555566` cho bản lưu 41 byte (một lần chạy cụ thể; nonce ngẫu nhiên nên mỗi lần một khác):

```
02                                byte định dạng (ChaCha20-Poly1305)
8faba8e1c0cfdf9eb81bdcc8          nonce 96 bit
8c40565e2713596c2b537dee          bản mã, đúng 12 byte như bản rõ
02d5dac2646adb982ccc2fe9a81df8b3  tag Poly1305 128 bit
```

2.2.4. Nguyên lý và quy trình giải mã (Decryption)

Vì ChaCha20 là mã dòng, phép giải mã về bản chất giống hệt phép mã hóa: XOR bản mã với cùng dòng khóa, $P_j = C_j \oplus \text{ChaCha20Block}(K, 1 + j, N)$. Điểm quan trọng của AEAD là **thứ tự**: phải xác thực thẻ **trước** rồi mới giải mã, và nếu thẻ sai thì không được trả ra bất kỳ byte bản rõ nào (Hình 2.3).



Hình 2.3. Quy trình giải mã: xác thực trước, giải mã sau

Các bước giải mã một trường trong hệ thống:

1. Đọc blob từ CSDL, tách `định dạng (1 byte) ‖ N (12 byte) ‖ C ‖ T (16 byte)`. Định dạng không biết hoặc blob quá ngắn thì từ chối ngay.
2. Dựng lại AAD từ ngữ cảnh đang đọc, ví dụ `02 ‖ "customers.national_id:<id khách hàng>"`.
3. Sinh lại khóa Poly1305 một lần từ ChaCha20Block(K, 0, N).
4. Tính T’ = Poly1305(AAD … C …) và so sánh T’ với T bằng phép so sánh **thời gian hằng** (không dừng ở byte sai đầu tiên, để không lộ thông tin qua thời gian phản hồi).
5. Nếu khớp: giải mã P = C ⊕ ChaCha20(K, 1, N). Nếu lệch: ném `DecryptionFailed`.

Kiểm tra thẻ thất bại khi xảy ra bất kỳ trường hợp nào sau: bản mã hoặc thẻ bị sửa dù chỉ một bit, sai khóa, sai ngữ cảnh (bản mã bị chép sang cột hay bản ghi khác), hoặc byte định dạng bị đổi. Đoạn mã giải mã thu gọn trong script minh họa của nhóm:

```
def aead_decrypt(key, nonce, sealed, aad):
    ciphertext, tag = sealed[:-16], sealed[-16:]
    otk = chacha20_block(key, 0, nonce)[:32]          # khóa Poly1305 dùng một lần
    expected = poly1305_mac(otk, _mac_data(aad, ciphertext))
    diff = 0
    for x, y in zip(expected, tag):                   # so sánh không dừng sớm
        diff |= x ^ y
    if diff or len(tag) != 16:
        raise InvalidTagError                         # không trả ra byte bản rõ nào
    return chacha20_xor(key, 1, nonce, ciphertext)
```

Thử nghiệm ở Phụ lục D: lật một bit bản mã, hoặc chép bản mã CCCD sang một khách hàng khác, đều bị từ chối với lỗi “tag mismatch”.

2.2.5. Đánh giá độ an toàn và hiệu năng

a) Độ an toàn của ChaCha20. Khóa 256 bit cho không gian tìm kiếm vét cạn 2^{256}. Kể cả khi có máy tính lượng tử, thuật toán Grover chỉ giảm độ an toàn xuống khoảng 2^{128}, vẫn ngoài tầm với. Các kết quả thám mã tốt nhất đã công bố chỉ tấn công được phiên bản rút gọn **7 vòng** với độ phức tạp rất lớn [8], trong khi ChaCha20 chạy 20 vòng, nên biên an toàn rất rộng.

b) Độ an toàn của Poly1305. Với thông điệp dài L byte, xác suất một lần giả mạo thành công không vượt quá $8\lceil L/16 \rceil / 2^{106}$ [6]. Với một trường dưới 64 byte, xác suất này không quá 2^{-101}: kẻ tấn công phải thử hàng nghìn tỷ tỷ lần mới mong một lần trúng, và mỗi lần thử sai đều bị hệ thống ghi nhận.

c) Điều kiện sống còn: nonce không được lặp với cùng khóa. Nếu hai bản rõ P_1, P_2 được mã hóa với cùng (K, N) thì $C_1 \oplus C_2 = P_1 \oplus P_2$, lộ quan hệ giữa hai bản rõ; đồng thời khóa Poly1305 một lần bị dùng hai lần, kẻ tấn công có thể giải ra r và giả mạo thẻ. Hệ thống sinh nonce 96 bit **ngẫu nhiên** cho mỗi lần mã hóa. Theo nghịch lý ngày sinh, xác suất xuất hiện hai nonce trùng nhau sau q lần mã hóa xấp xỉ $q^2 / 2^{97}$. Với $q = 2^{32}$ (hơn 4 tỷ lần), xác suất là 2^{-33}; đây cũng là ngưỡng NIST khuyến nghị cho nonce ngẫu nhiên 96 bit của AES-GCM [14]. Quy mô của hệ thống (vài triệu khách hàng, mỗi người vài trường, thỉnh thoảng cập nhật) cỡ 2^{24} lần mã hóa, cho xác suất trùng khoảng 2^{-49}, có thể bỏ qua.

d) Kênh kề (side channel). AES cài đặt bằng phần mềm thường dùng bảng tra (T-table). Thời gian truy cập bảng phụ thuộc vào việc dữ liệu có nằm trong bộ nhớ đệm hay không, và chỉ số tra bảng lại phụ thuộc khóa, nên kẻ tấn công đo thời gian có thể khôi phục khóa [15][16]. ChaCha20 chỉ dùng phép cộng, XOR và xoay với số bit cố định, không tra bảng, không rẽ nhánh theo dữ liệu bí mật, nên chạy trong thời gian hằng ngay cả trên CPU không có lệnh mật mã chuyên dụng.

Bảng 2.5. So sánh ChaCha20-Poly1305 với AES-256-GCM

Tiêu chí	**ChaCha20-Poly1305**	**AES-256-GCM**
Chuẩn	IETF RFC 8439 (2018)	NIST FIPS 197 (2001) + SP 800-38D (2007)
Cấu trúc	Mã dòng ARX + MAC đa thức trên số nguyên modulo $2^{130} - 5$	Mã khối SPN ở chế độ CTR + GHASH trên GF(2^{128})

Tiêu chí	ChaCha20-Poly1305	AES-256-GCM
Khóa / nonce / tag	256 / 96 / 128 bit	256 / 96 / 128 bit
Cần phần cứng để nhanh và an toàn	Không	Có (AES-NI, PCLMULQDQ); không có thì chậm và dễ bị tấn công thời gian
Lặp nonce	Lộ XOR bản rõ, giả mạo được tag	Lộ XOR bản rõ, lộ khóa băm H → giả mạo tùy ý
Độ dài tối đa một bản tin	≈ 256 GiB	≈ 64 GiB
Giao thức dùng	TLS 1.3, QUIC, WireGuard, OpenSSH	TLS 1.2/1.3, IPsec, QUIC, WPA3
Chứng nhận FIPS 140	Không thuộc danh mục thuật toán được NIST phê duyệt	Được phê duyệt

e) Hiệu năng. Trên CPU có lệnh AES-NI và nhân không mang (carry-less multiply), AES-GCM được tăng tốc phần cứng nên thường nhanh hơn ChaCha20-Poly1305. Ngược lại, trên thiết bị không có các lệnh này (nhiều điện thoại, máy nhúng, máy ảo bị ẩn cờ CPU), ChaCha20-Poly1305 nhanh hơn nhiều lần. Đó là lý do TLS 1.3 cho phép máy khách ưu tiên bộ mã phù hợp với phần cứng của mình. Với bài toán của hệ thống, dữ liệu mỗi lần mã hóa chỉ vài chục byte nên chi phí chủ yếu là chi phí gọi hàm, khoảng vài micro giây, nhỏ hơn hàng trăm lần so với một truy vấn CSDL. Số liệu đo thực tế trình bày ở mục 3.5.3.

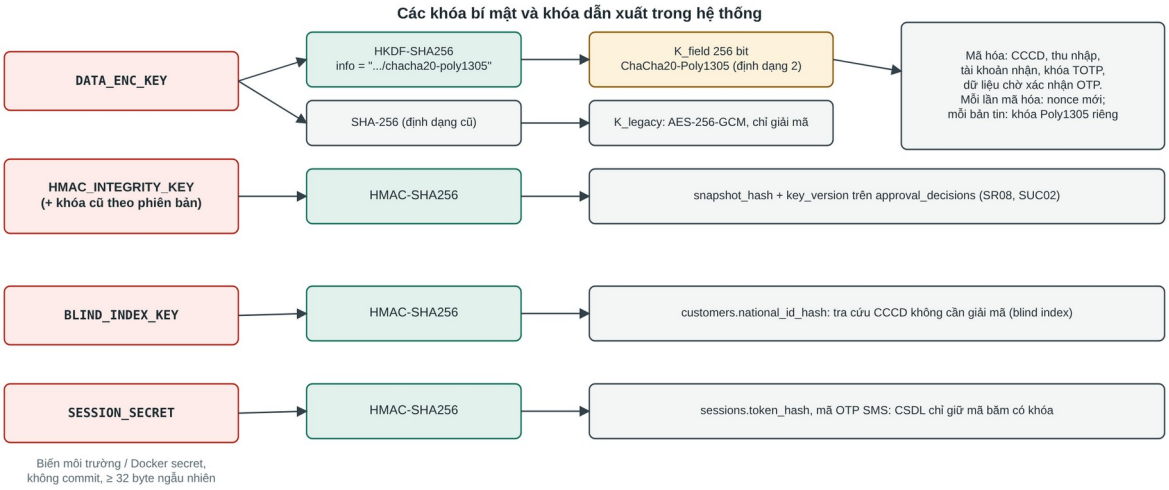
2.3. Các cơ chế hỗ trợ và hạ tầng liên quan

2.3.1. Quản lý và phân phối khóa

Thuật toán mạnh đến đâu cũng vô nghĩa nếu khóa bị lộ hoặc bị dùng sai mục đích. Hệ thống áp dụng các nguyên tắc sau (Hình 2.4):

1. **Tách khóa theo mục đích.** Có bốn bí mật độc lập: `DATA_ENC_KEY` (mã hóa trường), `HMAC_INTEGRITY_KEY` (ký snapshot phê duyệt), `BLIND_INDEX_KEY` (tra cứu CCCD) và `SESSION_SECRET` (băm token phiên, OTP). Lộ một khóa không kéo theo các khóa còn lại.
2. **Sinh khóa bằng nguồn ngẫu nhiên an toàn mật mã.** Mỗi bí mật dài tối thiểu 32 byte ngẫu nhiên, sinh bằng `secrets.token_urlsafe(48)` (384 bit).

3. **Khóa không nằm trong CSDL.** Khóa được nạp từ biến môi trường hoặc Docker secret khi khởi động ứng dụng, không commit lên Git (`.env` nằm trong `.gitignore`). Kẻ lấy được tệp sao lưu CSDL chỉ có bản mã.
4. **Dẫn xuất khóa có phân tách miền.** Khóa ChaCha20-Poly1305 không dùng thẳng chuỗi `DATA_ENC_KEY` mà được dẫn xuất bằng **HKDF-SHA256** (RFC 5869) [17] với tham số `info = "loan_system/field-encryption/chacha20-poly1305"`. HKDF gồm hai bước: *trích xuất* (HMAC chuỗi đầu vào thành một khóa giả ngẫu nhiên đồng đều) và *mở rộng* (HMAC khóa đó với `info` để ra khóa đúng độ dài). Nhờ `info`, cùng một bí mật gốc sinh ra các khóa độc lập cho từng thuật toán; khóa của định dạng cũ AES-256-GCM (SHA-256 của bí mật gốc) không trùng với khóa ChaCha20.
5. **Khóa dùng một lần cho mỗi bản tin.** Với mỗi lần mã hóa, nonce mới làm khóa Poly1305 một lần (r, s) cũng mới.
6. **Phiên bản hóa để xoay vòng.** Byte đầu của bản mã là phiên bản định dạng; snapshot phê duyệt lưu kèm `key_version`. Khi xoay khóa toàn vẹn, khóa cũ được giữ trong `HMAC_INTEGRITY_OLD_KEYS` để vẫn kiểm tra được hồ sơ đã duyệt trước đó.



Hình 2.4. Các khóa bí mật và khóa dẫn xuất trong hệ thống

Bảng 2.6. Các bí mật và vòng đời khóa

Bí mật	Mục đích	Lưu trữ	Xoay vòng
`DATA_ENC_KEY`	Dẫn xuất khóa ChaCha20-Poly1305 (HKDF)	Biến môi trường / Docker secret	Có phiên bản định dạng trong bản mã
`HMAC_INTEGRITY_KEY`	HMAC-SHA256 snapshot phê duyệt	Như trên	`key_version` lưu cùng mã băm, giữ khóa cũ

Bí mật	Mục đích	Lưu trữ	Xoay vòng
`BLIND_INDEX_KEY`	HMAC-SHA256 của CCCD để tra cứu	Như trên	Hạn chế: phải tính lại toàn bộ chỉ mục
`SESSION_SECRET`	Băm token phiên, mã OTP SMS	Như trên	Định kỳ; buộc mọi người đăng nhập lại
Khóa TOTP từng nhân viên	Sinh mã 6 số	CSDL, **đã mã hóa** bằng ChaCha20-Poly1305	Đăng ký lại TOTP

Phân phối khóa. Khóa mã hóa trường là khóa đối xứng chỉ máy chủ ứng dụng dùng, nên không phải trao đổi với bên nào khác: mô hình đơn giản và an toàn hơn so với việc phân phát khóa tới nhiều máy. Có hai chỗ cần phân phối khóa:

- **Khóa TOTP của nhân viên** là bí mật dùng chung giữa máy chủ và ứng dụng xác thực trên điện thoại. Máy chủ sinh 160 bit ngẫu nhiên, mã hóa rồi lưu, và trao cho nhân viên đúng **một lần** qua mã QR `otpauth://` hiển thị trên màn hình (kênh ngoài băng). Nhân viên phải nhập đúng một mã sinh từ khóa đó thì việc đăng ký mới hoàn tất (mục 3.3.4).
- **Khóa phiên TLS** cho dữ liệu truyền tải được thỏa thuận tự động trong bắt tay TLS 1.3 bằng trao đổi khóa Diffie–Hellman trên đường cong elliptic X25519. Máy chủ chứng minh danh tính bằng chứng chỉ số (PKI); hai bên dẫn xuất khóa bằng HKDF rồi bảo vệ dữ liệu bằng chính ChaCha20-Poly1305 (`TLS_CHACHA20_POLY1305_SHA256`), như thử nghiệm ở mục 3.4.2.

2.3.2. Chữ ký số / Mã xác thực thông điệp

Mã xác thực thông điệp (MAC) dùng một khóa bí mật chung để sinh và kiểm tra thẻ: ai giữ khóa thì vừa ký được vừa kiểm tra được. **Chữ ký số** dùng cặp khóa bất đối xứng: chỉ người giữ khóa riêng ký được, ai có khóa công khai cũng kiểm tra được, nên có thêm thuộc tính chống chối bỏ trước bên thứ ba. Trong một hệ thống mà cả bên ký lẫn bên kiểm tra đều là máy chủ ứng dụng, MAC là lựa chọn đủ dùng, đơn giản và nhanh hơn. Hệ thống dùng các cơ chế sau:

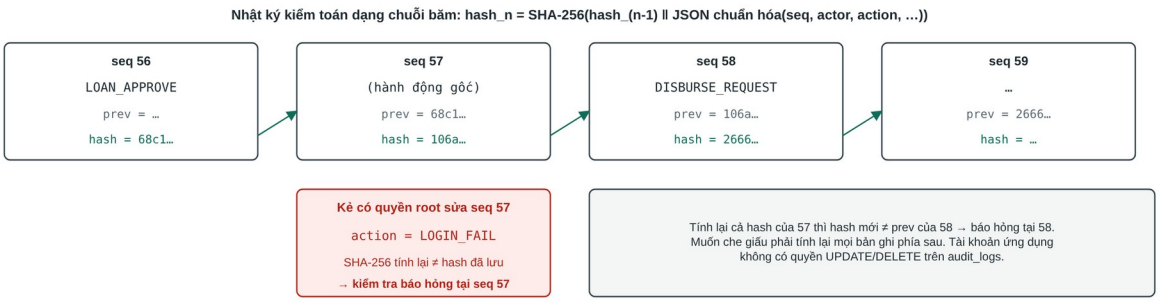
Bảng 2.7. Các cơ chế xác thực thông điệp trong hệ thống

Cơ chế	Dữ liệu được bảo vệ	Mục đích
Poly1305 (trong ChaCha20-Poly1305)	Từng trường mã hóa + ngữ cảnh AAD	Phát hiện sửa bản mã, chép bản mã sang bản ghi khác
HMAC-SHA256	Snapshot hồ sơ vay lúc phê duyệt:	Đối chiếu trước giải ngân

Cơ chế	Dữ liệu được bảo vệ	Mục đích
(HMAC_INTEGRITY_KEY)	mã hồ sơ, số tiền, hạn mức, kỳ hạn, lãi suất, tài khoản nhận	(SUC02); lệch thì khóa hồ sơ
HMAC-SHA256 (BLIND_INDEX_KEY)	Số CCCD	Tra cứu chính xác mà không giải mã, không lộ CCCD
HMAC-SHA256 (SESSION_SECRET)	Token phiên, mã OTP SMS	CSDL chỉ giữ mã băm có khóa; lộ bảng sessions không dùng được token
SHA-256 chuỗi băm	Nhật ký kiểm toán	Phát hiện sửa, xóa, chèn bản ghi nhật ký
SHA-256 checksum	Tệp mô hình chấm điểm	Phát hiện tệp mô hình bị thay (SR13)

HMAC (RFC 2104) [13] tính HMAC(K, m) = H((K ⊕ opad) ‖ H((K ⊕ ipad) ‖ m)), an toàn kể cả khi hàm băm H có điểm yếu mở rộng độ dài. Khi kiểm tra, hệ thống luôn dùng `hmac.compare_digest` để so sánh trong thời gian hằng.

Chuỗi băm nhật ký kiểm toán. Mỗi bản ghi nhật ký lưu `prev_hash` (mã băm của bản ghi liền trước) và `hash = SHA-256(prev_hash ‖ JSON chuẩn hóa(seq, actor, action, đối tượng, IP, mức độ, thời điểm, chi tiết))`; bản ghi đầu tiên nối vào chuỗi 64 số 0 (Hình 2.5). Sửa một bản ghi làm mã băm tính lại của nó khác với mã đã lưu; nếu kẻ tấn công tính lại cả mã băm thì bản ghi kế tiếp lại không khớp `prev_hash`; xóa bản ghi tạo ra khoảng hở ở `seq`. Muốn che giấu, kẻ tấn công phải tính lại toàn bộ chuỗi phía sau, và tài khoản ứng dụng vốn không có quyền `UPDATE`, `DELETE` trên bảng này.



Hình 2.5. Nhật ký kiểm toán dạng chuỗi băm

Về chữ ký số. Snapshot phê duyệt hiện được "ký" bằng HMAC, nên về mặt mật mã, máy chủ (người giữ khóa) cũng có thể tạo ra mã băm hợp lệ; tính chống chối bỏ dựa vào nhật ký kiểm toán chứ chưa dựa vào chữ ký bất đối xứng của từng quản lý. Nâng

cấp lên chữ ký số Ed25519 hoặc ML-DSA với khóa riêng của từng người duyệt là hướng phát triển nêu ở Chương 4.

2.3.3. Cơ chế kiểm soát truy cập và xác thực người dùng

a) Xác thực mật khẩu bằng Argon2id. Mật khẩu không bao giờ được lưu dạng rõ hay băm nhanh (MD5, SHA-1). Hệ thống dùng Argon2id (RFC 9106) [18] với tham số mặc định của `argon2-cffi`: bộ nhớ 64 MiB, 3 lượt, 4 luồng, salt ngẫu nhiên, ví dụ chuỗi lưu `$argon2id$v=19$m=65536,t=3,p=4$...`. Việc tốn 64 MiB bộ nhớ cho mỗi lần băm làm tấn công dò mật khẩu bằng GPU/ASIC đắt đỏ hơn nhiều lần. Khi tên đăng nhập không tồn tại, hệ thống vẫn chạy Argon2 trên một mã băm giả để thời gian phản hồi không tiết lộ tài khoản nào có thật. Mật khẩu tối thiểu 10 ký tự; sai 5 lần liên tiếp thì khóa tạm 15 phút, phù hợp hướng dẫn NIST SP 800-63B [28].

b) Xác thực hai yếu tố TOTP (RFC 6238). Nhân viên bắt buộc dùng mã 6 số từ ứng dụng xác thực. Mã được tính bằng HOTP(K, ⌊t/30⌋), trong đó HOTP (RFC 4226) lấy HMAC-SHA1 của bộ đếm thời gian, cắt động 31 bit rồi lấy 6 chữ số cuối [19][20]. Hệ thống chấp nhận lệch ±1 bước (±30 giây), lưu `totp_last_step` để **một mã không dùng được hai lần**, và dùng chung bộ đếm khóa tài khoản với mật khẩu để biết mật khẩu cũng không dò được mã TOTP. Các thao tác nhạy cảm (giải ngân, tạo nhân viên, đổi vai trò, đổi chính sách) yêu cầu **xác thực lại** bằng TOTP (step-up).

c) Phiên đăng nhập phía máy chủ thay cho JWT. Sau khi đăng nhập, máy chủ sinh token ngẫu nhiên 256 bit, gửi cho trình duyệt trong cookie `HttpOnly; Secure; SameSite=Strict`, còn CSDL chỉ lưu HMAC-SHA256(`SESSION_SECRET`, token). Nhóm chọn phiên phía máy chủ thay cho JWT vì đăng xuất, hết hạn sau 15 phút không hoạt động và hủy phiên khi đổi mật khẩu đều có hiệu lực **ngay lập tức**; với JWT, token đã cấp vẫn hợp lệ đến khi hết hạn. Phiên đi qua các giai đoạn `SETUP` (đổi mật khẩu tạm, đăng ký TOTP), `MFA` (đã qua mật khẩu, chờ TOTP) và `FULL`; chỉ phiên `FULL` mới dùng được chức năng nghiệp vụ.

d) Kiểm soát truy cập dựa trên vai trò (RBAC). Hệ thống có 7 vai trò: Khách hàng, Nhân viên tín dụng, Thẩm định viên, Quản lý phê duyệt, Nhân viên giải ngân, Kiểm soát viên và Quản trị viên. Mỗi vai trò gắn một tập quyền (ví dụ `APPLICATION_VERIFY`, `CUSTOMER_VIEW_PII`, `LOAN_APPROVE`, `DISBURSE`, `AUDIT_VERIFY`). Mọi endpoint kiểm tra quyền ở **máy chủ**; ẩn nút trên giao diện chỉ để tiện dùng, không phải biện pháp bảo

mật. Yêu cầu truy cập đối tượng không thuộc quyền sở hữu (IDOR) nhận 404, và mọi lần bị từ chối đều được ghi `ACCESS_DENIED`.

e) Phân tách nhiệm vụ (SoD). Người tạo hồ sơ, người tiếp nhận, người thẩm định, người phê duyệt và người giải ngân của cùng một hồ sơ vay phải là những người khác nhau; vi phạm thì bị chặn (403) và ghi `SOD_VIOLATION`, báo cho Kiểm soát viên. Một vai trò không được giữ từ hai quyền trở lên trong chuỗi `APPRAISAL_SUBMIT → LOAN_APPROVE → DISBURSE`, Quản trị viên không được giữ vai trò nghiệp vụ cho vay và không tự gán vai trò cho chính mình. Khoản vay trên 50 triệu đồng cần hai quản lý phê duyệt độc lập.

f) Các biện pháp bổ trợ. Chống CSRF theo kiểu double-submit cookie; giới hạn tần suất 10 lượt mỗi phút mỗi IP cho đăng nhập và OTP, 30 lượt mỗi phút mỗi người cho nộp hồ sơ, tải giấy tờ; mỗi lần nhân viên xem CCCD, thu nhập đầy đủ được ghi `VIEW_PII`, quá 20 lượt mỗi giờ thì phát cảnh báo `CRITICAL`; giấy tờ được kiểm tra kiểu thật qua magic bytes và in watermark tên người xem.

CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG

3.1. Giới thiệu tổng quan hệ thống

Tên ứng dụng: Hệ thống quản lý và xét duyệt vay tín dụng tiêu dùng (mã nguồn `consumer-loan-approval-system`, gói Python `loan_system`).

Bài toán giải quyết: quản lý toàn bộ vòng đời một khoản vay tín chấp tiêu dùng cá nhân từ 5 đến 100 triệu đồng, kỳ hạn 6–36 tháng, tại một công ty tài chính giả định: khách hàng đăng ký và nộp hồ sơ vay trực tuyến, nhân viên tín dụng kiểm tra hồ sơ, hệ thống tra cứu CIC và chấm điểm tín dụng, chuyên viên thẩm định lập tờ trình, quản lý phê duyệt (hai người với khoản trên 50 triệu), nhân viên giải ngân chuyển tiền, sau đó hệ thống theo dõi thanh toán, quá hạn và tất toán. Hệ thống được thiết kế theo nguyên lý *Security by Design*: phân quyền RBAC, phân tách nhiệm vụ, mã hóa dữ liệu cá nhân, nhật ký kiểm toán chuỗi băm và kiểm tra toàn vẹn hồ sơ trước khi giải ngân.

Môi trường triển khai: máy chủ ứng dụng Python 3.12 chạy FastAPI trên Uvicorn; CSDL MySQL 8.4; triển khai bằng Docker Compose với hai mạng tách biệt (`app` và `data`, mạng `data` là mạng nội bộ, MySQL không mở cổng ra ngoài). Thực nghiệm trong báo cáo chạy trên máy ảo Linux 4 nhân Intel Xeon 2,1 GHz, MySQL 8.4.13 trong Docker.

3.2. Phân tích yêu cầu hệ thống

3.2.1. Yêu cầu chức năng (Functional Requirements)

Bảng 3.1. Yêu cầu chức năng (rút gọn từ đề cương đồ án, cột cuối nêu cơ chế bảo mật đi kèm)

Mã	Chức năng	Tác nhân	Cơ chế bảo mật liên quan
FR01.1	Đăng nhập, đăng xuất, xác thực hai yếu tố cho nhân viên	Mọi người dùng	Argon2id, TOTP, phiên phía máy chủ
FR01.2	Quản lý tài khoản nhân viên	Quản trị viên	Xác thực lại TOTP, mật khẩu tạm ngẫu nhiên
FR01.3	Quản lý vai trò, quyền; chính sách phê duyệt	Quản trị viên	Quy tắc xung đột quyền, cấm tự gán vai trò
FR01.4	Tra cứu nhật ký kiểm toán, kiểm tra toàn vẹn chuỗi	Kiểm soát viên	Chuỗi băm SHA-256
FR02.1	Khách hàng đăng ký, cập nhật thông	Khách hàng	OTP qua SMS, mã hóa thu

Mã	Chức năng	Tác nhân	Cơ chế bảo mật liên quan
	tin cá nhân		nhập
FR02.2	Nhân viên tra cứu khách hàng (dữ liệu che theo vai trò)	NV tín dụng, Thẩm định viên	Blind index, che dữ liệu, ghi `VIEW_PII`
FR03.1	Tạo và nộp hồ sơ vay, tải giấy tờ	Khách hàng, NV tín dụng	**Mã hóa ChaCha20-Poly1305** CCCD, thu nhập, tài khoản nhận
FR04.1–2	Tra cứu CIC, chấm điểm tự động	Hệ thống	Checksum SHA-256 tệp mô hình
FR05.1–3	Thẩm định, phê duyệt (BR05, BR06), ký snapshot	Thẩm định viên, Quản lý phê duyệt	SoD, HMAC-SHA256 snapshot
FR06.1–3	Giải ngân sau kiểm tra toàn vẹn, xác nhận bằng OTP	NV giải ngân	SUC02, xác thực lại TOTP, khóa hồ sơ khi lệch
FR07, FR08	Thu nợ, quá hạn, tất toán, báo cáo	Nhiều vai trò	RBAC, nhật ký

3.2.2. Yêu cầu an toàn bảo mật và phi chức năng (Non-functional Requirements)

Các yêu cầu bảo mật SR01–SR14 được rút ra từ bảng STRIDE (mục 2.1.2) và đối chiếu với OWASP ASVS [25].

Bảng 3.2. Yêu cầu bảo mật và mức độ đáp ứng

Mã	Yêu cầu	Hiện thực trong hệ thống	Đáp ứng
SR01	Mật khẩu ≥ 10 ký tự, băm có salt, khóa tạm sau 5 lần sai	Argon2id (m=64 MiB, t=3, p=4); khóa 15 phút	Đạt
SR02	Nhân viên bắt buộc TOTP; giải ngân yêu cầu xác thực lại	RFC 6238, chống dùng lại mã; step-up	Đạt
SR03	RBAC, kiểm tra quyền ở máy chủ cho mọi yêu cầu	Dependency `require(permission)`	Đạt
SR04	Kiểm tra quyền sở hữu, chống IDOR	Trả 404 cho đối tượng không thuộc quyền	Đạt
SR05	Thực thi phê duyệt kép, SoD ở tầng nghiệp vụ	`SegregationOfDuties`, ghi `SOD_VIOLATION`	Đạt
SR06	**Mã hóa mức trường CCCD, tài khoản, thu nhập; khóa ngoài CSDL; HTTPS**	**ChaCha20-Poly1305 + HKDF (ADR 0005); TLS 1.3 thử nghiệm ở máy chủ ứng dụng**	**Đạt mã hóa trường; TLS mới ở mức thử nghiệm**

Mã	Yêu cầu	Hiện thực trong hệ thống	Đáp ứng
SR07	Che dữ liệu theo vai trò	`079******234`, `012****789`	Đạt
SR08	Mã băm snapshot khi phê duyệt, đối chiếu trước giải ngân	HMAC-SHA256 có phiên bản khóa	Đạt
SR09	Nhật ký ghi thêm, chuỗi băm; cảnh báo xem PII bất thường	`audit_logs` chỉ `SELECT, INSERT`; ngưỡng 20 lượt/giờ	Đạt
SR10	Kiểm tra đầu vào, ORM, chống mass assignment, giới hạn tệp	Pydantic, SQLAlchemy, magic bytes, ≤ 5 MB	Đạt
SR11	Phiên hết hạn 15 phút; cookie HttpOnly, Secure, SameSite	Phiên phía máy chủ, token băm HMAC	Đạt
SR12	Giới hạn tần suất	Cửa sổ trượt trong bộ nhớ tiến trình	Đạt (một tiến trình)
SR13	Kiểm tra checksum mô hình chấm điểm	SHA-256 đăng ký trong migration	Đạt
SR14	Thu thập tối thiểu, ghi nhận đồng ý	`consent_at`, `consent_version`	Đạt

Các yêu cầu phi chức năng chung: thời gian phản hồi tra cứu dưới 2 giây (NFR01), giao diện tiếng Việt (NFR03), kiến trúc phân tầng dễ thay thế mô-đun (NFR04), tiền tính bằng `Decimal` chính xác đến đồng (NFR05) và tuân thủ quy định bảo vệ dữ liệu cá nhân (NFR06). Riêng với mã hóa, yêu cầu hiệu năng được cụ thể hóa thành: **chi phí mã hóa, giải mã một trường không vượt quá 1% thời gian xử lý một yêu cầu**. Mục 3.5.3 cho thấy chi phí thực tế khoảng 2–6 micro giây, cỡ 0,01% của một yêu cầu web điển hình (vài chục mili giây).

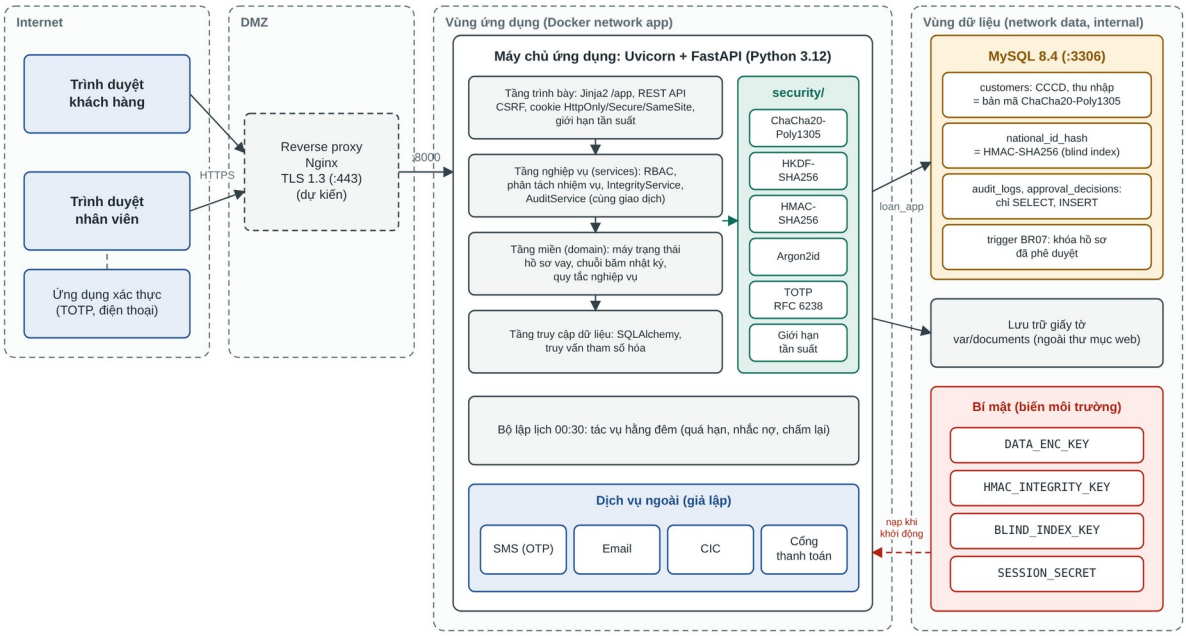
3.3. Thiết kế kiến trúc hệ thống

3.3.1. Mô hình tổng thể

Hệ thống theo mô hình client – server nhiều tầng, chia vùng mạng như Hình 3.1:

- **Vùng Internet:** trình duyệt của khách hàng và nhân viên; ứng dụng xác thực TOTP trên điện thoại nhân viên.
- **DMZ:** reverse proxy Nginx kết thúc TLS 1.3 ở cổng 443 (thiết kế đã có, trong `docker-compose.yml` ghi chú sẽ bổ sung).

- **Vùng ứng dụng:** máy chủ FastAPI với bốn tầng (trình bày, nghiệp vụ, miền, truy cập dữ liệu) và mô-đun `security/` dùng chung: ChaCha20-Poly1305, HKDF, HMAC, Argon2id, TOTP, giới hạn tần suất.
- **Vùng dữ liệu:** MySQL chỉ gắn vào mạng nội bộ; ứng dụng kết nối bằng tài khoản `loan_app` có quyền tối thiểu theo từng bảng; thư mục giấy tờ nằm ngoài thư mục web.
- **Bí mật:** bốn khóa nạp từ biến môi trường khi khởi động, **tách khỏi CSDL** – đây là điều kiện để mã hóa mức trường có ý nghĩa.



Hình 3.1. Kiến trúc triển khai và vị trí các cơ chế bảo mật

Mã hóa được đặt ở **tầng ứng dụng** chứ không giao cho CSDL (như tính năng mã hóa tablespace của MySQL), vì mã hóa phía CSDL chỉ chống được kẻ lấy ổ đĩa, còn người có quyền truy vấn vẫn đọc được dữ liệu rõ. Với mã hóa ở tầng ứng dụng, cả quản trị viên CSDL lẫn kẻ khai thác SQL Injection chỉ thấy bản mã.

3.3.2. Công nghệ và công cụ sử dụng

Bảng 3.3. Công nghệ và công cụ

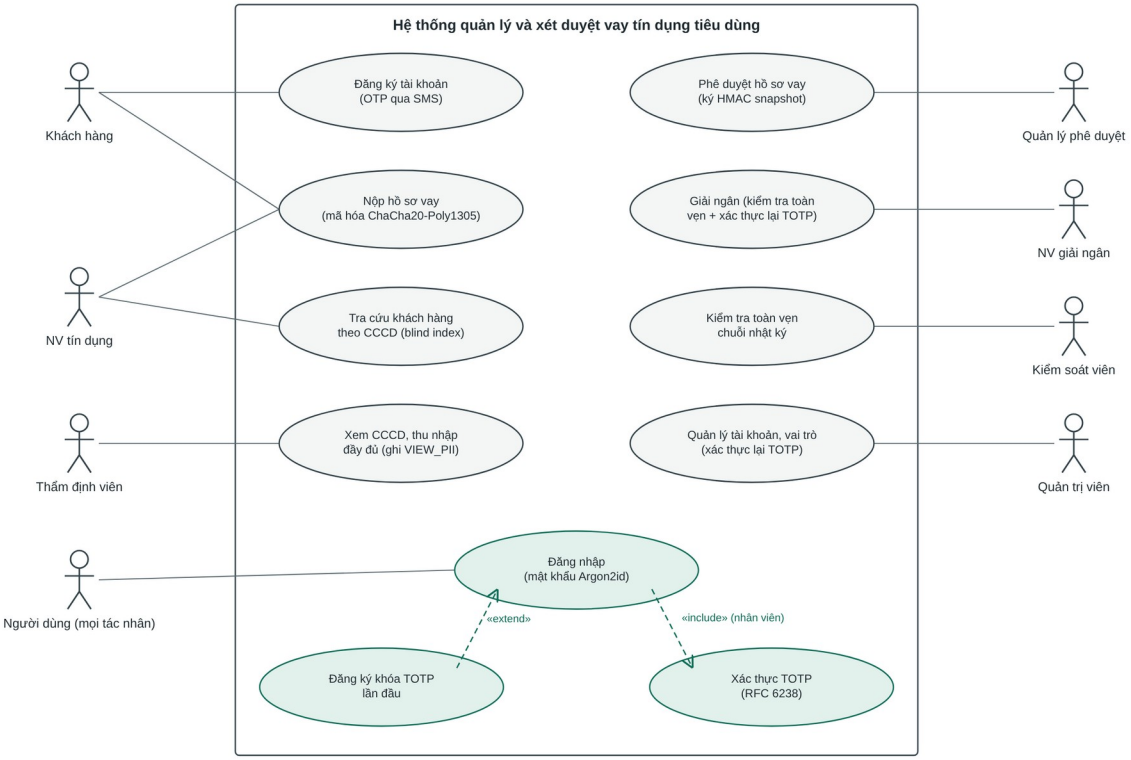
Hạng mục	Công cụ (phiên bản)	Vai trò trong đề tài
Ngôn ngữ	Python 3.12	Ngôn ngữ cài đặt
Web framework	FastAPI 0.141, Pydantic 2.13, Uvicorn 0.54	REST API, kiểm tra dữ liệu vào, máy chủ ASGI (hỗ trợ TLS)
Giao diện	Jinja2 3.1	Trang HTML dưới `/app`

Hạng mục	Công cụ (phiên bản)	Vai trò trong đề tài
ORM, CSDL	SQLAlchemy 2.1, Alembic 1.20, PyMySQL 1.2, MySQL 8.4	Ánh xạ đối tượng – quan hệ, migration, trigger
Mật mã	**`cryptography` 50.0.1 (OpenSSL 4.0.2 đóng gói sẵn)**	**`ChaCha20Poly1305`, `HKDF`, `AESGCM` (đọc định dạng cũ), `ChaCha20`**
Băm mật khẩu	argon2-cffi 25.1	Argon2id
Thư viện chuẩn	`hmac`, `hashlib`, `secrets`	HMAC-SHA256, SHA-256, CSPRNG, TOTP
Ảnh, PDF	Pillow 12.3, fpdf2 2.8	Watermark giấy tờ, lịch trả nợ PDF
Kiểm thử	pytest 9.1, httpx 0.28, mypy 2.3 (strict)	Kiểm thử miền và HTTP, kiểm tra kiểu
Triển khai	Docker, Docker Compose, uv	Đóng gói, mô phỏng vùng mạng
Thực nghiệm	Playwright (Chromium), OpenSSL CLI, curl	Chụp giao diện, kiểm tra TLS 1.3

Nhóm dùng thư viện `cryptography` (lớp bọc OpenSSL do PyCA duy trì) [26] cho mã chạy thật thay vì tự cài đặt, theo nguyên tắc "không tự viết mật mã cho môi trường thật": bản cài đặt của thư viện đã được kiểm định rộng rãi, chạy thời gian hằng và dùng lệnh SIMD. Bản tự cài đặt bằng Python thuần của nhóm chỉ dùng để học và minh họa (mục 2.2, Phụ lục D).

3.3.3. Sơ đồ ca sử dụng (Use Case Diagram)

Hình 3.2 tập trung vào các ca sử dụng có xử lý bảo mật. Tác nhân trừu tượng *Người dùng* là khái quát hóa của mọi tác nhân, ai cũng phải *Đăng nhập*; với nhân viên, *Đăng nhập* bao gồm (*include*) *Xác thực TOTP*, và lần đăng nhập đầu tiên được mở rộng (*extend*) bởi *Đăng ký khóa TOTP lần đầu*. *Giải ngân* và *Quản lý tài khoản, vai trò* yêu cầu xác thực lại TOTP ngay trước khi thực hiện.

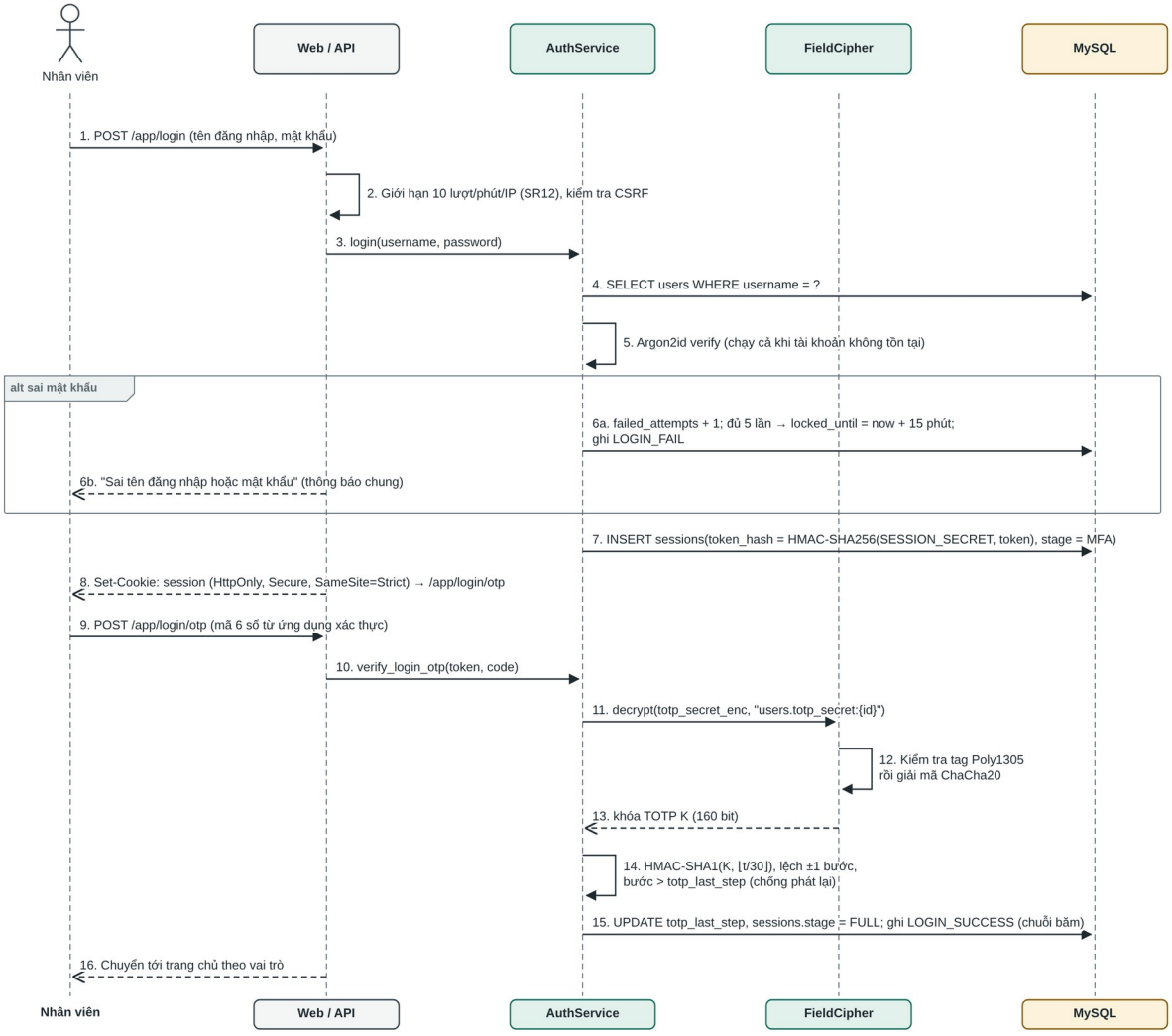


Hình 3.2. Sơ đồ ca sử dụng các chức năng liên quan bảo mật

Các ca sử dụng có dùng ChaCha20-Poly1305: *Nộp hồ sơ vay* (mã hóa CCCD, thu nhập, tài khoản nhận), *Xem CCCD, thu nhập đầy đủ* (giải mã có ghi nhật ký), *Đăng ký khóa TOTP* và *Xác thực TOTP* (mã hóa, giải mã khóa TOTP), *Phê duyệt* và *Giải ngân* (giải mã tài khoản nhận để dựng snapshot).

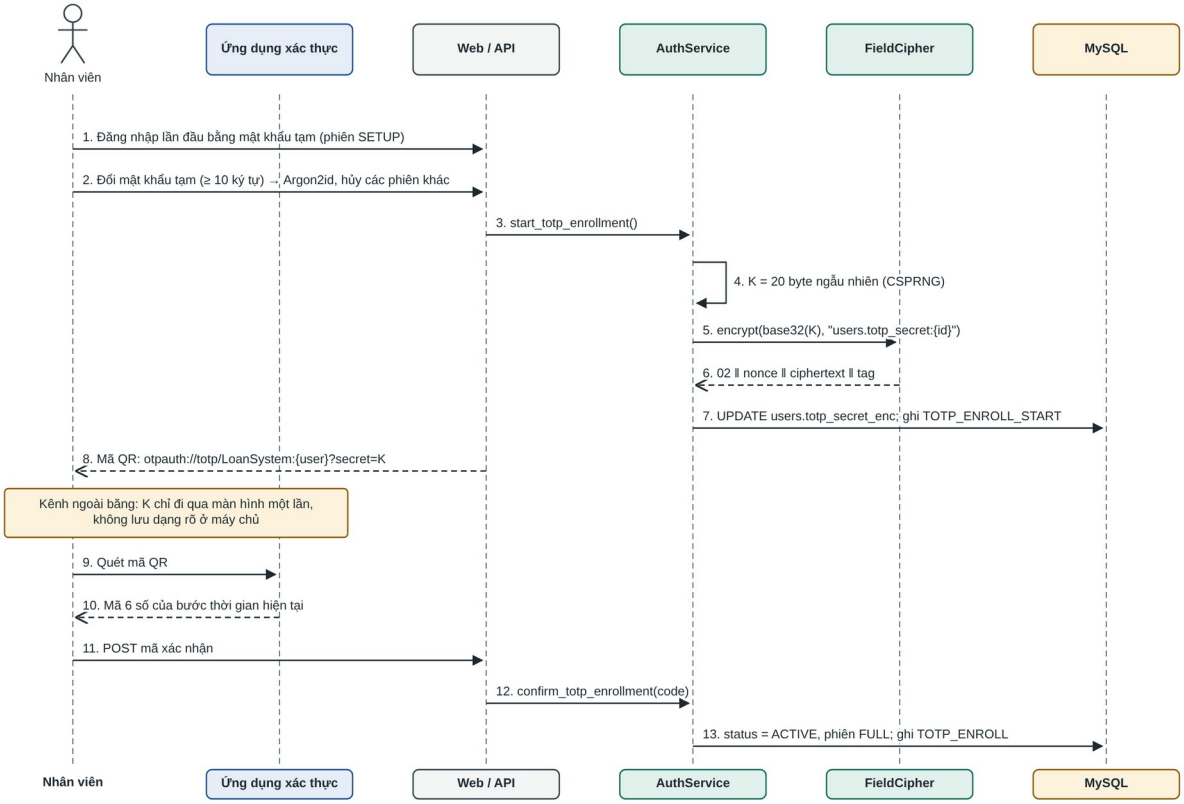
3.3.4. Sơ đồ tuần tự xử lý bảo mật (Sequence Diagram)

a) Đăng nhập hai yếu tố (Hình 3.3). Mật khẩu được kiểm tra bằng Argon2id. Nếu đúng, máy chủ tạo phiên ở giai đoạn `MFA` và chỉ lưu mã băm HMAC của token. Ở bước TOTP, `FieldCipher` giải mã khóa TOTP của nhân viên: thẻ Poly1305 được kiểm tra trước, đúng mới giải mã. Sau đó máy chủ tính mã theo RFC 6238, chặn mã đã dùng và nâng phiên lên `FULL`. Sai mật khẩu hay sai TOTP đều tăng chung một bộ đếm khóa tài khoản.



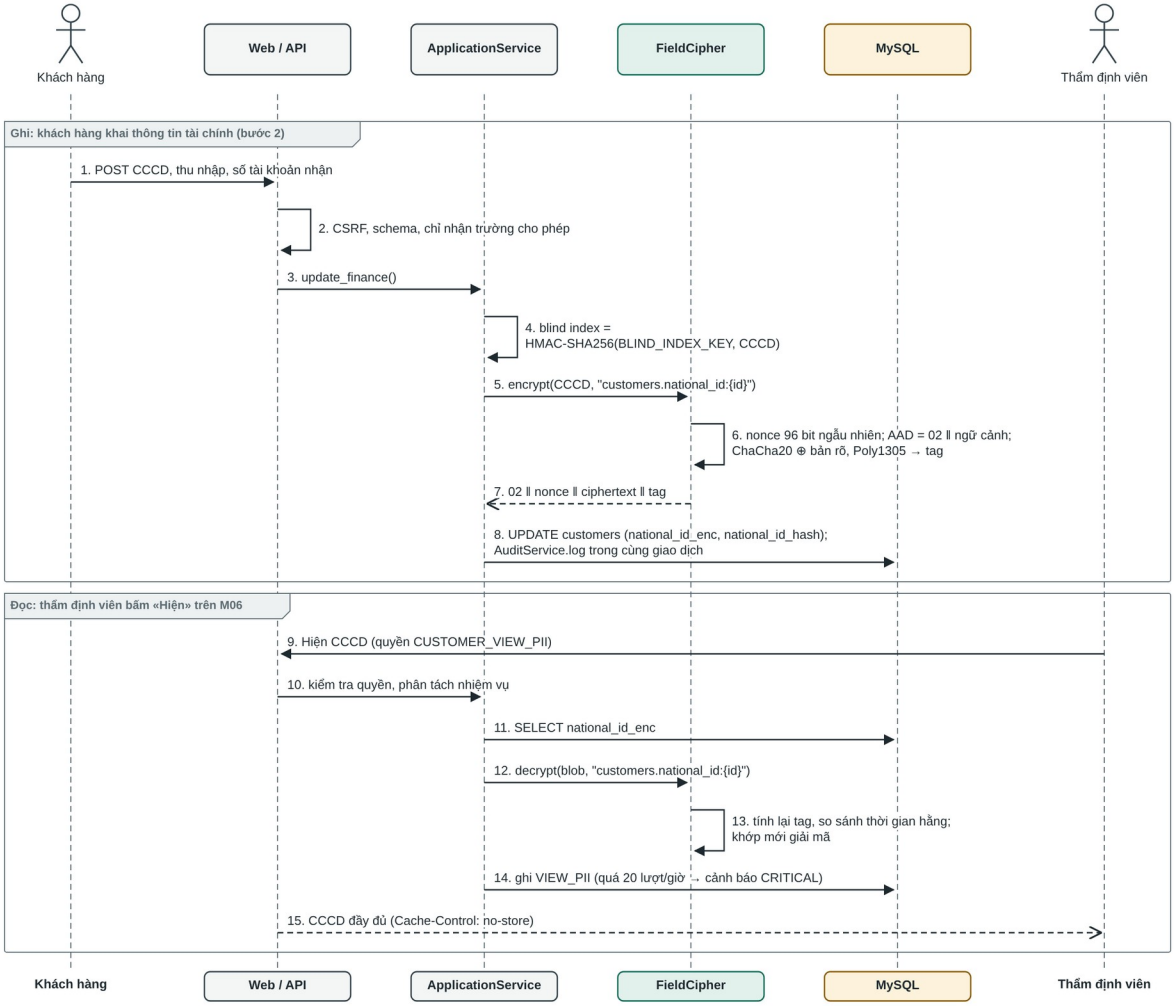
Hình 3.3. Sơ đồ tuần tự đăng nhập hai yếu tố

b) Đăng ký khóa TOTP – trao đổi khóa bí mật dùng chung (Hình 3.4). Đây là bước phân phối khóa duy nhất giữa máy chủ và người dùng. Khóa 160 bit sinh bằng CSPRNG, được mã hóa bằng ChaCha20-Poly1305 trước khi lưu, rồi trao cho nhân viên qua mã QR đúng một lần. Việc đăng ký chỉ hoàn tất khi nhân viên chứng minh đã nhận đúng khóa bằng cách nhập một mã hợp lệ.



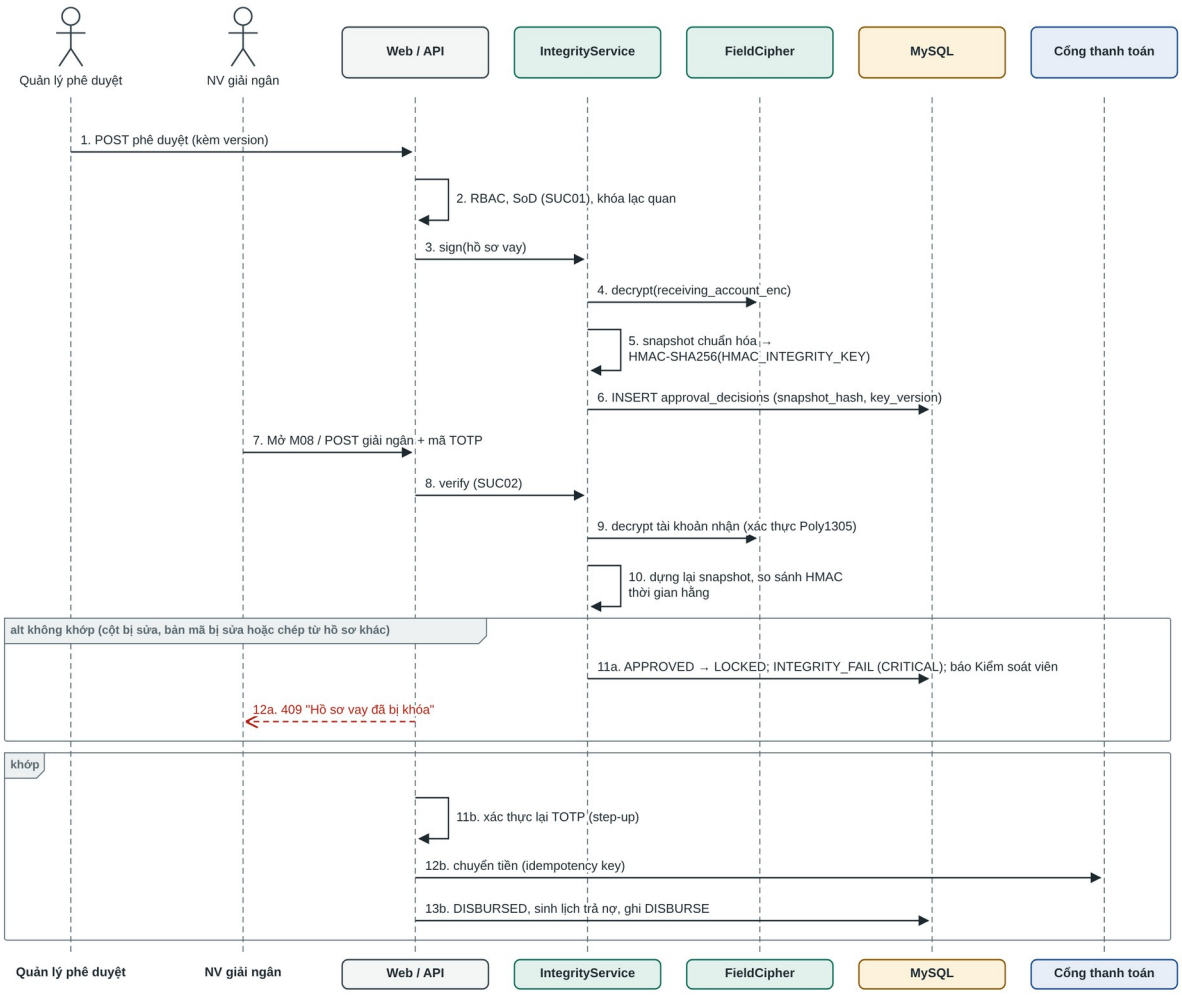
Hình 3.4. Sơ đồ tuần tự đăng ký khóa TOTP

c) Mã hóa và giải mã trường nhạy cảm (Hình 3.5). Luồng ghi: khi khách hàng khai thông tin tài chính, tầng nghiệp vụ tính blind index của CCCD rồi gọi `FieldCipher.encrypt` với ngữ cảnh `customers.national_id:{id}`; CSDL chỉ nhận `02 ‖ nonce ‖ bản mã ‖ tag`. Luồng đọc: khi thẩm định viên bấm "Hiện", hệ thống kiểm tra quyền `CUSTOMER_VIEW_PII` và phân tách nhiệm vụ, giải mã (xác thực trước), ghi `VIEW_PII` vào nhật ký rồi trả dữ liệu kèm `Cache-Control: no-store`.



Hình 3.5. Sơ đồ tuần tự mã hóa, giải mã trường nhạy cảm

d) Phê duyệt và giải ngân có kiểm tra toàn vẹn (Hình 3.6). Khi phê duyệt, `IntegrityService` giải mã tài khoản nhận, ghép snapshot chuẩn hóa và ký bằng HMAC-SHA256. Trước khi giải ngân, snapshot được dựng lại từ chính các cột hiện tại của hồ sơ và so sánh trong thời gian hằng. Nếu cột nào bị sửa, hoặc bản mã tài khoản nhận bị sửa hay bị chép từ hồ sơ khác (thẻ Poly1305 sai), hồ sơ chuyển sang `LOCKED`, nhật ký ghi `INTEGRITY_FAIL` mức `CRITICAL`, Kiểm soát viên và Quản lý phê duyệt được báo, và không có lệnh chuyển tiền nào được gửi đi.



Hình 3.6. Sơ đồ tuần tự phê duyệt và giải ngân có kiểm tra toàn vẹn

3.4. Hiện thực hóa các cơ chế bảo mật

3.4.1. Quy trình sinh và lưu trữ khóa an toàn

Bước 1 – Sinh bí mật gốc. Người vận hành sinh bốn bí mật độc lập bằng CSPRNG của hệ điều hành, mỗi bí mật 48 byte ngẫu nhiên (384 bit):

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Bước 2 – Lưu ngoài CSDL và ngoài mã nguồn. Các bí mật được đặt trong tệp `.env` (đã có trong `.gitignore`) hoặc Docker secret; `docker-compose.yml` bắt buộc khai báo cả bốn (`${DATA_ENC_KEY:?}`) nên thiếu khóa thì container không khởi động. Tệp `.env.example` chỉ chứa tên biến, không có giá trị. Tài khoản MySQL của ứng dụng không có quyền DDL và chỉ có `SELECT, INSERT` trên hai bảng chỉ ghi thêm.

Bước 3 – Dẫn xuất khóa làm việc khi khởi tạo `FieldCipher`. Mã thật trong `src/loan_system/security/crypto.py`:

```
FORMAT_AES_GCM = 1
FORMAT_CHACHA20_POLY1305 = 2
CURRENT_FORMAT = FORMAT_CHACHA20_POLY1305
NONCE_BYTES = 12  # 96 bit, như nhau ở cả hai định dạng
_CHACHA20_KEY_INFO = b"loan_system/field-encryption/chacha20-poly1305"

def _derive_key(secret: str, info: bytes) -> bytes:
    """Khóa 256 bit cho một mục đích (HKDF-SHA256, RFC 5869)."""
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
info=info).derive(secret.encode())

class FieldCipher:
    def __init__(self, secret: str) -> None:
        self._aeads = {
            FORMAT_CHACHA20_POLY1305: ChaCha20Poly1305(_derive_key(secret,
_CHACHA20_KEY_INFO)),
            FORMAT_AES_GCM: AESGCM(_legacy_key(secret)),  # SHA-
256(secret), chỉ còn để giải mã
        }
```

Bước 4 – Khóa theo người dùng. Khóa TOTP của từng nhân viên được sinh bằng `secrets.token_bytes(20)` (160 bit, độ dài khuyến nghị cho HMAC-SHA1), mã hóa bằng `FieldCipher` với ngữ cảnh `users.totp_secret:{user_id}` rồi mới lưu vào cột `users.totp_secret_enc`.

Bước 5 – Xoay vòng. Với khóa toàn vẹn: tăng `HMAC_INTEGRITY_KEY_VERSION`, chuyển khóa cũ vào `HMAC_INTEGRITY_OLD_KEYS` (JSON `{"1": "khóa-cũ"}`); quyết định phê duyệt cũ vẫn kiểm tra được nhờ `key_version` lưu kèm. Với thuật toán mã hóa: byte định dạng cho phép thêm thuật toán mới mà dữ liệu cũ vẫn đọc được, đúng như cách nhóm vừa chuyển từ AES-256-GCM sang ChaCha20-Poly1305 mà **không cần migration**.

3.4.2. Hiện thực mã hóa dữ liệu tĩnh (Data at Rest) và dữ liệu truyền tải (Data in Transit)

a) Dữ liệu tĩnh: mã hóa mức trường. Mỗi trường nhạy cảm được mã hóa riêng, với ngữ cảnh gắn chặt vào bảng, cột và khóa chính của bản ghi:

Bảng 3.4. Các trường được mã hóa và ngữ cảnh AAD

Bảng.cột	Dữ liệu	Ngữ cảnh (AAD = `02` ‖ ngữ cảnh)
`customers.national_id_enc`	Số CCCD	`customers.national_id:{customer_id}`
`customers.monthly_income_enc`	Thu nhập hằng tháng	`customers.monthly_income:{customer_id}`
`loan_applications.receivi`	Tài khoản nhận tiền giải ngân	`loan_applications.receiving_account:{application_id}`

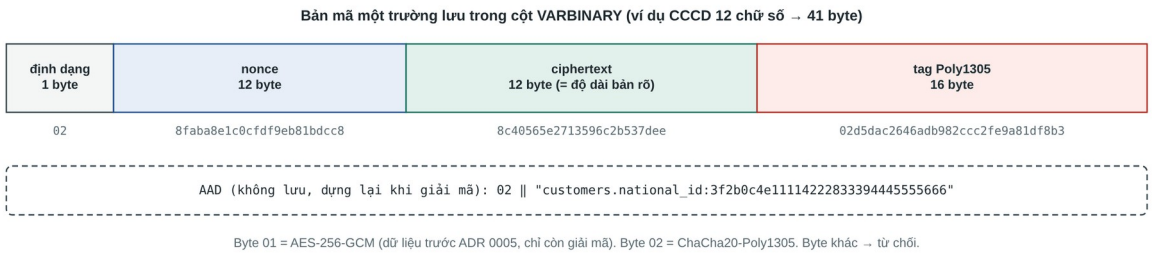
Bảng.cột	Dữ liệu	Ngữ cảnh (AAD = `02` ‖ ngữ cảnh)
`ng_account_enc`		
`disbursements.receiving_account_enc`	Tài khoản nhận trên lệnh giải ngân	`disbursements.receiving_account:{disbursement_id}`
`users.totp_secret_enc`	Khóa TOTP của nhân viên	`users.totp_secret:{user_id}`
`otp_challenges.payload_enc`	Dữ liệu chờ khách hàng xác nhận OTP (xóa khi xong)	`otp_challenges.payload:{challenge_id}`

Hàm mã hóa và giải mã của `FieldCipher`:

```
def _associated_data(fmt: int, context: str) -> bytes:
    if fmt == FORMAT_AES_GCM:
        return context.encode()
    return bytes([fmt]) + context.encode()      # byte định dạng cũng được xác thực

def encrypt(self, plaintext: str, *, context: str) -> bytes:
    nonce = os.urandom(NONCE_BYTES)             # nonce 96 bit mới cho mỗi lần mã hóa
    sealed = self._aeads[CURRENT_FORMAT].encrypt(
        nonce, plaintext.encode(), _associated_data(CURRENT_FORMAT, context)
    )
    return bytes([CURRENT_FORMAT]) + nonce + sealed   # 02 ‖ N ‖ C ‖ T

def decrypt(self, data: bytes, *, context: str) -> str:
    aead = self._aeads.get(data[0]) if data else None
    if aead is None or len(data) <= 1 + NONCE_BYTES:
        raise DecryptionFailed
    nonce, sealed = data[1 : 1 + NONCE_BYTES], data[1 + NONCE_BYTES :]
    try:
        return aead.decrypt(nonce, sealed, _associated_data(data[0], context)).decode()
    except InvalidTag as exc:                   # thư viện kiểm tra tag trước khi giải mã
        raise DecryptionFailed from exc
```



Hình 3.7. Định dạng bản mã lưu trong cơ sở dữ liệu

Ba quyết định thiết kế đáng chú ý:

1. **Ngữ cảnh trong AAD chống tấn công tráo bản mã.** Kẻ có quyền ghi CSDL không đọc được số tài khoản, nhưng có thể *chép* bản mã số tài khoản của mình sang hồ sơ của người khác để chuyển hướng tiền giải ngân. Vì AAD chứa `id` của hồ sơ, bản mã chép sang sẽ không qua được kiểm tra thẻ (thử nghiệm ở mục 3.5.3).
2. **Byte định dạng nằm trong AAD.** Đổi `02` thành `01` để ép hệ thống giải mã bằng thuật toán khác (tấn công hạ cấp) sẽ bị từ chối.
3. **Blind index để tra cứu mà không giải mã hàng loạt.** Cột `national_id_hash = HMAC-SHA256(BLIND_INDEX_KEY, CCCD)` cho phép tìm khách hàng theo CCCD và kiểm tra trùng CCCD khi đăng ký mà không phải giải mã toàn bộ bảng; khóa riêng nên kẻ lấy CSDL không thể băm thử cả không gian CCCD để dò ngược.

Dữ liệu thật trong CSDL sau khi nạp dữ liệu mẫu (truy vấn bằng tài khoản `root` của MySQL) cho thấy mọi bản mã đều bắt đầu bằng byte `02`, dài 41 byte cho CCCD 12 chữ số, và không có dấu vết bản rõ:

```
mysql> SELECT HEX(national_id_enc), LENGTH(national_id_enc),
national_id_hash FROM customers LIMIT 2;
028E275CF0C86C51B894A6C102435524C7BD73013A4AF13B108440CA8BB2406188E7E9D6392
202C6FB  41  0c56bb4a…f37e
02E0FFF1E3066D2333C873A2BFC06504826B2BC8C6C6D89A1D9C8061DDD9615C824696EEF45
9F1F5F7  41  21951b48…f395
mysql> SELECT username, LEFT(password_hash, 40) FROM users LIMIT 1;
tindung1   $argon2id$v=19$m=65536,t=3,p=4$uTCnw0KA7
```

Khi hiển thị, giá trị được giải mã trong bộ nhớ rồi che theo SR07 (`079******234`); chỉ người có quyền `CUSTOMER_VIEW_PII` mới xem được đầy đủ, và mỗi lần xem đều được ghi nhật ký.

b) Dữ liệu truyền tải. Mọi cookie phiên và cookie CSRF đặt cờ `Secure` nên trình duyệt chỉ gửi chúng qua HTTPS. Thiết kế triển khai đặt Nginx kết thúc TLS 1.3 ở DMZ; trong phạm vi đề tài, nhóm thử nghiệm TLS 1.3 trực tiếp trên Uvicorn với chứng chỉ ECDSA P-256 tự ký, và ép máy khách chỉ chào bộ mã ChaCha20-Poly1305:

```
$ uvicorn loan_system.app:app --port 8443 --ssl-keyfile tls.key --ssl-
certfile tls.crt
$ openssl s_client -connect localhost:8443 -tls1_3 -ciphersuites
TLS_CHACHA20_POLY1305_SHA256
Peer signature type: ECDSA
Server Temp Key: X25519, 253 bits
New, TLSv1.3, Cipher is TLS_CHACHA20_POLY1305_SHA256
$ curl -v --tlsv1.3 --tls13-ciphers TLS_CHACHA20_POLY1305_SHA256
https://localhost:8443/app/login
* SSL connection using TLSv1.3 / TLS_CHACHA20_POLY1305_SHA256 / X25519 /
id-ecPublicKey
< HTTP/1.1 200 OK
```

Kết quả cho thấy cùng một thuật toán bảo vệ dữ liệu ở cả hai trạng thái: khi truyền qua mạng (bản ghi TLS 1.3 mã hóa bằng ChaCha20-Poly1305 với khóa phiên thỏa thuận qua X25519) và khi nằm trong CSDL (mã hóa mức trường). Kết nối giữa ứng dụng và MySQL hiện đi trong mạng Docker nội bộ, chưa bật TLS (xem mục 4.2).

3.4.3. Xác thực tính toàn vẹn và chống giả mạo

Hệ thống chống giả mạo theo nhiều lớp, sao cho kẻ vượt qua được một lớp vẫn bị lớp sau phát hiện:

1. **Lớp CSDL – đặc quyền tối thiểu và trigger.** Tài khoản ứng dụng chỉ có `SELECT, INSERT` trên `audit_logs` và `approval_decisions`. Trigger `trg_loan_applications_approved_immutable` chặn mọi `UPDATE` làm đổi mã hồ sơ, số tiền, kỳ hạn, lãi suất, hạn mức hay bản mã tài khoản nhận của hồ sơ đã ở trạng thái `APPROVED`, `DISBURSED` hoặc `LOCKED` (BR07).
2. **Lớp mật mã trên từng trường – thẻ Poly1305.** Mọi sửa đổi trên bản mã, kể cả chép nguyên một bản mã hợp lệ từ bản ghi khác, làm việc kiểm tra thẻ thất bại.
3. **Lớp nghiệp vụ – HMAC snapshot (SR08, SUC02).** Snapshot ghép các trường quan trọng theo thứ tự cố định: `id | mã hồ sơ | số tiền yêu cầu | hạn mức duyệt | kỳ hạn duyệt | lãi suất | tài khoản nhận (giải mã trong bộ nhớ)`, ký bằng HMAC-SHA256 lúc phê duyệt và kiểm tra lại trước khi giải ngân:

```
def verify(self, application: LoanApplication) -> bool:
    decision = self.approving_decision(application)
    snapshot = self._snapshot(application)      # None nếu bản mã tài khoản
nhận không giải mã được
    if decision is None or snapshot is None:
        return False
    return self._keyring.verify(                # hmac.compare_digest, đúng
phiên bản khóa
        snapshot.canonical(), Signature(decision.key_version,
decision.snapshot_hash)
    )
```

4. **Lớp truy vết – chuỗi băm nhật ký.** Kiểm soát viên bấm "Kiểm tra toàn vẹn chuỗi" để tính lại toàn bộ chuỗi; bản ghi đầu tiên bị sửa, xóa hay chèn được chỉ ra chính xác theo số thứ tự.
5. **Lớp ứng dụng web.** Token CSRF so sánh thời gian hằng; cookie `SameSite=Strict`; mọi biểu mẫu ghi đều kiểm tra token.

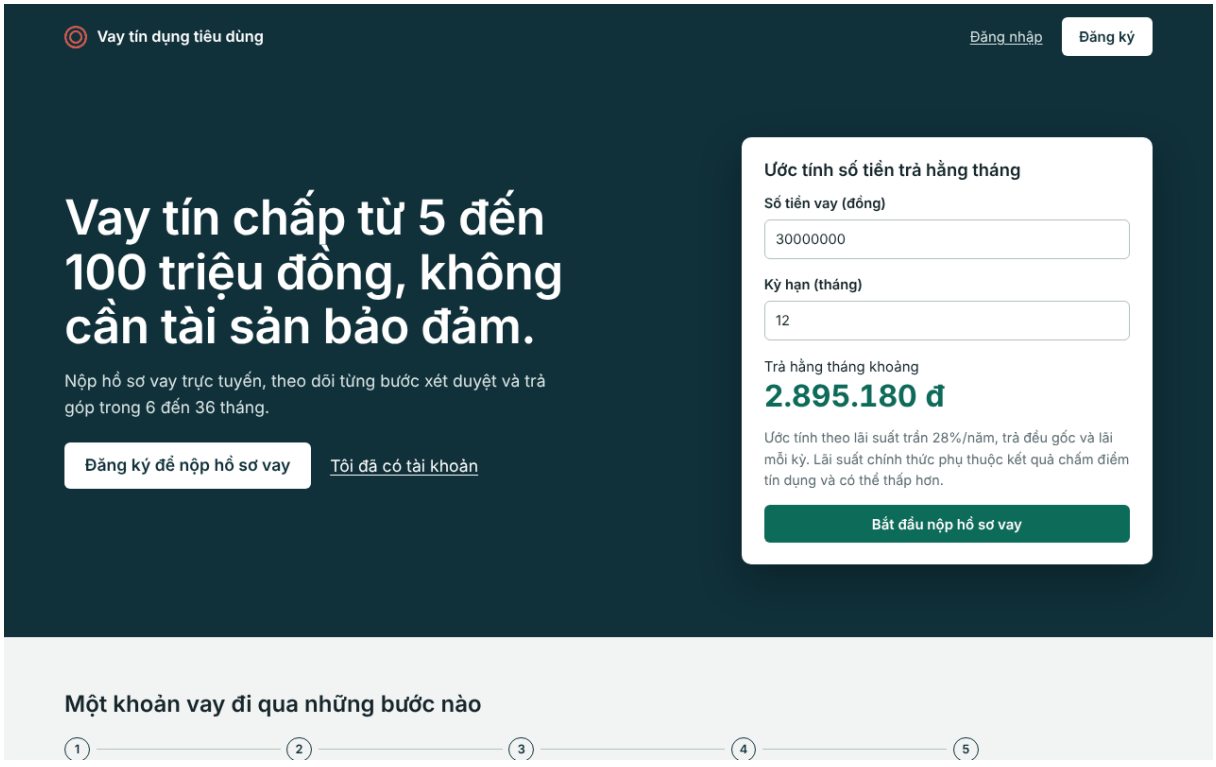
Lỗi phát hiện trong quá trình thực nghiệm và cách khắc phục. Khi chạy kịch bản tấn công tráo bản mã tài khoản nhận trên hệ thống thật, nhóm phát hiện màn hình giải ngân (M08) trả lỗi HTTP 500 thay vì khóa hồ sơ. Nguyên nhân: trang dựng phần hiển

thị hồ sơ (giải mã tài khoản nhận để che `012****789`) **trước** khi chạy kiểm tra toàn vẹn SUC02. Việc giải mã ném `DecryptionFailed` làm hỏng cả trang, nên hồ sơ không bị khóa và không có cảnh báo nào được ghi. Lỗi có sẵn từ trước, không do việc đổi thuật toán gây ra, và với AES-GCM cũng xảy ra y hệt. Nhóm đã sửa để phần hiển thị coi trường không xác thực được là "chưa có", còn việc phát hiện, khóa và cảnh báo vẫn do SUC02 đảm nhận. Nhóm cũng bổ sung hai bài kiểm thử hồi quy (qua REST API và qua trang web); cả hai thất bại trên mã cũ và đạt trên mã đã sửa. Đây là minh chứng cho giá trị của kiểm thử tấn công trên hệ thống thật: thuật toán đúng chưa đủ, luồng xử lý lỗi quanh thuật toán cũng phải đúng.

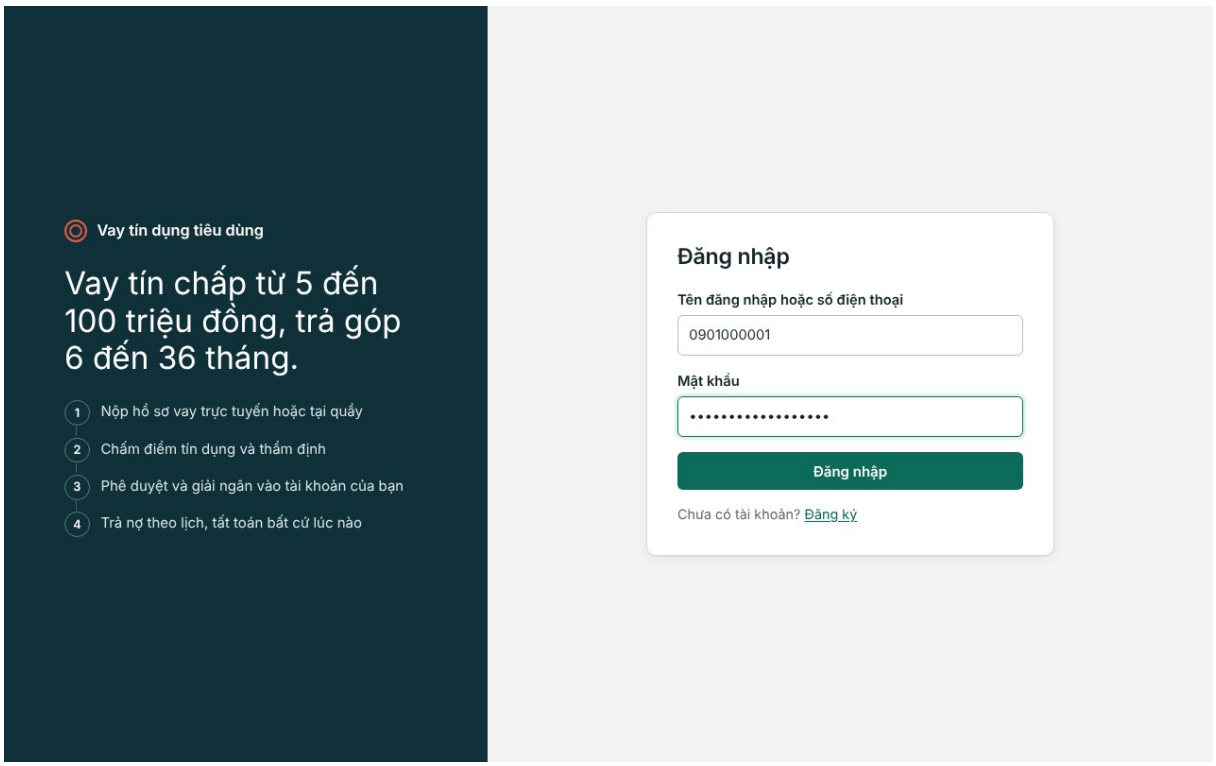
3.5. Kết quả thực nghiệm và giao diện chương trình

Dữ liệu dùng trong mục này được nạp bằng lệnh `seed_demo` đi qua chính REST API (6 nhân viên đủ vai trò, 6 khách hàng, 6 hồ sơ vay ở các trạng thái khác nhau), nên mọi trường đều được mã hóa và ký thật.

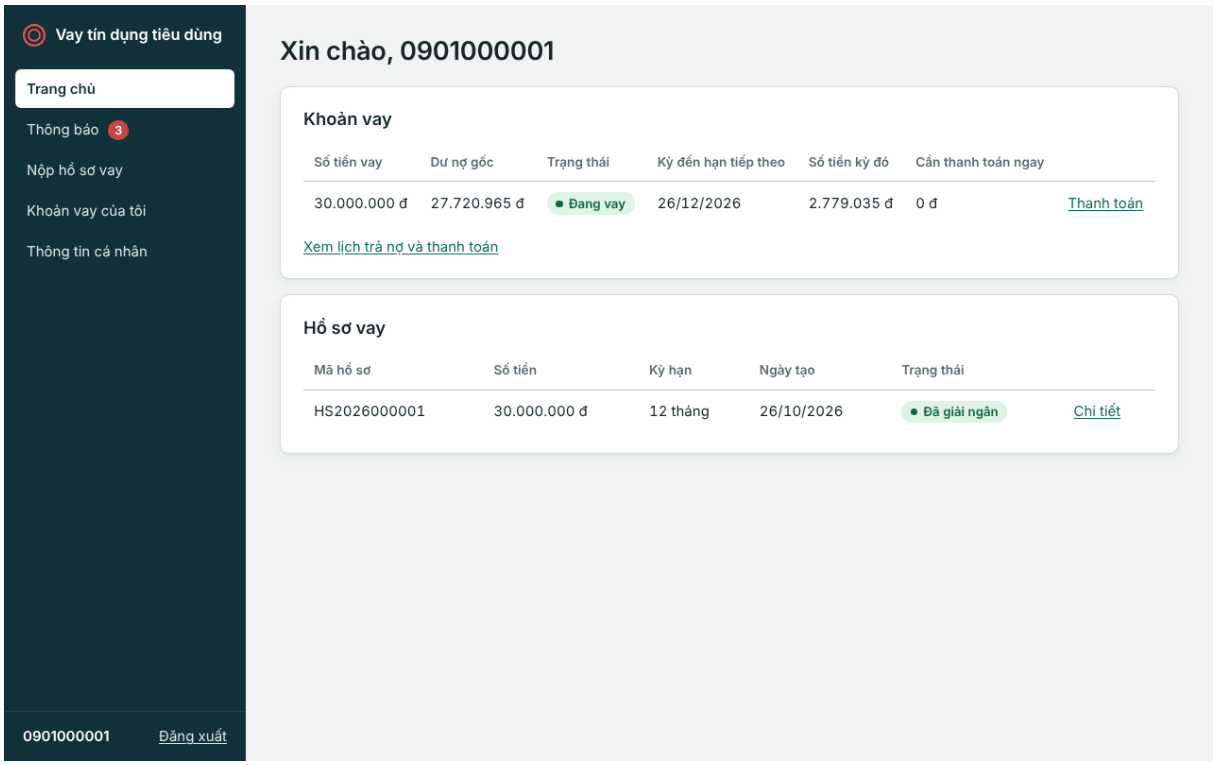
3.5.1. Giao diện chức năng người dùng (User Side)



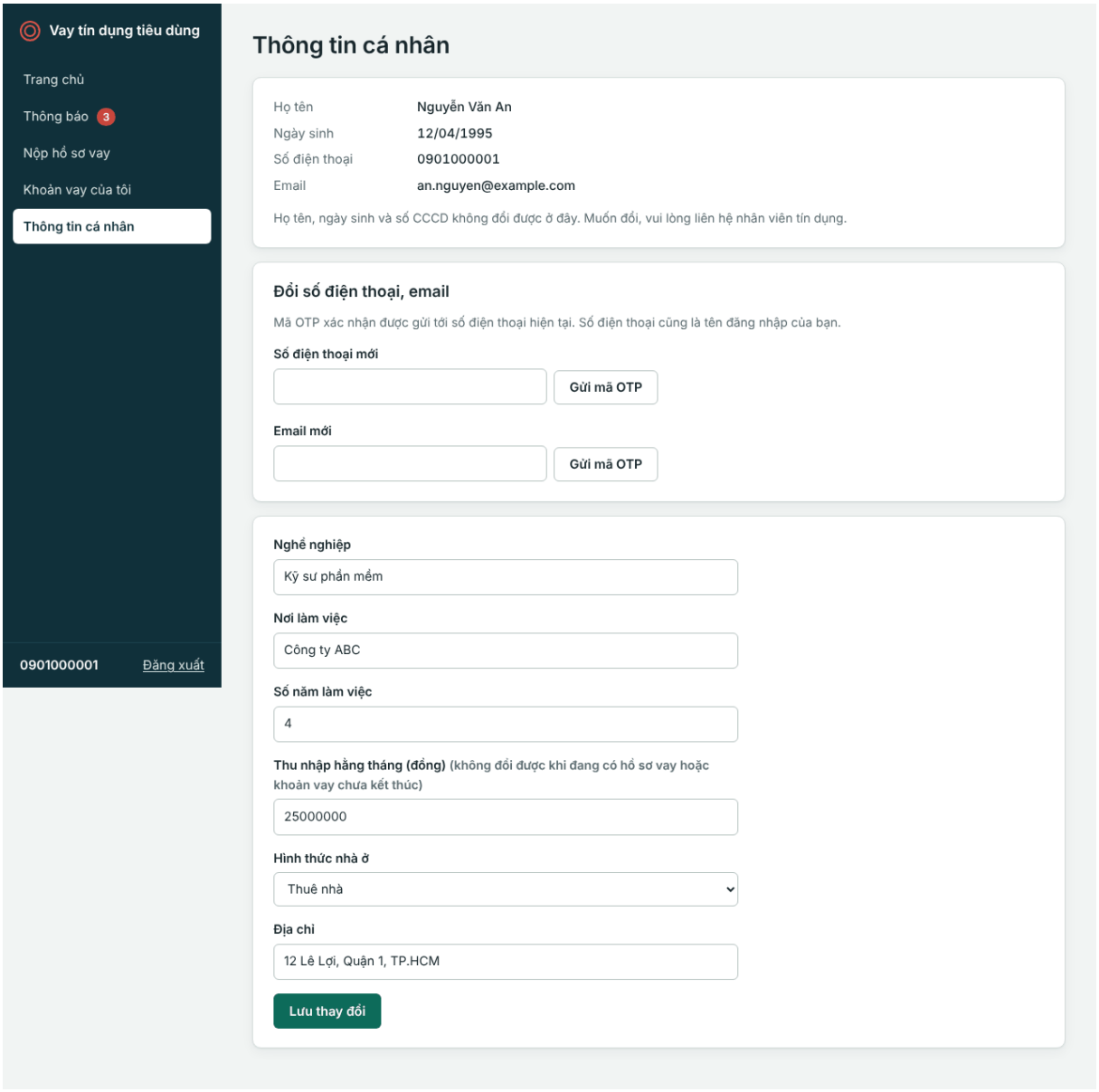
Hình 3.8. Trang giới thiệu của hệ thống



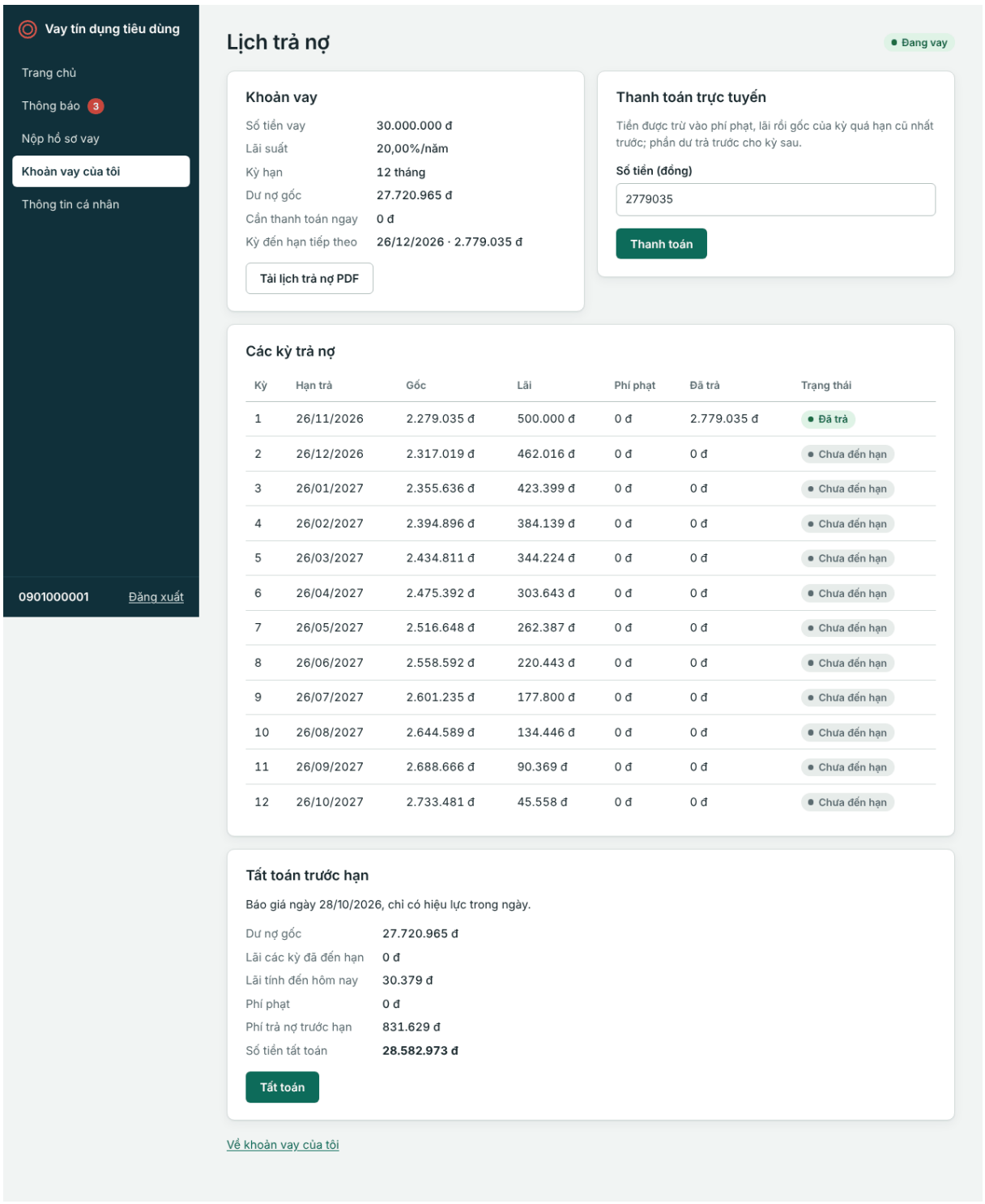
Hình 3.9. Đăng nhập khách hàng bằng số điện thoại và mật khẩu (Argon2id phía máy chủ)



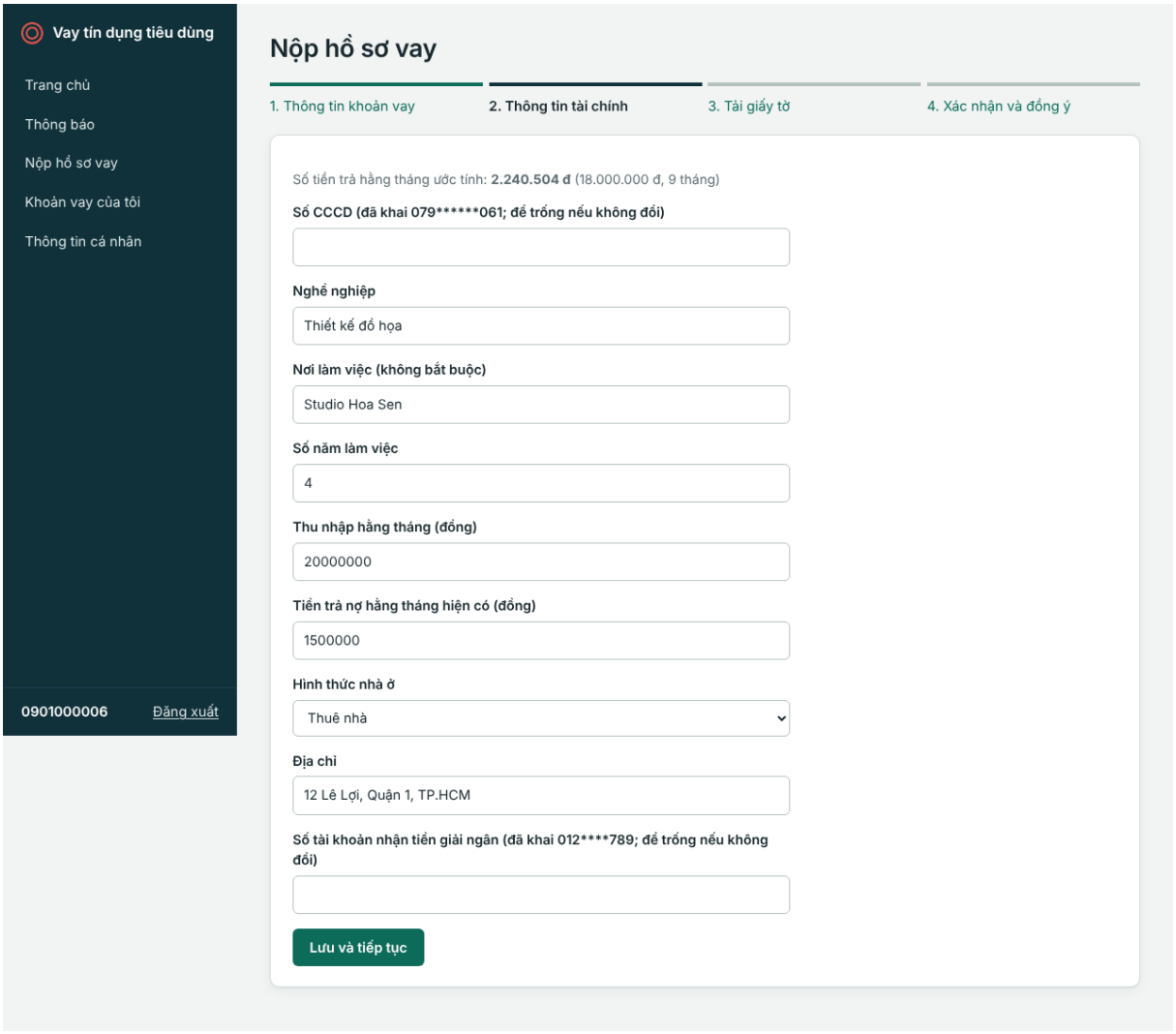
Hình 3.10. Trang chủ khách hàng: chỉ thấy hồ sơ vay của chính mình (SR04)



Hình 3.11. Thông tin cá nhân: thu nhập được giải mã cho chính chủ; đổi số điện thoại, email phải xác nhận OTP

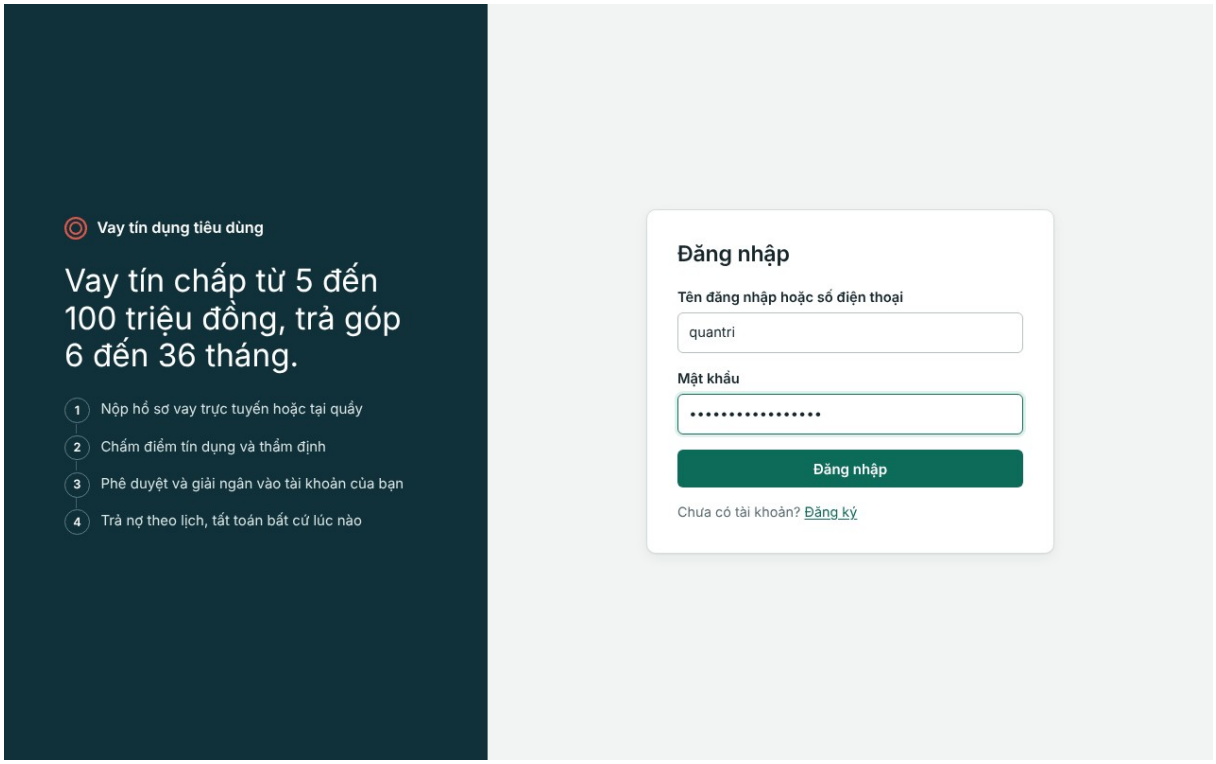


Hình 3.12. Chi tiết khoản vay và lịch trả nợ

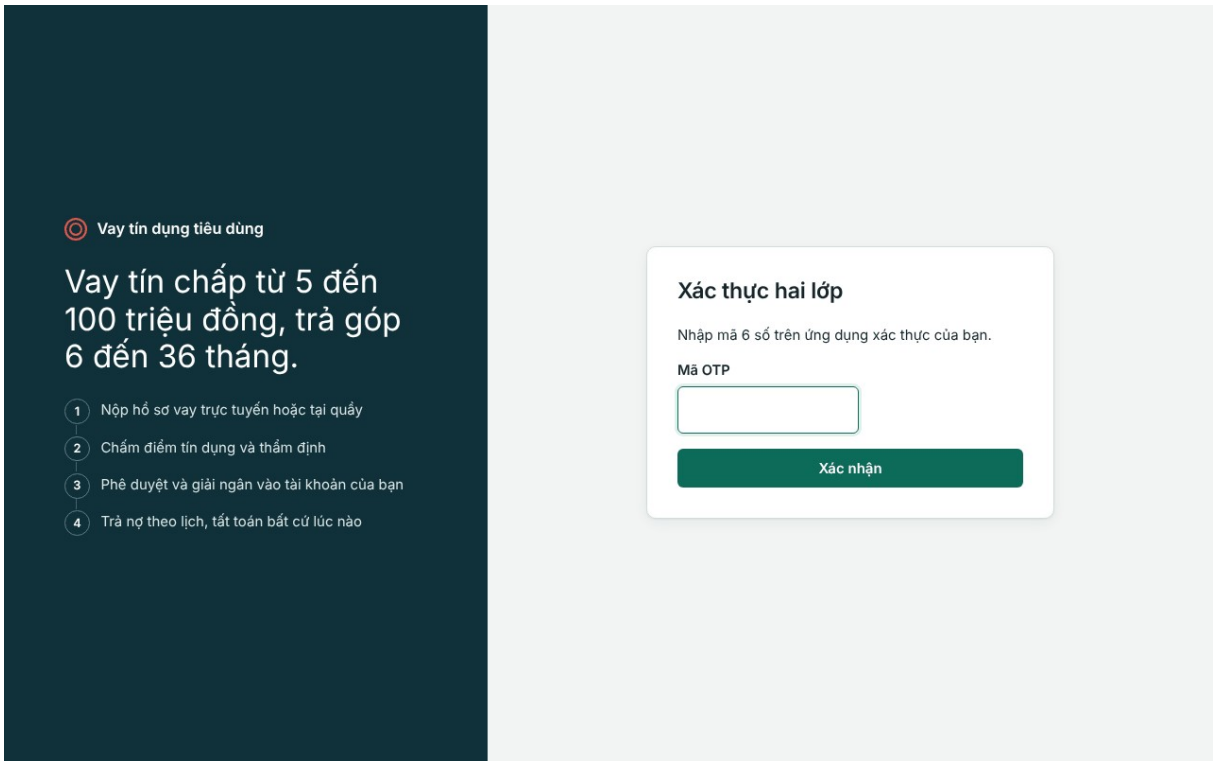


Hình 3.13. Bước 2 của biểu mẫu nộp hồ sơ: CCCD và tài khoản nhận đã khai chỉ hiện dạng che (`079******061`, `012****789`); giá trị mới được mã hóa bằng ChaCha20-Poly1305 trước khi lưu

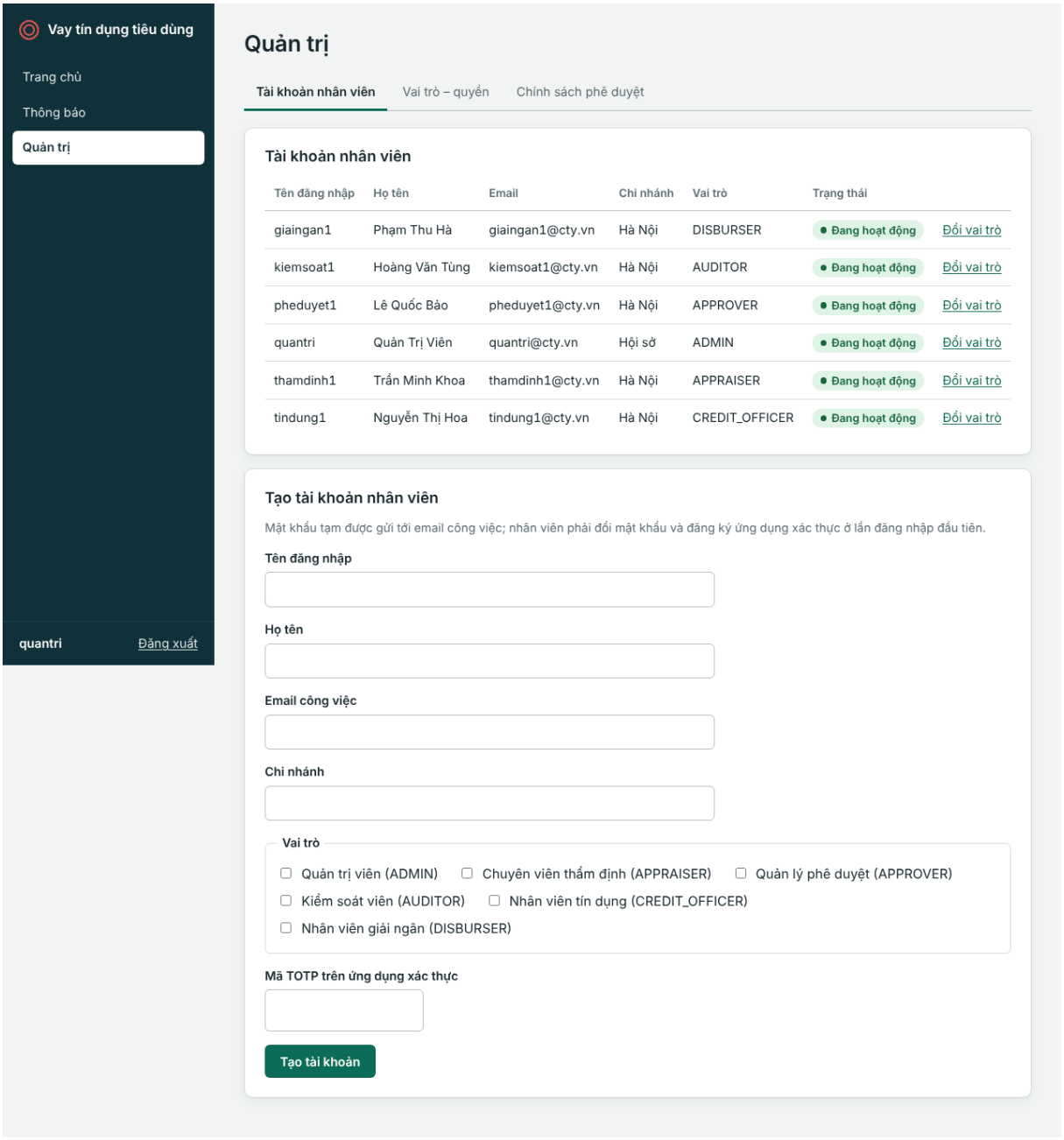
3.5.2. Giao diện quản trị hệ thống (Admin Side)



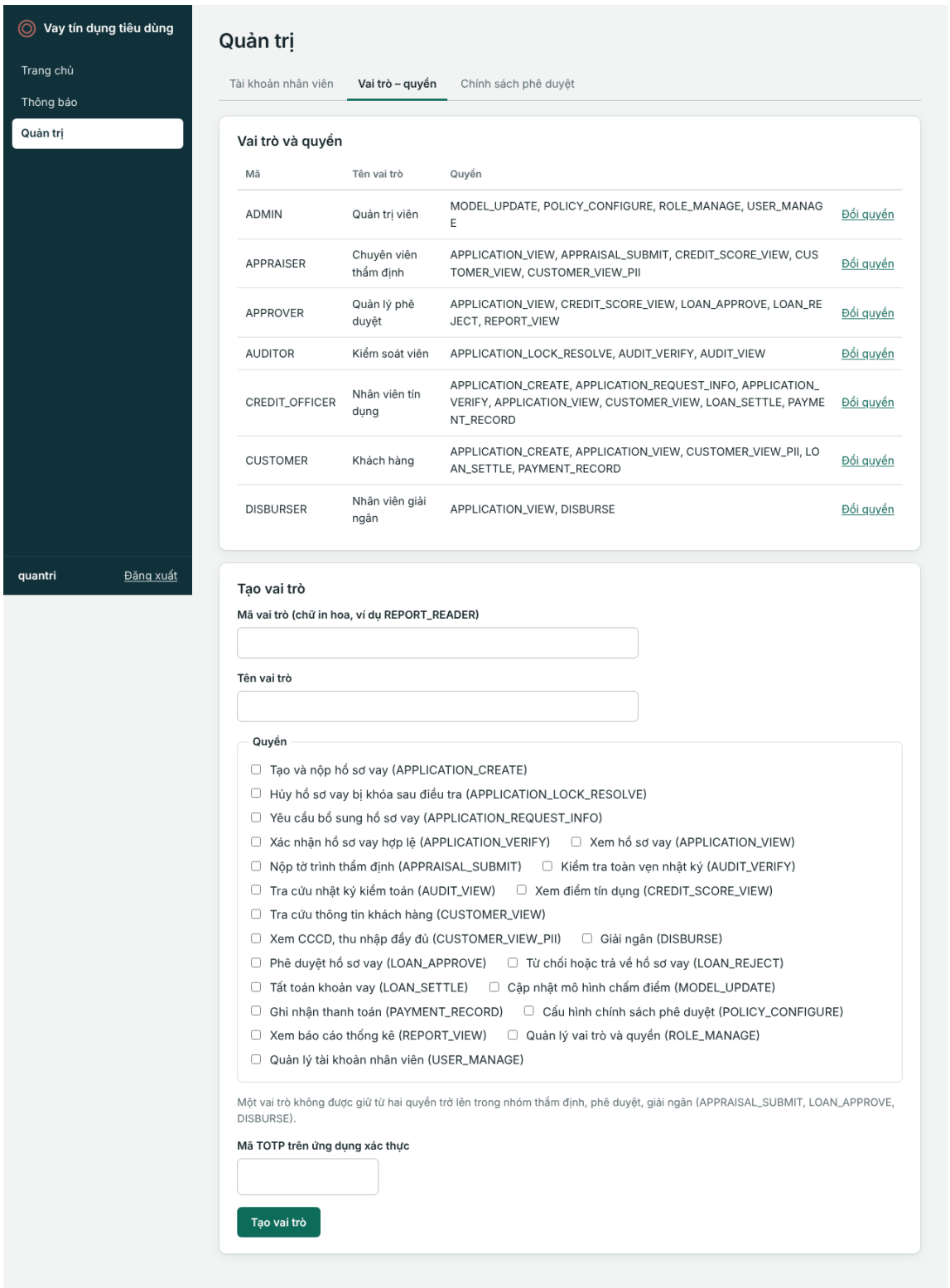
Hình 3.14. Nhân viên đăng nhập bằng mật khẩu (yếu tố thứ nhất)



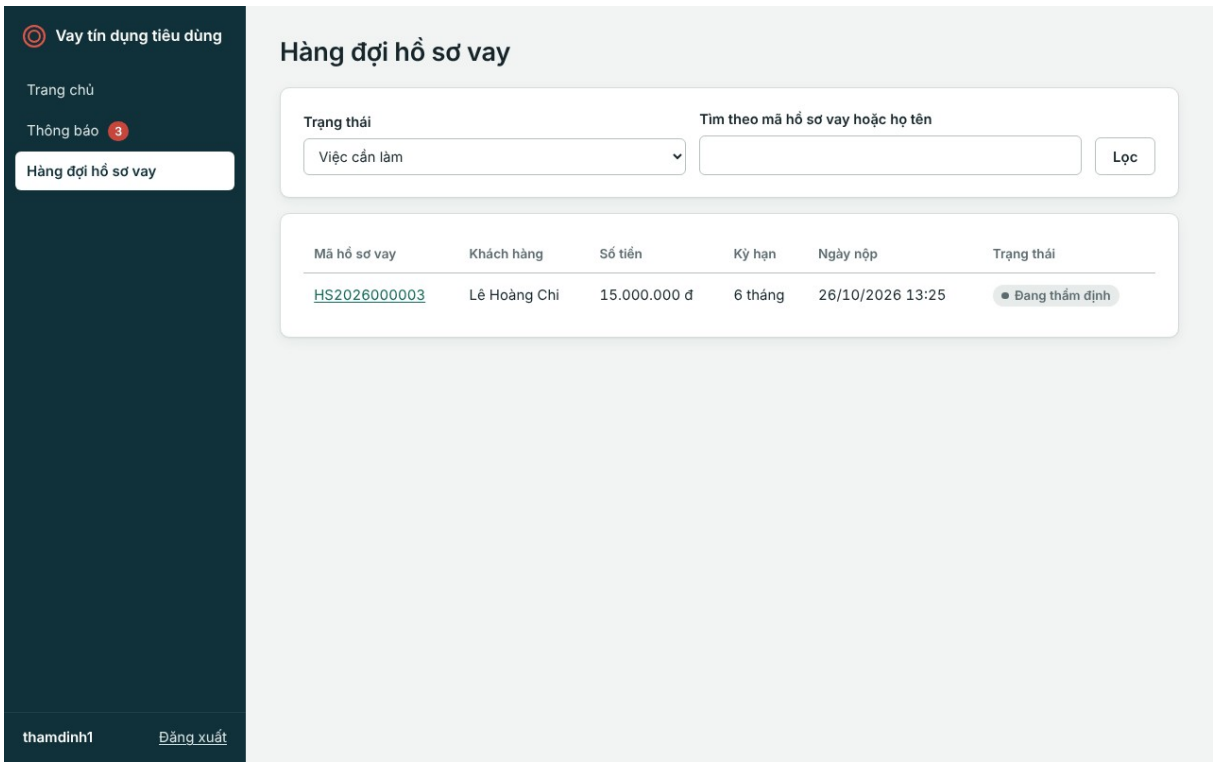
Hình 3.15. Nhập mã TOTP 6 số từ ứng dụng xác thực (yếu tố thứ hai)



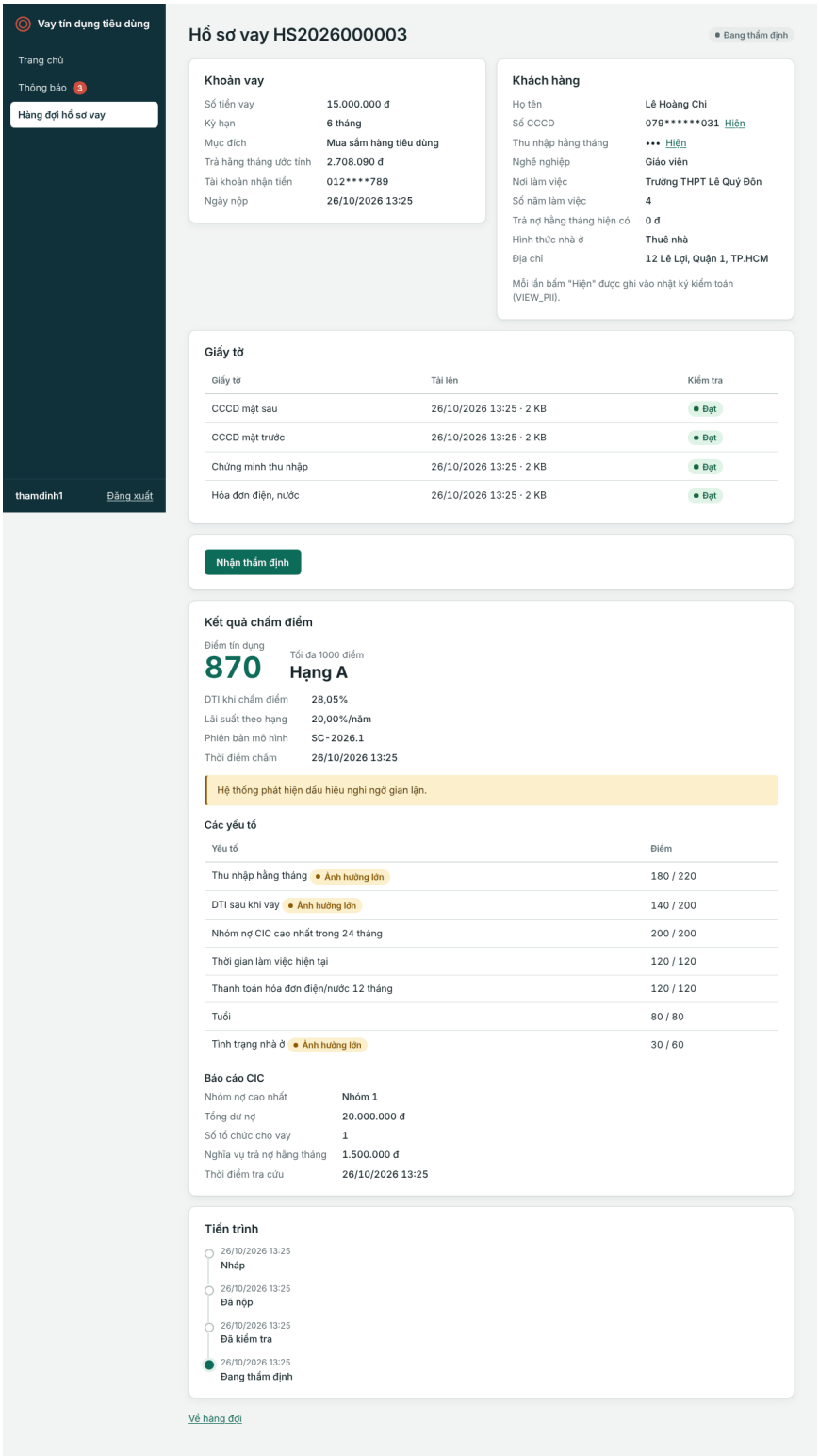
Hình 3.16. Quản trị viên quản lý tài khoản nhân viên; mọi thao tác ghi yêu cầu nhập lại TOTP



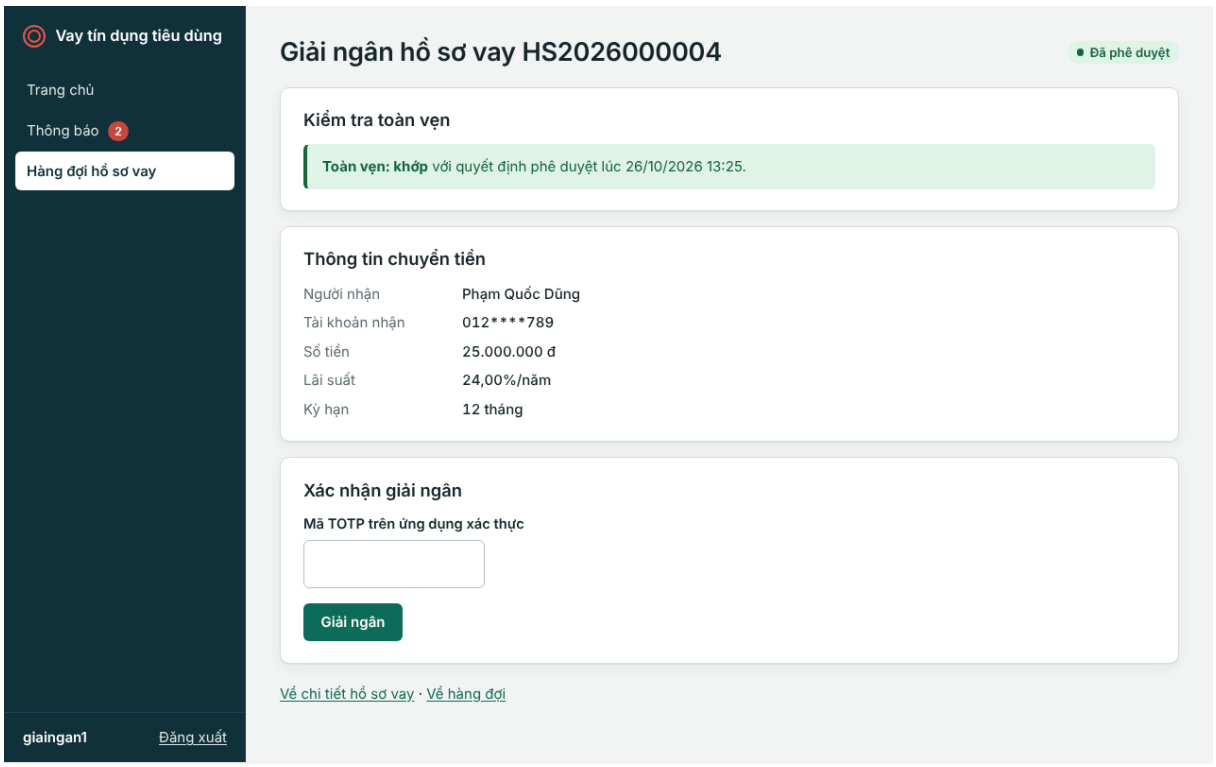
Hình 3.17. Ma trận vai trò – quyền (RBAC)



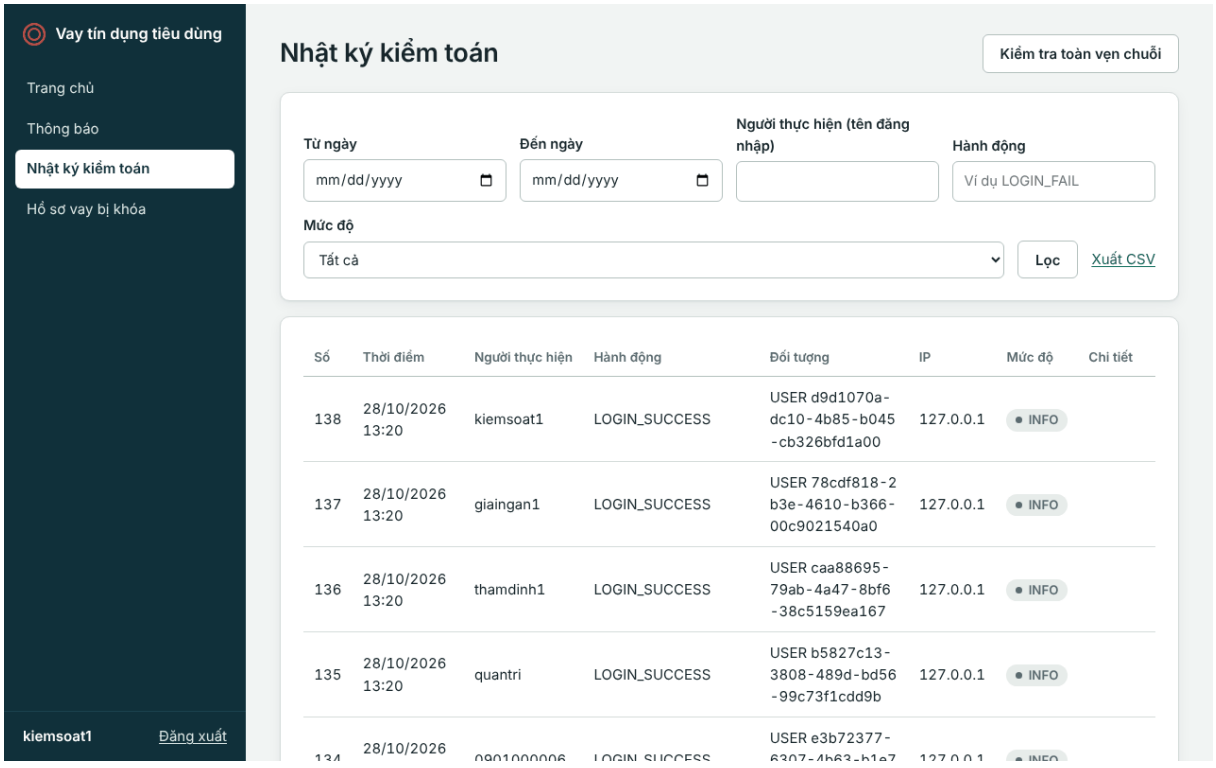
Hình 3.18. Hàng đợi hồ sơ vay của thẩm định viên



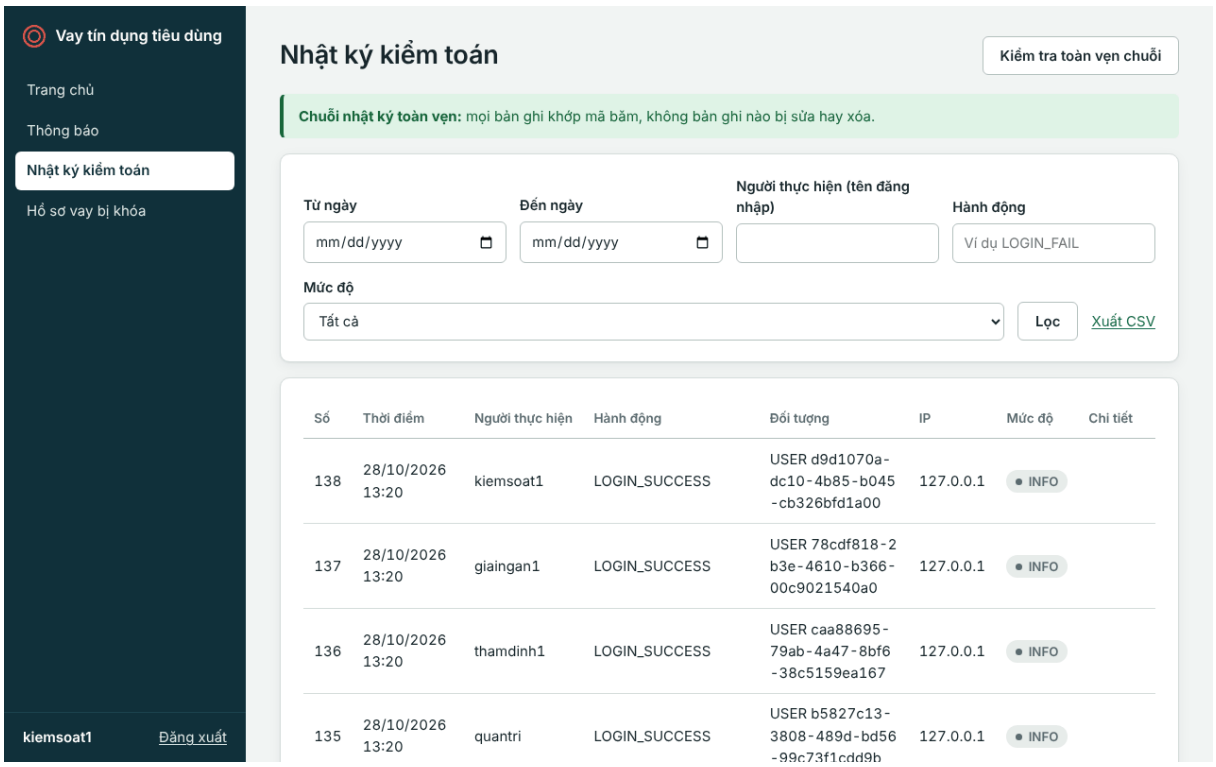
Hình 3.19. Màn hình thẩm định: CCCD, thu nhập che sẵn; bấm "Hiện" mới giải mã và ghi `VIEW_PII`



Hình 3.20. Màn hình giải ngân: "Toàn vẹn: khớp" với quyết định phê duyệt; tài khoản nhận hiện dạng che; giải ngân phải nhập TOTP



Hình 3.21. Nhật ký kiểm toán của Kiểm soát viên



Hình 3.22. Kết quả kiểm tra chuỗi băm khi chưa bị tấn công: "Chuỗi nhật ký toàn vẹn"

3.5.3. Thử nghiệm mã hóa, giải mã và đo đạc thời gian/hiệu năng

a) Kiểm chứng tính đúng của thuật toán. Script `scripts/chacha20_minh_hoa.py` cài đặt ChaCha20, Poly1305 và AEAD bằng Python thuần, rồi so với các vector thử chính thức của RFC 8439 và với thư viện `cryptography`:

Bảng 3.5. Kết quả kiểm chứng vector thử RFC 8439

Phép thử	Mục RFC 8439	Giá trị mong đợi (rút gọn)	Kết quả
Quarter-round	2.1.1	`ea2a92f4 cb1cf8ce 4581472e 5881c4bb`	Đạt
Hàm khối ChaCha20, 64 byte	2.3.2	`10f1e7e4d13b5915…a2503c4e`	Đạt
Mã hóa "Ladies and Gentlemen…" 114 byte	2.4.2	`6e2e359a2568f980…`; trùng 114/114 byte với thư viện	Đạt
Poly1305 "Cryptographic Forum Research Group"	2.5.2	tag `a8061dc1305136c6c22b8baf0c0127a9`	Đạt
AEAD ChaCha20-Poly1305	2.8.2	bản mã `d31a8d34648e60db…`, tag `1ae10b594f09e26a7e902ecbd0600691`	Đạt
50 bộ dữ liệu ngẫu nhiên	–	Trùng từng byte với	50/50

Phép thử	Mục RFC 8439	Giá trị mong đợi (rút gọn)	Kết quả
		`cryptography.ChaCha20Poly1305`	
Giải mã bản mã do `FieldCipher` của hệ thống sinh ra	–	Ra đúng `079095001234`	Đạt

b) Kiểm thử tự động của hệ thống. Toàn bộ bộ kiểm thử chạy trên MySQL 8.4 thật: **528/528 bài đạt** (khoảng 10 phút), `mypy --strict` không có lỗi trên 147 tệp nguồn. Riêng phần mã hóa trường có 12 bài kiểm thử tầng miền: mã hóa rồi giải mã đúng; cùng bản rõ cho bản mã khác nhau mỗi lần; bản mã bị sửa, sai ngữ cảnh, sai khóa đều bị từ chối; định dạng mới đúng 1 + 12 + n + 16 byte; dữ liệu AES-GCM cũ vẫn đọc được; đổi byte định dạng (0, 1, 3, 255) bị từ chối; blob rỗng hoặc bị cắt cụt bị từ chối.

c) Thử nghiệm tấn công giả mạo trên hệ thống thật. Nhóm đóng vai hai loại kẻ tấn công: (1) kẻ có tài khoản ứng dụng `loan_app`, (2) kẻ có tài khoản `root` của MySQL, gỡ được trigger.

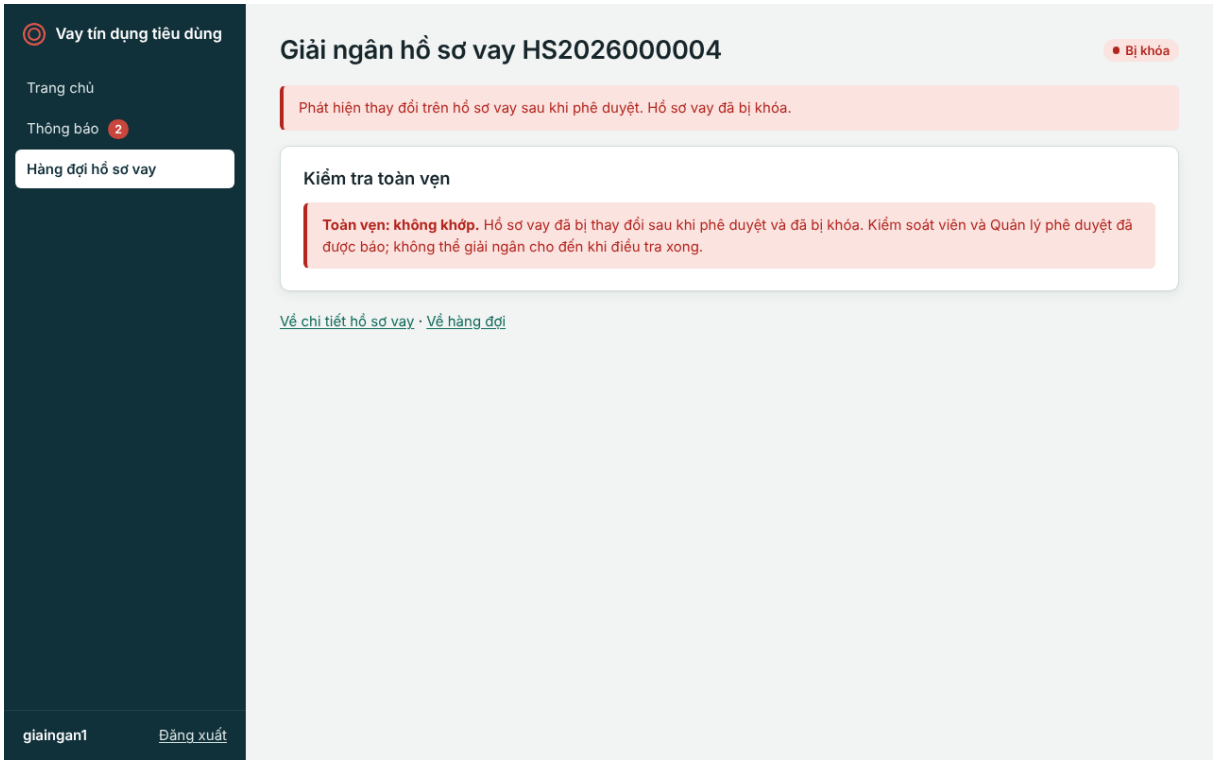
Bảng 3.6. Kịch bản tấn công giả mạo và kết quả

#	Kịch bản	Tác nhân	Lớp chặn	Kết quả quan sát
1	Đọc bảng `customers` để lấy CCCD	`root` (tương đương lộ tệp sao lưu)	Mã hóa ChaCha20-Poly1305	Chỉ thấy bản mã `02…`, không có khóa thì không giải mã được
2	Chép bản mã tài khoản nhận của HS2026000003 sang HS2026000004 (đã duyệt)	`loan_app`	Trigger BR07	`ERROR 1644 (45000): BR07: approved loan application is immutable`
3	Sửa, xóa bản ghi nhật ký số 57	`loan_app`	Đặc quyền tối thiểu	`ERROR 1142 (42000): UPDATE/DELETE command denied … for table 'audit_logs'`
4	Như #2 nhưng gỡ trigger trước rồi tạo lại	`root`	Thẻ Poly1305 + ngữ cảnh AAD → SUC02	Mở màn hình giải ngân: **409**, “Toàn vẹn: không khớp”, hồ sơ chuyển `Bị khóa`, nhật ký `INTEGRITY_FAIL` mức `CRITICAL`, Kiểm soát viên nhận thông báo (Hình 3.23 – 3.26)
5	Sửa `action` của bản ghi	`root`	Chuỗi băm	“Chuỗi nhật ký bị phá vỡ tại bản

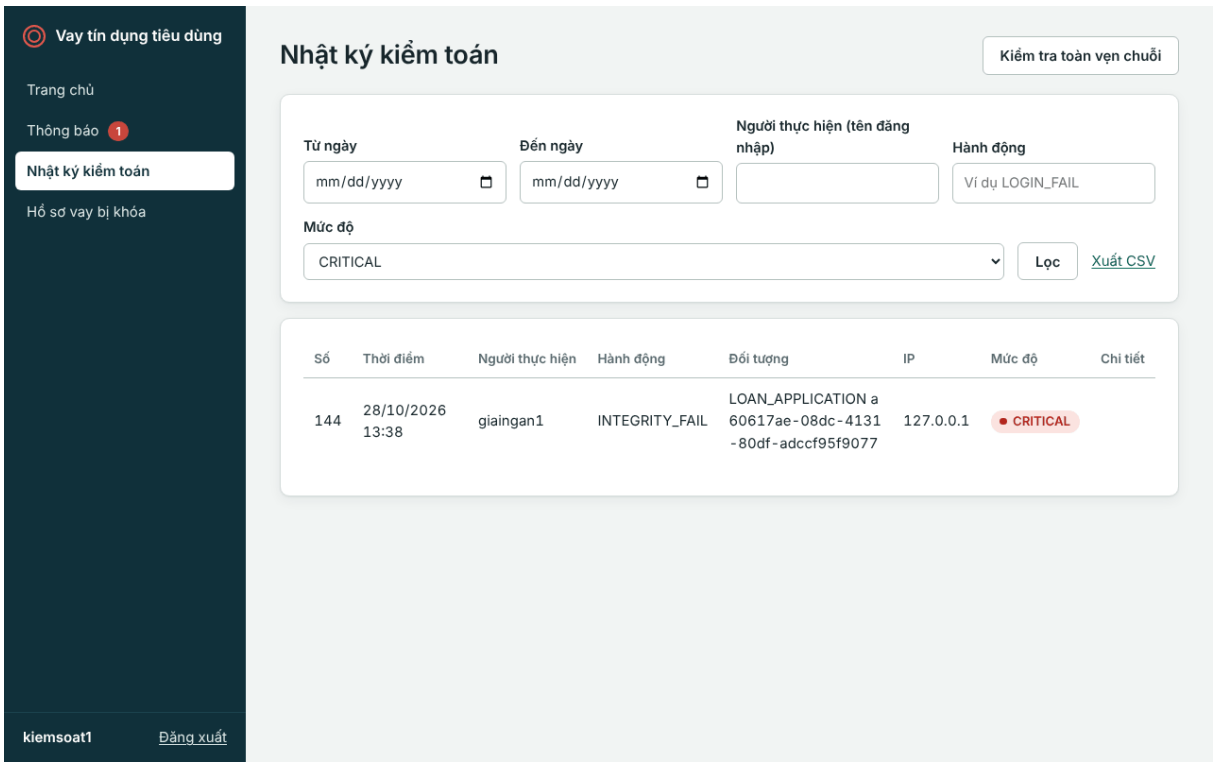
#	Kịch bản	Tác nhân	Lớp chặn	Kết quả quan sát
	nhật ký số 57 thành `LOGIN_FAIL`		SHA-256	ghi số 57” (Hình 3.27)
6	Lật 1 bit bản mã; đổi byte định dạng 02 → 01	Kiểm thử tự động	Thẻ Poly1305	`DecryptionFailed`, không trả ra bản rõ

Ở kịch bản 4, bản mã bị chép là một bản mã **hợp lệ** do chính hệ thống sinh ra, được mã hóa bằng đúng khóa. Kẻ tấn công không cần biết khóa mà vẫn có thể chuyển hướng tiền nếu hệ thống chỉ mã hóa mà không xác thực ngữ cảnh. Cơ chế AAD của ChaCha20-Poly1305 chặn đúng kiểu tấn công này:

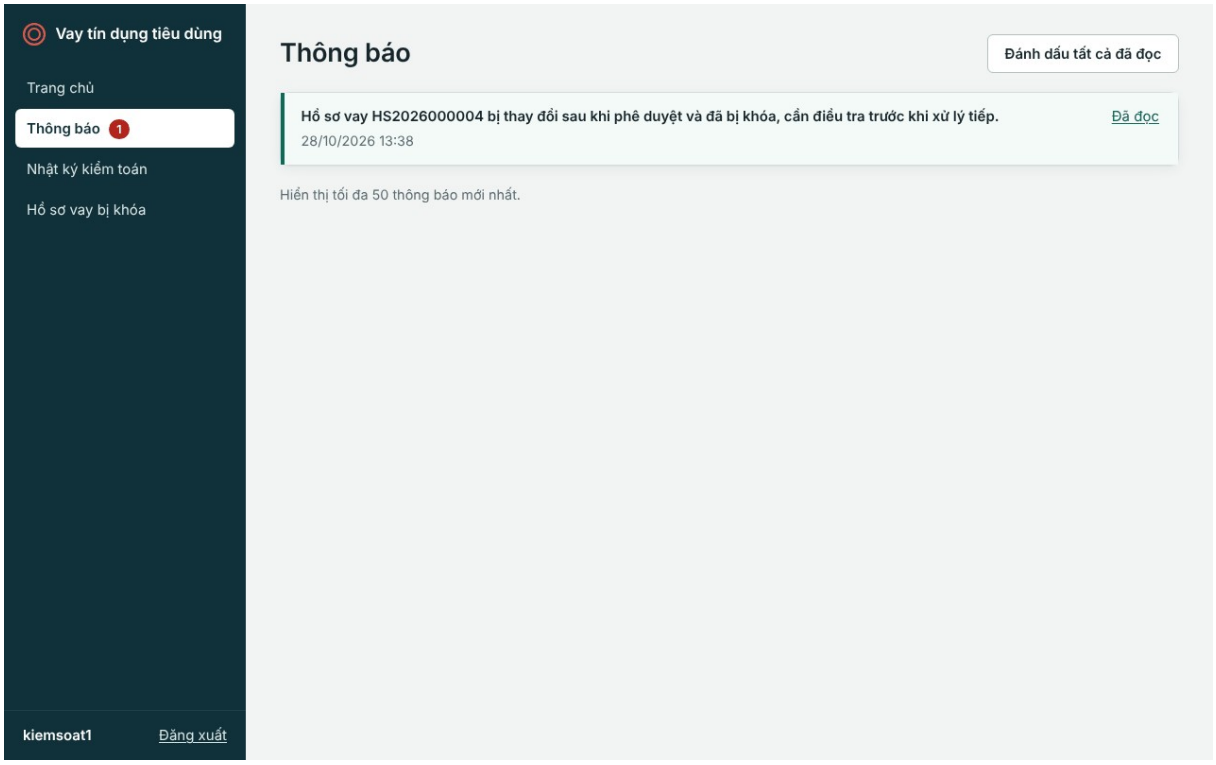
```
mysql> -- root: DROP TRIGGER …; UPDATE … SET receiving_account_enc = <bản
mã của HS2026000003>; CREATE TRIGGER …
code           status      HEX(receiving_account_enc)
HS2026000003   APPRAISING
02C122538C428D5E6DA8ED1384EB7C1FFE3F234A20C1B340FE6C874B8A0CEAB4D620C224B9F
968
HS2026000004   APPROVED
02C122538C428D5E6DA8ED1384EB7C1FFE3F234A20C1B340FE6C874B8A0CEAB4D620C224B9F
968
```



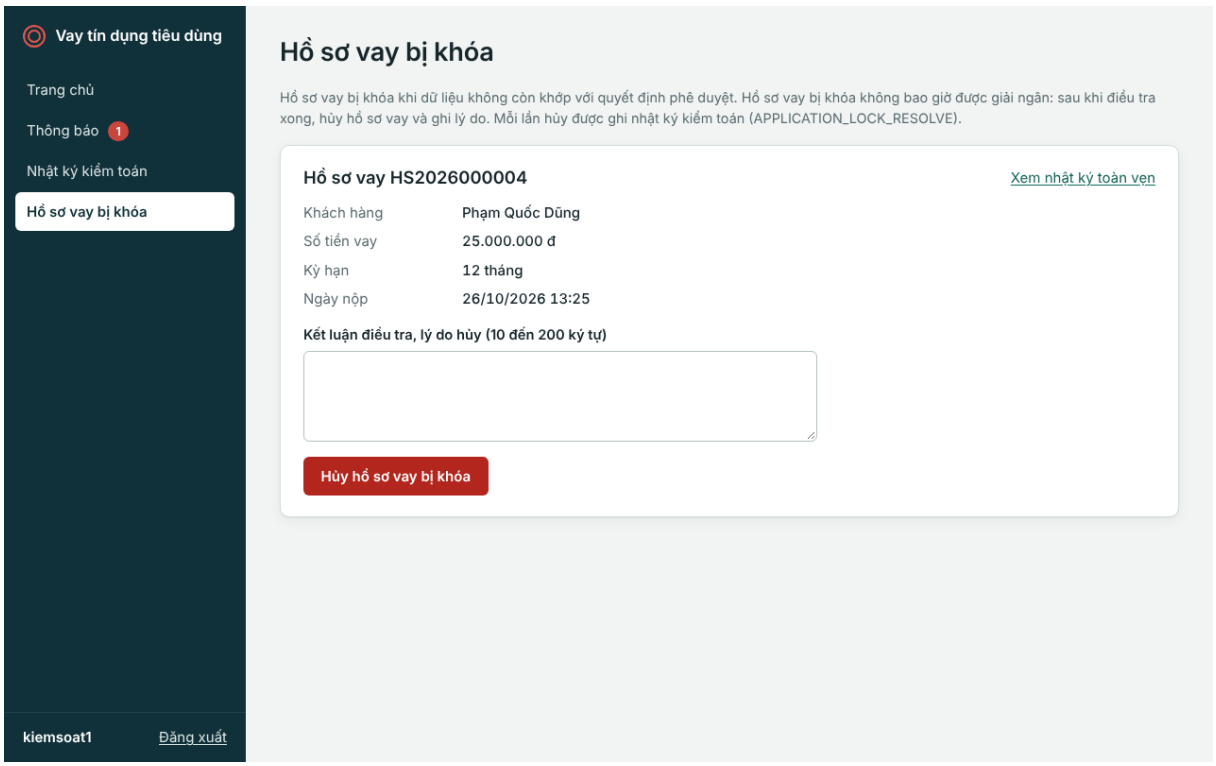
Hình 3.23. Bản mã tài khoản nhận bị tráo: “Toàn vẹn: không khớp”, hồ sơ HS2026000004 bị khóa, không thể giải ngân



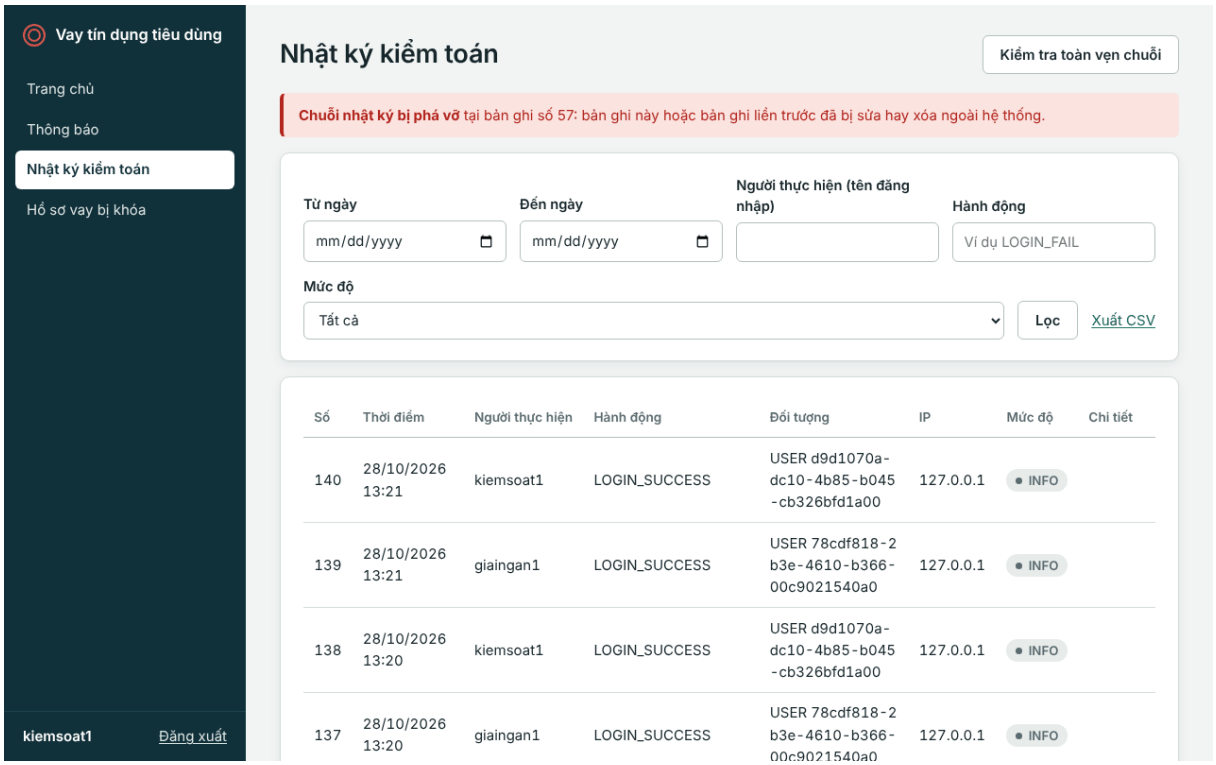
Hình 3.24. Nhật ký ghi `INTEGRITY_FAIL` mức `CRITICAL` kèm người thao tác, đối tượng và IP



Hình 3.25. Kiểm soát viên nhận thông báo cần điều tra



Hình 3.26. Hồ sơ vay bị khóa chờ Kiểm soát viên điều tra và xử lý



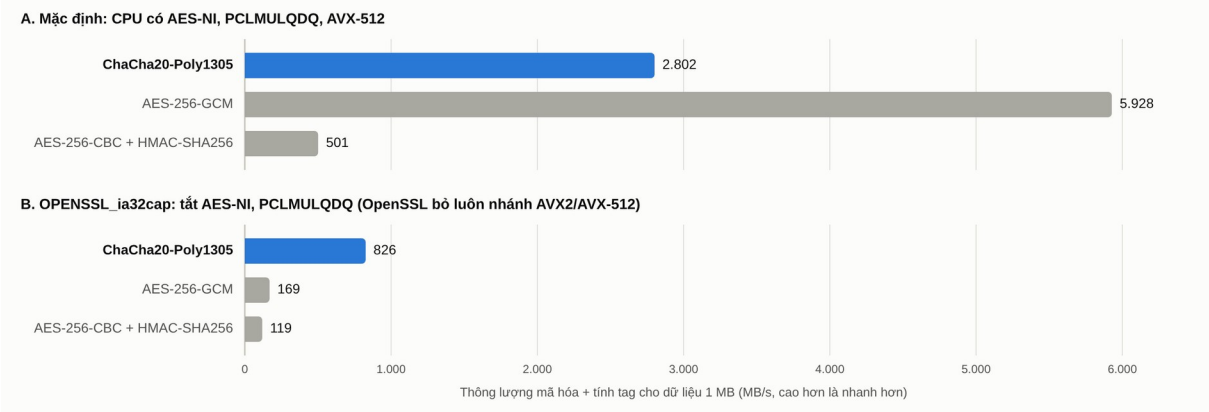
Hình 3.27. Sửa trực tiếp bản ghi nhật ký số 57 bằng tài khoản root: kiểm tra chuỗi chỉ đúng vị trí bị sửa

d) Đo hiệu năng. Script `scripts/do_hieu_nang.py` đo trên máy ảo 4 nhân Intel Xeon 2,1 GHz, Python 3.12, `cryptography` 50.0.1 (OpenSSL 4.0.2). Mỗi giá trị là trung vị của 7 lần đo, mỗi lần chạy lặp ít nhất 0,2 giây. Hai chế độ:

- **Chế độ A (mặc định):** CPU có AES-NI, PCLMULQDQ, AVX2, AVX-512 – điển hình cho máy chủ hiện đại.
- **Chế độ B:** đặt `OPENSSL_ia32cap="~0x200000200000000"` để tắt AES-NI và PCLMULQDQ. Nhóm quan sát thấy ở chế độ này ChaCha20 (vốn không dùng AES-NI) cũng chậm đi khoảng 3 lần, tức OpenSSL đồng thời bỏ các nhánh mã AVX2/AVX-512. Vì vậy chế độ B mô phỏng một CPU không có tăng tốc mật mã và SIMD đời mới, gần với thiết bị yếu hoặc máy đời cũ.

Bảng 3.7. Thông lượng mã hóa theo kích thước dữ liệu (MB/s, cao hơn là tốt hơn)

Thuật toán	64 B	1 KB	16 KB	1 MB	5 MB
Chế độ A – có AES-NI					
ChaCha20-Poly1305	22,8	346,6	2.015,1	2.801,8	2.344,5
AES-256-GCM	25,0	413,3	3.250,0	5.928,1	5.463,6
AES-256-CBC + HMAC-SHA256	6,7	105,3	465,1	501,3	265,2
Chế độ B – không AES-NI, không AVX2/AVX-512					
ChaCha20-Poly1305	22,2	186,8	606,9	826,1	735,0
AES-256-GCM	13,9	72,6	149,1	169,1	183,1
AES-256-CBC + HMAC-SHA256	3,1	33,8	94,9	119,1	82,0



Hình 3.28. Thông lượng mã hóa dữ liệu 1 MB ở hai chế độ CPU (số liệu Bảng 3.7)

Bảng 3.8. Thời gian mã hóa, giải mã một trường (micro giây, chế độ A)

Dữ liệu	Bản lưu	ChaCha20-Poly1305 mã hóa / giải mã	AES-256-GCM mã hóa / giải mã	`FieldCipher` đầy đủ (định dạng 2) mã hóa / giải mã
CCCD (12 B)	41 B	2,25 / 2,00	1,12 / 0,60	3,92 / 3,18
Tài khoản nhận (14 B)	43 B	2,53 / 2,12	1,55 / 0,95	5,71 / 4,39
Khóa TOTP (32 B)	61 B	3,28 / 2,15	1,56 / 0,79	4,23 / 4,69
Dữ liệu chờ OTP (~300 B)	327 B	3,44 / 1,98	1,28 / 0,59	4,01 / 2,70

Ở chế độ B, cùng các phép đo cho ChaCha20-Poly1305 khoảng 1,0–3,3 µs và AES-256-GCM khoảng 1,6–3,3 µs (số liệu đầy đủ ở Phụ lục B). Cột `FieldCipher` gồm cả chi phí của Python: đổi chuỗi sang byte, sinh nonce, ghép byte định dạng và dựng AAD. Chênh lệch dưới khoảng 1–2 µs giữa các lần chạy nằm trong sai số đo của máy ảo.

Bảng 3.9. Thời gian các cơ chế hỗ trợ (chế độ A)

Cơ chế	Thời gian	Ghi chú
Argon2id băm một mật khẩu	≈ 79 ms	Cố ý chậm và tốn 64 MiB để chống dò mật khẩu
Argon2id kiểm tra mật khẩu	≈ 72 ms	Chạy một lần mỗi lượt đăng nhập
HMAC-SHA256 blind index (CCCD)	≈ 1,9 µs	
Kiểm tra mã TOTP (3 bước thời gian)	≈ 16 µs	Gồm giải mã base32 và 3 lần HMAC-SHA1
Khởi tạo `FieldCipher` (gồm HKDF)	≈ 6,7 µs	Mỗi yêu cầu một lần
Kiểm tra chuỗi băm 10.000 bản ghi nhật ký	≈ 60 ms	Khoảng 6 µs mỗi bản ghi

Nhận xét:

1. **Trên máy chủ có AES-NI**, AES-256-GCM nhanh hơn ChaCha20-Poly1305 khoảng 2,1 lần với dữ liệu lớn (5.928 so với 2.802 MB/s ở 1 MB). Kết quả này đúng với lý thuyết ở mục 2.2.5: AES-GCM được tăng tốc bằng phần cứng chuyên dụng.
2. **Khi không có tăng tốc phần cứng**, tình thế đảo ngược: ChaCha20-Poly1305 nhanh hơn AES-256-GCM khoảng **4,9 lần** ở 1 MB (826 so với 169 MB/s) và

3,4–4 lần ở 1–16 KB. AES-GCM mất tới 97% thông lượng (từ 5.928 xuống 169 MB/s), còn ChaCha20-Poly1305 vẫn giữ ở mức hàng trăm MB/s. Đây chính là lý do Google, WireGuard và TLS 1.3 chọn ChaCha20-Poly1305 cho thiết bị di động và thiết bị không có AES-NI.

3. **Ghép thủ công AES-CBC + HMAC** chậm nhất ở cả hai chế độ (cần hai lượt qua dữ liệu và đệm PKCS#7). Việc dùng một thuật toán AEAD vừa nhanh hơn vừa tránh được các lỗi ghép sai thứ tự như MAC-then-encrypt và padding oracle.
4. **Với bài toán của hệ thống** (trường vài chục byte), mã hóa hay giải mã một trường chỉ mất khoảng 2–6 µs ở cả hai thuật toán và cả hai chế độ. Một trang thẩm định giải mã ba trường tốn dưới 20 µs, nhỏ hơn hàng nghìn lần so với thời gian một lượt Argon2id (≈ 70 ms) hay một truy vấn CSDL. Đổi sang ChaCha20-Poly1305 không ảnh hưởng tới thời gian phản hồi, trong khi tăng độ an toàn trên phần cứng không có AES-NI.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Các kết quả đã đạt được

Bảng 4.1. Đối chiếu kết quả với mục tiêu

Mục tiêu (mục 1.3)	Kết quả	Minh chứng
Hệ thống hóa nguyên tắc an toàn, mô hình đe dọa	Ánh xạ CIA, xác thực, STRIDE vào từng cơ chế của hệ thống	Bảng 2.1, 2.2
Hiểu sâu ChaCha20-Poly1305	Trình bày ma trận trạng thái, quarter-round, hàm khối, Poly1305, cấu trúc AEAD, quy trình giải mã; tự cài đặt lại	Mục 2.2; `chacha20_minh_hoa.py`
Kiểm chứng tính đúng	Đạt toàn bộ vector thử RFC 8439; 50/50 bộ ngẫu nhiên trùng thư viện	Bảng 3.5, Phụ lục D
Đánh giá an toàn, hiệu năng	Phân tích biên an toàn, giới hạn nonce, kênh kề; đo hai chế độ CPU	Mục 2.2.5, Bảng 3.7 – 3.9
Đưa thuật toán vào hệ thống thật	`FieldCipher` định dạng 2 với HKDF, AAD ngữ cảnh; tương thích dữ liệu AES-GCM cũ, không cần migration; ADR 0005	Mục 3.4; 528/528 bài kiểm thử đạt
Thử nghiệm tấn công	6 kịch bản đều bị chặn hoặc phát hiện; phát hiện và sửa một lỗi xử lý giả mạo ở màn hình giải ngân	Bảng 3.6, Hình 3.23 – 3.27
Dữ liệu truyền tải	TLS 1.3 thương lượng được `TLS_CHACHA20_POLY1305_SHA256` với X25519	Mục 3.4.2

Tóm lại, đề tài đã đi trọn vòng từ lý thuyết đến sản phẩm: hiểu thuật toán đến mức tự cài đặt lại được, rồi đưa thuật toán chuẩn vào bảo vệ dữ liệu cá nhân của một hệ thống xét duyệt vay đang chạy, có số liệu kiểm chứng và thử nghiệm tấn công.

4.2. Hạn chế của đề tài

- **Chưa thuộc danh mục FIPS.** ChaCha20-Poly1305 không nằm trong danh mục thuật toán được NIST phê duyệt cho FIPS 140. Tổ chức nào bắt buộc tuân thủ FIPS phải giữ AES-256-GCM. Thiết kế phiên bản định dạng của hệ thống cho phép chuyển lại dễ dàng.
- **Giới hạn của nonce ngẫu nhiên 96 bit.** An toàn tới khoảng 2^{32} lần mã hóa trên một khóa. Quy mô hiện tại còn rất xa ngưỡng này, nhưng hệ thống chưa có bộ

đếm số lần mã hóa để cảnh báo khi cần xoay khóa. Thư viện `cryptography` chưa có XChaCha20-Poly1305.

- **Dữ liệu định dạng cũ chưa được mã hóa lại hàng loạt.** Bản ghi AES-GCM chỉ chuyển sang ChaCha20-Poly1305 khi trường đó được ghi lại.
- **Khóa nằm trong biến môi trường của máy chủ ứng dụng.** Kẻ chiếm được máy chủ ứng dụng sẽ đọc được khóa; chưa dùng HSM hoặc dịch vụ quản lý khóa (KMS) và mã hóa phong bì (envelope encryption).
- **Tệp giấy tờ (ảnh CCCD, sao kê lương) chưa được mã hóa trên đĩa**, mới được bảo vệ bằng phân quyền, lưu ngoài thư mục web và watermark khi xem.
- **Dữ liệu truyền tải mới ở mức thử nghiệm.** Chưa đưa Nginx TLS 1.3 vào `docker-compose.yml`; kết nối ứng dụng – MySQL chưa bật TLS.
- **HMAC không chống chối bỏ trước bên thứ ba.** Máy chủ giữ khóa nên về mặt mật mã cũng tạo được mã băm hợp lệ.
- **Giới hạn tần suất lưu trong bộ nhớ của một tiến trình**, chưa dùng chung khi chạy nhiều tiến trình.
- **Số liệu hiệu năng đo trên một máy ảo**; chế độ B là mô phỏng bằng cờ của OpenSSL chứ không phải thiết bị thật.

4.3. Hướng phát triển trong tương lai

1. **Mã hóa phong bì với KMS/Vault:** mỗi bản ghi hoặc mỗi nhóm dữ liệu có khóa dữ liệu riêng, khóa này được khóa chủ trong HSM/KMS bọc lại; xoay khóa chủ mà không phải mã hóa lại dữ liệu.
2. **XChaCha20-Poly1305 hoặc AES-GCM-SIV** (chống lạm dụng nonce) khi thư viện hỗ trợ, để bỏ giới hạn 2^{32} của nonce ngẫu nhiên.
3. **Tác vụ mã hóa lại** chạy nền chuyển toàn bộ bản ghi định dạng 1 sang định dạng 2, có thống kê tiến độ, sau đó bỏ hẳn khóa AES-GCM cũ.
4. **Mã hóa tệp giấy tờ** bằng ChaCha20-Poly1305 theo luồng (chia đoạn, mỗi đoạn một nonce dẫn xuất, AAD chứa số thứ tự đoạn và cờ đoạn cuối).
5. **Chữ ký số cho quyết định phê duyệt:** mỗi quản lý có cặp khóa Ed25519 (hoặc ML-DSA hậu lượng tử, FIPS 204), khóa riêng nằm trên thiết bị của người duyệt, để có chống chối bỏ thật sự.
6. **Hoàn thiện dữ liệu truyền tải:** Nginx với TLS 1.3, HSTS, trao đổi khóa lai hậu lượng tử X25519MLKEM768 và bật TLS cho kết nối tới MySQL.
7. **Giới hạn tần suất dùng chung qua Redis** và tích hợp quét bảo mật tự động (SAST/DAST) vào CI.

TÀI LIỆU THAM KHẢO

[1] Y. Nir, A. Langley, “ChaCha20 and Poly1305 for IETF Protocols”, RFC 8439, IRTF, 6/2018.

[2] Y. Nir, A. Langley, “ChaCha20 and Poly1305 for IETF Protocols”, RFC 7539, IRTF, 5/2015 (đã được RFC 8439 thay thế).

[3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed., Pearson, 2020.

[4] D. J. Bernstein, “ChaCha, a variant of Salsa20”, Workshop Record of SASC 2008: The State of the Art of Stream Ciphers, 2008.

[5] D. J. Bernstein, “The Salsa20 family of stream ciphers”, trong *New Stream Cipher Designs: The eSTREAM Finalists*, LNCS 4986, Springer, 2008.

[6] D. J. Bernstein, “The Poly1305-AES message-authentication code”, Fast Software Encryption (FSE 2005), LNCS 3557, Springer, 2005.

[7] G. Procter, “A Security Analysis of the Composition of ChaCha20 and Poly1305”, IACR Cryptology ePrint Archive, 2014/613, 2014.

[8] J.-P. Aumasson, S. Fischer, S. Khazaei, W. Meier, C. Rechberger, “New Features of Latin Dances: Analysis of Salsa, ChaCha, and Rumba”, Fast Software Encryption (FSE 2008), LNCS 5086, Springer, 2008.

[9] NIST, “Advanced Encryption Standard (AES)”, FIPS PUB 197, 2001 (cập nhật 2023).

[10] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3”, RFC 8446, IETF, 8/2018.

[11] Google Security Blog, “Speeding up and strengthening HTTPS connections for Chrome on Android”, 4/2014.

[12] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel”, Network and Distributed System Security Symposium (NDSS), 2017.

[13] H. Krawczyk, M. Bellare, R. Canetti, “HMAC: Keyed-Hashing for Message Authentication”, RFC 2104, IETF, 1997.

[14] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC”, NIST SP 800-38D, 2007.

[15] D. J. Bernstein, “Cache-timing attacks on AES”, báo cáo kỹ thuật, 2005.

[16] D. A. Osvik, A. Shamir, E. Tromer, “Cache Attacks and Countermeasures: the Case of AES”, CT-RSA 2006, LNCS 3860, Springer, 2006.

[17] H. Krawczyk, P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF)”, RFC 5869, IETF, 2010.

[18] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications”, RFC 9106, IRTF, 2021.

[19] D. M’Raihi và cộng sự, “HOTP: An HMAC-Based One-Time Password Algorithm”, RFC 4226, IETF, 2005.

[20] D. M’Raihi, S. Machani, M. Pei, J. Rydell, “TOTP: Time-Based One-Time Password Algorithm”, RFC 6238, IETF, 2011.

[21] Chính phủ nước CHXHCN Việt Nam, Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân, 2023.

[22] Quốc hội nước CHXHCN Việt Nam, Luật Bảo vệ dữ liệu cá nhân, 2025.

[23] Ngân hàng Nhà nước Việt Nam, Thông tư 09/2020/TT-NHNN quy định về an toàn hệ thống thông tin trong hoạt động ngân hàng, 2020.

[24] IBM Security, *Cost of a Data Breach Report 2024*, 2024.

[25] OWASP Foundation, *Application Security Verification Standard (ASVS) 4.0.3* và *OWASP Top 10:2021*.

[26] Python Cryptographic Authority, tài liệu thư viện `cryptography` – mục *Authenticated encryption* và *Key derivation functions*, https://cryptography.io.

[27] M. Thomson, S. Turner, “Using TLS to Secure QUIC”, RFC 9001, IETF, 2021.

[28] NIST, “Digital Identity Guidelines: Authentication and Lifecycle Management”, SP 800-63B.

PHỤ LỤC

Phụ lục A – Mã nguồn chính

Tệp	Nội dung
`src/loan_system/security/crypto.py`	`FieldCipher`: ChaCha20-Poly1305, HKDF, phiên bản định dạng, blind index
`src/loan_system/security/integrity.py`	Vòng khóa HMAC-SHA256 có phiên bản cho snapshot phê duyệt
`src/loan_system/services/integrity_service.py`	Ký và đối chiếu snapshot (SR08, SUC02)
`src/loan_system/domain/audit.py`	Chuỗi băm nhật ký kiểm toán
`src/loan_system/security/secrets.py`	Argon2id, HMAC token phiên và OTP
`src/loan_system/security/totp.py`	TOTP theo RFC 6238
`src/loan_system/services/auth_service.py`	Đăng nhập, phiên, TOTP, đăng ký khóa TOTP
`tests/domain/test_field_encryption.py`	Kiểm thử mã hóa mức trường
`docs/adr/0005-ma-hoa-truong-bang-chacha20-poly1305.md`	Quyết định kiến trúc đổi thuật toán
`docs/bao-cao-atbmhttt/scripts/chacha20_minh_hoa.py`	Cài đặt minh họa ChaCha20-Poly1305 bằng Python thuần
`docs/bao-cao-atbmhttt/scripts/do_hieu_nang.py`	Script đo hiệu năng

Hàm khối ChaCha20 và Poly1305 trong bản cài đặt minh họa (rút gọn):

```
def quarter_round(s, a, b, c, d):
    s[a] = (s[a] + s[b]) & MASK32; s[d] = rotl32(s[d] ^ s[a], 16)
    s[c] = (s[c] + s[d]) & MASK32; s[b] = rotl32(s[b] ^ s[c], 12)
    s[a] = (s[a] + s[b]) & MASK32; s[d] = rotl32(s[d] ^ s[a], 8)
    s[c] = (s[c] + s[d]) & MASK32; s[b] = rotl32(s[b] ^ s[c], 7)

def chacha20_block(key, counter, nonce):
    state = [*CONSTANTS, *struct.unpack("<8I", key), counter,
*struct.unpack("<3I", nonce)]
    working = state[:]
    for _ in range(10):                          # 10 vòng kép = 20 vòng
        quarter_round(working, 0, 4, 8, 12); quarter_round(working, 1, 5,
9, 13)
        quarter_round(working, 2, 6, 10, 14); quarter_round(working, 3, 7,
11, 15)
        quarter_round(working, 0, 5, 10, 15); quarter_round(working, 1, 6,
11, 12)
        quarter_round(working, 2, 7, 8, 13); quarter_round(working, 3, 4,
9, 14)
```

```
    return struct.pack("<16I", *[(w + s) & MASK32 for w, s in zip(working,
state)])

def poly1305_mac(key, message):
    r = int.from_bytes(key[:16], "little") &
0x0FFFFFFC0FFFFFFC0FFFFFFC0FFFFFFF
    s = int.from_bytes(key[16:], "little")
    acc = 0
    for i in range(0, len(message), 16):
        n = int.from_bytes(message[i : i + 16] + b"\x01", "little")
        acc = (acc + n) * r % ((1 << 130) - 5)
    return ((acc + s) & ((1 << 128) - 1)).to_bytes(16, "little")
```

Chuỗi băm nhật ký (`src/loan_system/domain/audit.py`):

```
def compute_hash(prev_hash: str, seq: int, content: AuditContent) -> str:
    fields = {"seq": seq, "actor_id": content.actor_id, "action":
content.action,
              "target_type": content.target_type, "target_id":
content.target_id,
              "ip_address": content.ip_address, "level": content.level,
              "created_at": content.created_at.isoformat()}
    if content.detail is not None:
        fields["detail"] = content.detail
    canonical = json.dumps(fields, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256((prev_hash + canonical).encode("utf-
8")).hexdigest()
```

Phụ lục B – Bảng số liệu thử nghiệm

Số liệu gốc dạng JSON nằm ở `so-lieu/hieu-nang-co-aes-ni.json` và `so-lieu/hieu-nang-khong-aes-ni.json`, mỗi tệp ghi kèm CPU, phiên bản Python, `cryptography`, OpenSSL và thời điểm đo.

Thời gian một trường ở chế độ B (micro giây)

Dữ liệu	ChaCha20-Poly1305 mã hóa / giải mã	AES-256-GCM mã hóa / giải mã	`FieldCipher` mã hóa / giải mã
CCCD (12 B)	2,25 / 1,04	2,09 / 1,60	2,71 / 2,47
Tài khoản nhận (14 B)	3,33 / 1,66	3,10 / 2,11	3,95 / 3,01
Khóa TOTP (32 B)	2,47 / 1,47	2,22 / 1,85	3,42 / 2,60
Dữ liệu chờ OTP (~300 B)	2,60 / 1,40	3,33 / 2,82	2,22 / 1,70

Cơ chế hỗ trợ ở chế độ B: Argon2id băm 64,8 ms, kiểm tra 64,4 ms; blind index 2,2 µs; kiểm tra TOTP 11,8 µs; khởi tạo `FieldCipher` 8,2 µs; kiểm tra chuỗi 10.000 bản ghi 65,7 ms. Các cơ chế này không dùng AES nên gần như không đổi giữa hai chế độ.

Kiểm thử tự động: 528 bài đạt, 0 bài lỗi, chạy trên MySQL 8.4.13 trong khoảng 10 phút 10 giây; `mypy --strict` trên 147 tệp không có lỗi.

Phụ lục C – Hướng dẫn cài đặt

1. Cài Python 3.12, uv và MySQL 8.0.16 trở lên (hoặc Docker).
2. Cài thư viện và tạo tệp cấu hình:

```
uv sync
cp .env.example .env
```

3. Sinh bốn khóa bí mật khác nhau và điền vào `.env` (`SESSION_SECRET`, `DATA_ENC_KEY`, `HMAC_INTEGRITY_KEY`, `BLIND_INDEX_KEY`), cùng thông tin kết nối MySQL:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

4. Tạo database, tài khoản ứng dụng quyền tối thiểu, chạy migration và nạp dữ liệu mẫu:

```
uv run python -m loan_system.create_database
uv run python -m loan_system.seed_demo
```

5. Chạy ứng dụng tại http://127.0.0.1:8000/app. Tài khoản demo và khóa TOTP nằm trong `var/demo_accounts.txt`:

```
uv run uvicorn loan_system.app:app --reload
```

6. Chạy thử TLS 1.3 với ChaCha20-Poly1305 (chứng chỉ tự ký, chỉ để thử nghiệm):

```
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -
keyout tls.key -out tls.crt -days 7 -subj "/CN=localhost"
uv run uvicorn loan_system.app:app --port 8443 --ssl-keyfile tls.key --ssl-
certfile tls.crt
openssl s_client -connect localhost:8443 -tls1_3 -ciphersuites
TLS_CHACHA20_POLY1305_SHA256
```

7. Chạy script minh họa thuật toán và đo hiệu năng:

```
uv run python docs/bao-cao-atbmhttt/scripts/chacha20_minh_hoa.py
uv run python docs/bao-cao-atbmhttt/scripts/do_hieu_nang.py --json ket-
qua.json
OPENSSL_ia32cap="~0x200000200000000" uv run python
docs/bao-cao-atbmhttt/scripts/do_hieu_nang.py
```

8. Chạy kiểm thử (test API cần MySQL, đổi địa chỉ bằng `LOAN_TEST_MYSQL_URL`):

```
uv run pytest
```

Phụ lục D – Kết quả chạy script minh họa ChaCha20-Poly1305

```
== 1. Quarter round, RFC 8439 section 2.1.1 ==
in : a=11111111 b=01020304 c=9b8d6f43 d=01234567
out: a=ea2a92f4 b=cb1cf8ce c=4581472e d=5881c4bb
[PASS] quarter round: ea2a92f4cb1cf8ce4581472e5881c4bb

== 2. Block function, RFC 8439 section 2.3.2 ==
initial state (constants | key | counter | nonce):
    61707865 3320646e 79622d32 6b206574
    03020100 07060504 0b0a0908 0f0e0d0c
    13121110 17161514 1b1a1918 1f1e1d1c
    00000001 09000000 4a000000 00000000
after 20 rounds:
    837778ab e238d763 a67ae21e 5950bb2f
```

```
    c4f2d0c7 fc62bb2f 8fa018fc 3f5ec7b7
    335271c2 f29489f3 eabda8fc 82e46ebd
    d19c12b4 b04e16de 9e83d0cb 4e3c50a2
output = after 20 rounds + initial state:
    e4e7f110 15593bd1 1fdd0f50 c47120a3
    c7f4d1c7 0368c033 9aaa2204 4e6cd4c3
    466482d2 09aa9f07 05d7c214 a2028bd9
    d19c12b5 b94e16de e883d0cb 4e3c50a2
[PASS] serialized block:
10f1e7e4d13b5915500fdd1fa32071c4c7d1f4c733c068030422aa9ac3d46c4ed2826446079
faa0914c2d705d98b02a2b5129cd1de164eb9cbd083e8a2503c4e

== 3. Encryption, RFC 8439 section 2.4.2 ==
[PASS] first 16 bytes of ciphertext: 6e2e359a2568f98041ba0728dd0d6981
[PASS] all 114 bytes equal cryptography's ChaCha20; decrypt(encrypt(m)) ==
m

== 4. Poly1305, RFC 8439 section 2.5.2 ==
r (clamped) = 0806d5400e52447c036d555408bed685
s           = 1bf54941aff6bf4afdb20dfb8a800301
block 1: n = 16f4620636968706172676f7470797243  acc =
2c88c77849d64ae9147ddeb88e69c83fc
block 2: n = 16f72472068637261657365522 06d7572  acc =
2d8adaf23b0337fa7cccfb4ea344b30de
block 3: n = 00000000000000000000000000017075  acc =
28d31b7caff946c77c8844335369d03a7
[PASS] Poly1305 tag: a8061dc1305136c6c22b8baf0c0127a9

== 5. AEAD ChaCha20-Poly1305, RFC 8439 section 2.8.2 ==
[PASS] first 16 bytes of ciphertext: d31a8d34648e60db7b86afbc53ef7ec2
[PASS] tag: 1ae10b594f09e26a7e902ecbd0600691
[PASS] identical to cryptography.ChaCha20Poly1305

== 6. Own AEAD vs cryptography on random inputs ==
[PASS] 50/50 random cases identical to cryptography.ChaCha20Poly1305

== 7. A value encrypted by the system's FieldCipher, opened by this code ==
stored (41 bytes):
028faba8e1c0cfdf9eb81bdcc88c40565e2713596c2b537dee02d5dac2646adb982ccc2fe9a
81df8b3
  format=02 nonce=8faba8e1c0cfdf9eb81bdcc8
ciphertext=8c40565e2713596c2b537dee tag=02d5dac2646adb982ccc2fe9a81df8b3
[PASS] decrypted: 079095001234
[PASS] flip 1 bit of ciphertext: rejected, tag mismatch
[PASS] copy to another customer: rejected, tag mismatch
```